TILERA®

MULTICORE DEVELOPMENT ENVIRONMENT SYSTEM PROGRAMMER'S GUIDE

DOCUMENT RELEASE 3.60
DOC. NO. UG509
JUNE 2014
TILERA CORPORATION

Publishing Information:

Document Number	UG509
Document Release	3.60
Date	20 June 2014

Contact Information:

Tilera Corporation
Information info@tilera.com **Web Site http://www.tilera.com**

CONTENTS

PREFACE

About This Guide

This guide describes how to customize the system software environment for the TILE-Gx™ processor.

This guide is relative to the Multicore Development Environment™ (MDE) Release 4.3.

Intended Audience

This guide is intended for use by software developers and administrators at the system level who are responsible for configuration and customization of the operating system, device drivers, and other software that runs in kernel space on the Tile Processor.

Note: This guide assumes that you are familiar with *Programming the TILE-Gx Processor* (UG505).

Changes from the Previous Release

The major changes in this guide from MDE 4.2 are:

New sections

- Section 1.4.1 “Creating a TILE-Gx72 Simulator Image” on page 5.
- Section 1.8.6 “Booting with a Root Filesystem Available over NFS” on page 11.
- Added a description of the hypervisor statistics gathering feature. See Section 2.5 “Gathering Hypervisor Statistics” on page 25.
- The instructions about building Linux from source files now reflects the upgrade to the Linux 3.10 kernel. See Section 3.1 “Building Linux from Source” on page 35.
- Changes have been made to Chapter 3: Linux Interfaces and Customization to reflect the importance of the `initramfs` booting scheme. See in particular, for example, Section 3.4 “Controlling the Linux Initramfs Boot Process” on page 55 and Section 3.4.2 “Adding User Files to the Initramfs Boot Image” on page 58.
- Changes have been made to Chapter 3: Linux Interfaces and Customization to describe the MiCA-related `sysfs` files and to explain resetting the compression shim. See in particular Section 3.8.4 “Resetting MiCA Compression Shims Using sysfs” on page 62.
- Changes have been made to Table 3-1 to describe the `hardlockup` and `lockup_dumpall` linux boot options.
- Changes to Chapter 4: Making a Bootrom to replace `tile-mkboot` commands with the more recent `tile-monitor` equivalents. See in particular, for example, Section 4.5.1 “Creating the Default Bootrom for a TILEmpower-Gx Appliance” on page 67 and Section 4.5.2 “Creating a Bootrom for both Hypervisor and Linux Images” on page 67.

- Changes have been made to Chapter 6: I/O Devices and Drivers to reflect changes to the x86 Linux Host PCIe driver. See in particular, for example, Section 6.3.3 "Supporting Multiple PCIe Endpoint Ports" on page 76, and Section 6.3.7 "Configuring the Host Ring Buffer Size" on page 78.
- Chapter 6: I/O Devices and Drivers adds support for both shared and dedicated instances in the memprof driver. See in particular, for example Section 6.8 "memprof Driver" on page 87.

Document Organization

This guide is organized as follows:

- Chapter 1: System Startup Boot Process

  Describes how to use MDE system software; explains important concepts with regard to tiles, describes the architectures of the Tilera® hypervisor and SMP Linux, and gives an overview of I/O device drivers for the Tile Processor and their configuration.

- Chapter 2: Hypervisor Customization

  Provides specific information on hypervisor configuration, including commands, subcommands, and options that can be specified in the hypervisor configuration file.

- Chapter 3: Linux Interfaces and Customization

  Describes how to customize the Linux kernel to adapt it to specific requirements.

- Chapter 4: Making a Bootrom

  Describes how to build a bootrom using various methods.

- Chapter 5: Using the mboot Boot Loader

  Describes how to use the boot loader, `mboot`, which facilitates booting from a hard disk, the network, or compact flash.

- Chapter 6: I/O Devices and Drivers

  Describes each of the supported I/O devices and their drivers in greater detail.

- Chapter 7: Developing Hypervisor Drivers

  Describes the hypervisor drivers, which provide access to hardware by means of GXIO.

- Chapter 8: Using the Bare Metal Environment (BME)

  Describes Tilera's Bare Metal Environment (BME), which lets advanced users run applications that require direct access to the hardware.

The guide also contains two appendixes, Appendix A— Hypervisor Console Escape Commands and Appendix B— mboot CLI Commands, and an Index.

MDE Documentation

Note: `TILERA_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is required by many Tilera tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

Tilera MDE Document Index

The Tilera MDE Document Index provides links to online copies of the MDE documentation, in both HTML and PDF formats:

```
$TILERA_ROOT/doc/index.html
```

Location of PDF Files in the Kit

The PDF versions of MDE documents can be found at this location:

```
$TILERA_ROOT/doc/pdf/
```

Man Pages and Info Pages

To view a man page for an x86 tool, run:

```
$ tile-man toolname
```

To view a man page for a Tile tool, run this on a TILE system:

```
$ man toolname
```

To view an info page for a Tile tool, run this on a TILE system:

```
$ info toolname
```

IDE Online Help

The IDE has its own Tilera IDE Online Documentation, which you can access when you use the IDE.

To get to the online help:

- Start `tile-eclipse`.

Select the menu command Help → Help Contents.

- When the online help browser is displayed, select the document Tilera IDE Online Documentation in the left navigation pane, if it is not already selected.

Technical or Customer Support

You can reach Tilera customer support in either of the following ways:

- Visit the Tilera Web site at `http://www.tilera.com`.
- E-mail questions to `support@tilera.com`.

Notation Conventions

The following table lists notation conventions used in this guide.

Example	Description
{ }	Alternative required items in syntax descriptions appear within curly brackets. If the contents are separated by a vertical bar, you must choose one of the items.
[ ]	Optional items in syntax descriptions appear within brackets. If the contents are separated by a vertical bar, you must choose one of the items.
. . .	An ellipse in syntax specifies that the preceding element can be repeated an arbitrary number of times.

Example	Description
`$`	This is the system prompt for host-side commands.
`#`	This is the system prompt for all Tile-side commands and for those x86 host-side commands that must be run as `root`.
`command`	A command name appears in the Courier New font
`computer output`	Computer output appears in the Courier New font.
filename	Non-keyword placeholders appear in italics.
GUI item	A graphical user interface (GUI) item such as a button or menu name appears in red text with the Arial font.
new term	An important word or phrase appears in italics.
Tile Processor™	A processor in the TILE-Gx processor™ family.
`TILERA_ROOT`	`TILERA_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is expected by many Tilera tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

CHAPTER 1 SYSTEM STARTUP BOOT PROCESS

This chapter describes:

1.1 Tile Processor Boot Process

From the user's perspective, the Tile Processor™ boots from a single boot image. Internally, the boot process occurs in the following stages:

1. The boot process starts with a hardware reset of the Tile Processor chip. After hardware reset, each tile executes a small built-in bootstrap program. This program, termed the *level-0 bootstrap*, is responsible for copying an initial amount of boot code from an external device to the tile's L2 cache, and then executing it.

   Exactly how that boot code is provided depends on the specific device from which the Tile Processor is being booted. Some boot devices act as a slave, simply accepting data pushed into the chip from outside.

   Other boot devices act as a master, making specific requests to off-chip hardware to retrieve boot data. (For example, the Serial Peripheral Interface Read-Only Memory, SPI ROM or SROM, interface boots in this manner.)

   Whichever device is in use, the first portion of the boot image is transmitted from the device, over the iMesh™, to one specific tile on the chip that is near that I/O device. This first piece of boot code is termed the *level-1 bootstrap*.

2. The level-1 bootstrap does basic initialization of the chip. Its primary task is to replicate itself from the initial boot tile to all of the other tiles and to fully configure the iMesh network coordinates. It also configures the memory controllers and performs some preliminary tests of the memory. Finally, the level-1 bootstrap loads the second portion of the boot image, the hypervisor code, into memory, and then jumps to it.

3. The hypervisor finishes the initialization of the chip. It reads the third and final portion of the boot image into memory. This is the hypervisor file system, composed of the hypervisor configuration file and supervisor code. Based on the supplied configuration, the hypervisor initializes the on-chip devices, allocating dedicated tiles to manage them if needed. It then loads the supervisor (the OS) into memory and begins executing it on one tile.
4. Finally, the supervisor initializes its own internal data structures and performs any startup operations required to configure its allocated resources. In particular, it uses the hypervisor to enable virtual memory mapping, and eventually uses the hypervisor to enable any other tiles allocated to the supervisor. Once the supervisor is running, it executes an initialization program or script to bring up OS services or applications.

   The default Linux initialization script might start services such as the `shepherd` process, or the `tempmond` temperature monitoring daemon. It might also configure network interfaces. When initialization is complete, the supervisor is available to run user applications.

If the Tilera® boot loader, `mboot`, is the first supervisor to boot, it loads the second (and final) supervisor from an I/O device, such as a compact flash (CF) card or a SATA disk drive. The second supervisor then performs the normal initialization steps described above.

For details on the boot process and hypervisor startup, see the "Tilera Hypervisor Theory of Operation" technical note in the MDE Document Index.

Use the `tile-monitor` tool to interact with a Tile Processor™ from an x86 host. It provides a single, integrated interface for many common tasks, including creating bootroms, booting Tile devices, transferring files between the development host and the Tile environment, mounting host files into the Tile Linux filesystem, running applications, and using debugging and profiling tools. For complete details about using the tile-monitor tool, see Section 2.1 "What Is tile-monitor?" in the *Multicore Development Environment User Guide* (UG501).

1.2 Bootrom Files

Bootroms are used internally as part of the boot process, but the only time you will need to manage one directly is when booting from SROM.

Tilera ships various prebuilt images for booting the Tilera system, nested together in different combinations. Customers can build their own variations of these images (and sub images). This explains what the prebuilt images and possibilities are and references instructions for customizing them.

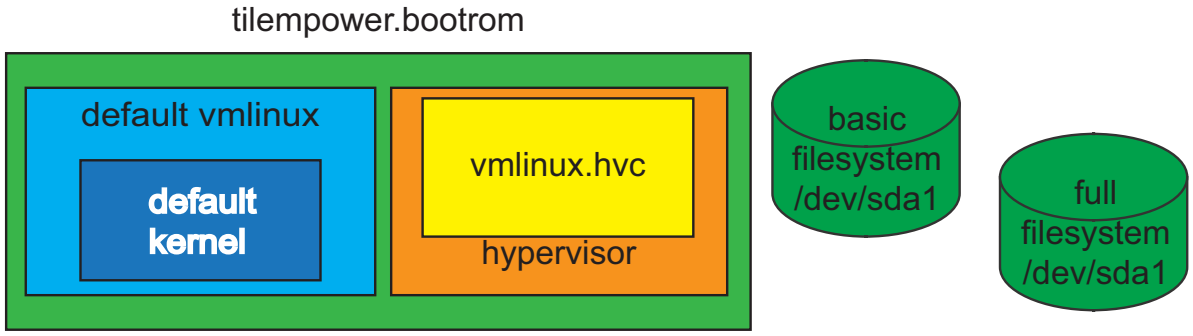

Figure 1-1: Components of the tilempower.bootrom File

To boot a Tilera system you must have hypervisor code, a hypervisor configuration file (a text file consumed by the hypervisor specifying options and drivers), a linux kernel and a filesystem. Sometimes, you use multiple kernels and/or filesystems to boot the system. The image that the system boots is called a bootrom file, as it is frequently burned into SPI-ROM, although you can also stream this file to the device over USB or other interfaces. At a minimum, the bootrom file must contain the hypervisor, the hypervisor configuration file, and the linux kernel. Sometimes the kernel's `vmlinux` file contains an initial RAM filesystem, sometimes the filesystem is a separate file within the bootrom, and sometimes it resides solely on external non-volatile storage.

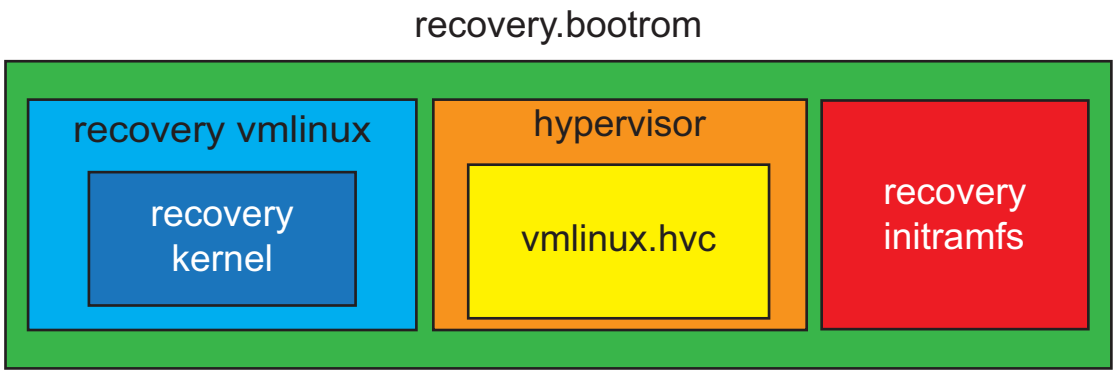

Figure 1-2: Components of the recovery.bootrom File

When booting a PCIe card such as the TILEncore-Gx36, `tile-monitor` creates a bootrom file dynamically, using a hypervisor configuration file to guide the process. See Section 1.3 "Hypervisor Configuration Files" on page 4.

The default configuration file is `$TILERA_ROOT/tile/etc/hvc/vmlinux.hvc`. The `tile-monitor` command queries the target device to ascertain various properties such as the existence of PCIe links and the chip's tile grid geometry. Variables are set to reflect these properties, and are passed to the `tile-mkboot` command to create the appropriate bootrom file. Read the `vmlinux.hvc` file for more detail. You can use the `tile-monitor`'s `--create-hvc` option to capture the particular configuration file that will be used.

The Linux images described in Section 1.4 "Simulator Image Files" on page 5 can be used by `mboot`, or to create bootroms using `tile-mkboot`. (See Chapter 4: Making a Bootrom.)

Use the `--bootrom-file` option to specify an alternate bootrom file, which you can create with `tile-mkboot`.

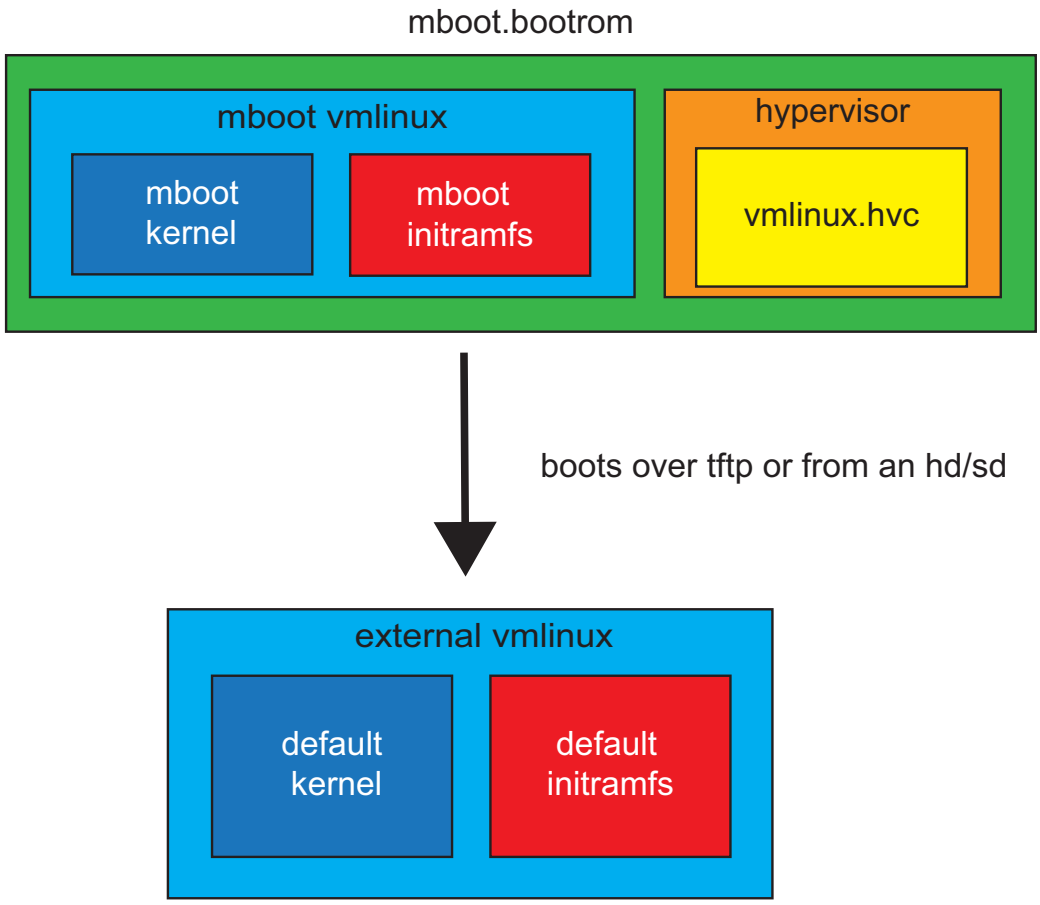

Figure 1-3: Components of the mboot.bootrom File

When you use an `mboot.bootrom`, you boot a minimal system including an `mboot` initramfs filesystem, and then obtain a more complete kernel and initramfs filesystem from an external vmlinux.

1.3 Hypervisor Configuration Files

Use `tile-monitor`'s `--hvc` option to choose an alternate hypervisor configuration file.

A few examples are shipped in `$TILERA_ROOT/tile/etc/hvc/`:

- `vmlinux.hvc` — used by default.
- `vmlinux-sim.hvc` — used to create the standard simulator image files.

Note: The `options.hvh` file is included by many of the other files and supports hypervisor configuration options.

1.4 Simulator Image Files

The following simulator image files are available in `$TILERA_ROOT/lib/boot/`:

- `1x1.image`, `2x2.image`, `4x4.image` — simulator images that configure the corresponding tile grids, and no I/O devices.
- `gx8036.image` — a simulator image that configures a tile grid and the memory controllers corresponding to an actual TILE-Gx8036™ processor.
- `gx8036-dataplane.image` — a simulator image that configures tile 0,0 for non-dataplane operations and all other tiles set aside for dataplane mode running ZOL.
- `gx8016.image` — a simulator image that configures a tile grid and the memory controllers corresponding to a TILE-Gx8016™ processor.

The `gx8016.image`, `gx8036.image`, and `gx8036-dataplane.image` files define the hardware resources of the fully-functioned Gx-8*xxx* family of parts. You can also use them to simulate the Gx-5*xxx* families, with the appropriate number of tiles.

1.4.1 Creating a TILE-Gx72 Simulator Image

To create a simulator image for a TILE-Gx72™ processor, use the following command:

```
$ tile-monitor --simulator --functional --config gx8072 --create-image
gx8072.image
[monitor] Creating 'gx8072.image'...
[monitor] Creating 'gx8072.image'... done.
```

To create a simulator image that configures tile 0,0 for non-dataplane operations and all other tiles set aside for dataplane mode running ZOL, replace the `--create-image` argument with the following arguments to the `tile-monitor` command:

```
--hvx "dataplane=1-71" --create-image gx8072-dataplane.image
```

1.5 Booting-Related Files

Some booting-related files, typically components that are included in boot images, are shipped in `$TILERA_ROOT/tile/boot/`:

- `classifier` — the default program for the mPIPE classifier
- `hv` — the hypervisor binary
- `hv_l1boot` — the hypervisor level-1 booter
- `hv_lhboot` — the hypervisor level-0 booter
- `pci_preboot.bin`— the hypervisor PCIe pre-booter
- `sromboot.bin` — the SROM booter
- `sromboot.dbg` — debug symbols for the SROM booter
- `vmlinux` — the standard Linux image
- `vmlinux_mboot` — the Linux image for mboot operations
- `initramfs.cpio.xz` — the initial RAM filesystem image for Linux

1.6 Booting with tile-monitor

The `tile-monitor` program manages communication between host-based development tools and processes running on the Tile Processor. You can use `tile-monitor` as a command-line tool on the development host. The `tile-monitor` communicates with a `shepherd` process running on the Tile Processor to control and get status from target processes.

`tile-monitor` is the recommended interface for booting the Tile Processor. After booting, you can use the `tile-monitor` command prompt to run processes on a hardware development platform or the Tilera simulator, running the Linux operating system.

For example, the following command boots a TILEncore-Gx36 card using a standard Linux bootrom file, runs the `ls` program, and exits:

```
$ $TILERA_ROOT/bin/tile-monitor -- ls
bin   dev   etc   init   lib  mnt  opt  proc  sbin  sys  tmp  usr  var
```

The following command does the same thing, but boots the simulator rather than the hardware:

```
$ $TILERA_ROOT/bin/tile-monitor --image 1x1 -- ls
bin   dev   etc   init   lib  mnt  opt  proc  sbin  sys  tmp  usr  var
```

You can use a boot image that captures the state of the simulator with a particular chip configuration, after booting the OS and starting the `shepherd` process, to save time. Use the `--image` or `--image-file` option to request the use of a particular boot image. Use the `--create-image` option to create a new boot image.

See "Tile Processor Monitor" (`tile-monitor.html`) in the "Target Platform Tools" section of the MDE Document Index or the man page at `tile-man tile-monitor`. (These are identical.)

1.7 Generating Bootrom Files and Rebooting Natively

You can also generate bootrom files and reboot natively (that is, by running `tile-monitor` on the Tile Processor, not on a host system).

The native version of `tile-monitor` supports a `--self` option to install a bootrom on, and reboot, the local system. It also supports the `--net` option in the same way as the host system version. Options to control `tile-sim` and cards accessed by means of PCIe or USB are not included in the native version.

For example:

```
$ tile-monitor --hvc xxx --hvd xxx --hvx xxx --self
```

To support the native `tile-monitor`, the native version of `tile-reboot` also has a `--self` option.

Tilera also provides native versions of `tile-mkboot` and `tile-mkrom`.

`tile-mkboot` automatically looks for `.hvc` files in the standard directory of `.hvc` files, predefines `CHIP_WIDTH` and `CHIP_HEIGHT` based on the current system when run natively, and has a `--linux` option that makes it get `vmlinux` (and in the future, perhaps other files) from a standard location.

For more information, see the manpages for `tile-monitor`, `tile-mkboot`, and `tile-reboot`, or see their HTML files in the MDE Document Index. The HTML files are identical to the man pages.

1.8 Additional Ways to Boot the TILE-Gx Processor

Section 1.6 "Booting with tile-monitor" on page 6 describes one way to boot the TILE-Gx Processor. This section describes additional ways.

1.8.1 Booting from PCIe

In a PCIe environment, it is recommended that you use `tile-monitor` to boot the Tile Processor, because you get the benefit of its consistency checks. For example, the `tile-monitor` method makes sure that the bootrom being used was built for the chip that is actually on your card.

However, it is possible to boot the card without using `tile-monitor`. The host-side driver includes support for booting and rebooting the Tile Processor using the special file `/dev/tilegxpci0/boot`. The host driver automatically resets the Tile Processor when `/dev/tilegxpci0/boot` is opened. Writing to the file causes the written data to be injected as the processor's bootrom. Thus, booting the chip is as simple as executing:

```
cat myfile.bootrom > /dev/tilegxpci0/boot
```

The host driver's boot mechanism blocks any stream communication between the Tile Processor and the host, and rebooting drains any in-flight data. Consequently, no data are transmitted using `/dev/tilegxpci0/`*N* in the interval between opening and closing the `/dev/tilegxpci0/boot` file.

1.8.2 Booting from USB

Booting by means of USB is similar to booting by means of PCIe, as explained in Section 1.8.1 "Booting from PCIe". As with PCIe, the use of `tile-monitor` is recommended, but in its absence, a USB target can be booted by writing the contents of a bootrom file to `/dev/tile-usb0/boot`.

1.8.3 Booting from SROM

You can boot the Tile Processor image from the on-board SROM device. This is primarily useful if there is no PCIe or USB connection, but can also be done even if the Tile Processor is connected to one of those interfaces. In general, the device must be configured to boot from SROM, for instance using jumpers or DIP switches. See the specific device's user guide for details.

The following example shows how to boot the `mboot` boot loader.

Depending on the device, it might be delivered factory-programmed with a default SROM image. If not, or to replace the SROM contents, use the following procedure. Note that the SROM contains space for two `mboot` bootroms; the following procedure overwrites both slots. To lock one of the slots as a backup, use the `sbim --lock` option. If the device does not have a PCIe connection, but is accessible over the network, use the `tile-monitor --network` option.

1. Boot the target card, then put a boot image in its SROM by issuing the following commands to the Tile Processor:

```
$ $TILERA_ROOT/bin/tile-monitor --rtc
Command: upload-tile /usr/lib/boot/mboot.bootrom
Command: upload-tile /boot/sromboot.bin
Command: run /usr/sbin/sbim --install-booter /boot/sromboot.bin --yes
Command: run /usr/sbin/sbim --install /usr/lib/boot/mboot.bootrom
Command: run /usr/sbin/sbim --install /usr/lib/boot/mboot.bootrom
Command: quit
```

This installs the default `mboot` bootrom file. Change the file uploaded with the first command given to `tile-monitor` to install something different.

2. Shut down the target machine and reconfigure it to boot from the SROM. See the user's guide for the respective Tilera card for information on how to configure the DIP switch to support booting from SROM.
3. Power on the machine. If using a PCIe adapter, Tilera recommends plugging the card's serial port into another machine instead of the PC into which it is installed. The Tile Processor starts to boot as soon as power is applied, and if it is plugged into the same PC, all of the boot output will be gone long before the PC boots.
4. Do one of the following, depending on whether the boot is successful:
   - If the boot is successful and you are able to bring up the network, copy a new boot image onto the card and install it into the SROM to try different configurations.
   - Otherwise, shut down the target, reconfigure it to disable SROM boot, and try again.

Note: Use the `tile-monitor --reflash` argument to update the SROM image on a networked native TILE-Gx platform with a new bootrom file image. Without the `--reflash` argument, you can only reboot a networked TILE-Gx platform using the same bootrom file that the platform used the previous time it booted.

The Tile Processor can be booted from several other on-board devices, including I^2C (as an I^2C slave device) and the UART. The Tilera Board Test Kit (`tile-btk`) provides support for booting over the UART.

The I^2C booting method must be driven by another processor, likely located on a customer's own custom hardware; therefore, Tilera does not supply tools to perform the boot operation by means of I^2C. See the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404) for information on the specific protocols used to boot over this device.

Note: The same bootroms that are bootable over the PCIe interface are also bootable over the USB and UART interfaces. Once that code is loaded into the processor, the boot proceeds independently from the device used.

See Section 1.9 "SROM Boot Image Management (sbim)" on page 11 for information on how to use the `sbim` program (on the Tile Processor) or the `tile-sbim` program (on the host) to manage the contents of a bootable SROM.

1.8.4 Booting from a Solid-State Drive or Compact Flash

Using the boot loader, `mboot`, you can boot the Tile Processor image over a solid-state drive (SSD) or CFast device. To access the SSD or CFast device, the `mboot` kernel must have the IDE driver and the EXT2 file system module installed, either as a built-in kernel component or as a loadable kernel module. The standard `mboot.bootrom` includes these components. Note that when we refer to SSD in the text that follows we are including CF.

To boot the Tile Processor from a compact flash device, use the mboot CLI to perform the following steps. For more information on the mboot CLI, see Section 5.3 "mboot Command-Line Interface" on page 70 and Appendix B— mboot CLI Commands.

1. Complete the steps in Section 1.8.3 "Booting from SROM" on page 7. This is a prerequisite to booting from an SSD device.

2. Use the `mboot ls` command to list all available files on the storage devices in the system:

   ```
   mboot: ls
   The image files in HD:

   vmlinux.bootrom

   The image files in MEM:
   ```

3. If needed, use the `tftp` command to download a kernel image onto the SSD:

   `mboot: tftp` *`hostname-or-IP-address`* `vmlinux -hd`

4. Use the `boot` command to reboot the system using the boot image specified in the command line:

   ```
   mboot: boot -hd -i vmlinux
   ```

5. To boot the system from a drive by default, use the mboot `bootparam` command to set the default device and default image name from which the system should boot. The following command configures the system to boot from the `vmlinux` bootrom image on the SSD device on the next reboot:

   ```
   mboot: bootparam -dev hd -img vmlinux
   ```

Creating a File System

Support for the EXT2 and EXT3 file systems is configured in both the `mboot` and standard Linux kernels.

To create a file system on the Tile Processor card, boot the card, mount the EXT3 support utilities by means of `tile-monitor`, and run the file system creation program as follows:

```
$ $TILERA_ROOT/bin/tile-monitor --root -- mkfs.ext3 /dev/hda1
```

To mount the new file system for use, execute the following commands on the Tile Processor, using `tile-monitor` or some other communication mechanism such as `telnet` or the serial console:

```
# mkdir mountpoint-path
# mount /dev/hda1 mountpoint-path
```

Repairing a Damaged Partition Table

1. To repair a damaged partition table on a compact flash or SSD card, do the following on the Tile Processor card:

   ```
   # fdisk /dev/hda1
   ```

2. At the `fdisk` prompt, `h` will give you a brief summary of the available commands. To create one large, empty partition on the disk, you can run the following commands:

   a. The `o` command will create a new empty partition table, overwriting all previous content on the compact flash or SSD card.

   b. You can then use the `n` command to add a new partition, selecting `p` when prompted to create a `primary partition`, `1` to choose the first partition, and hitting return to accept the default first and last cylinder.

      This will create one partition that is the size of the whole disk.

c. Finally, use the w command to write the new table to disk and exit.

```
Command (m for help): o
Building a new DOS disklabel. Changes will remain in memory only, until you
decide to write them. After that the previous content will not be recoverable.

The number of cylinders for this disk is set to 7937.
There is nothing wrong with that, but this is larger than 1024, and could in
certain setups cause problems with:
1) software that runs at boot time (e.g., old versions of LILO)
2) booting and partitioning software from other OSs
   (e.g., DOS FDISK, OS/2 FDISK)

Command (m for help): n
Command action
   e   extended
   p   primary partition (1-4)
p
Partition number (1-4): 1
First cylinder (1-7937, default 1): Using default value 1
Last cylinder or +size or +sizeM or +sizeK (1-7937, default 7937): Using
default value 7937

Command (m for help): w
The partition table has been altered!
```

At this point, it is possible to build a file system on the compact flash card; see Section "Creating a File System" on page 9.

Note: To run interactive programs such as `fdisk` on the Tile Processor, use `telnet` or `tile-console`, which provides access to a console port on the Tile Processor with support for interactive login to Linux. See the section on "Using tile-console to Monitor Output from the OS" in the *Multicore Development Environment User Guide* (UG501) for details.

1.8.5 Booting from a SATA Drive

This section describes how to boot the Tile Processor from a SATA hard disk drive.

Creating a SATA File System

Boot the system into the standard initramfs file system. Because this configuration does not include the disk management tools, it is necessary to copy or mount them from elsewhere. The easiest way to do this is with the `tile-monitor --root` option to mount and upload all the necessary Tile-side components from the installed `$TILERA_ROOT/tile/` directory.

As on any other Linux system, you need to partition the SATA driver with `fdisk /dev/sda` (for example). Then you need to create new mountable file systems with `mkfs.ext3 /dev/sda1`(for example).

Here is an example command that performs these tasks:

```
$ tile-monitor --net addr --root \
    --run - fdisk /dev/sda - \
    --run - mkfs.ext3 /dev/sda1 - \
    --quit
```

To make the SATA drive partitions available to Linux, you can now `mount /dev/sda1 /mnt` (for example). Or see Section "Booting from a SATA File System" below.

Booting from a SATA File System

Assume that you have a Tile system and that you have created a SATA file system as described in Section "Creating a SATA File System". Follow these steps to boot from a SATA file system:

1. If needed, boot the system into the standard `initramfs` file system:

   ```
   $ tile-monitor --net addr --quit
   ```

   Note: Booting is not necessary if you have already booted into the initramfs file system as described in Section "Creating a SATA File System".

2. Use `xzdec` and `ssh` to install the Tile tarball and decompress on x86:

   ```
   xzdec $TILERA_ROOT/tile.tar.xz | \
     ssh -x root@addr 'mkdir /a; mount /dev/sda1 /a; tar -C /a -xf -'
   ```

3. To burn a new bootrom into the SROM, which will cause booting to pivot root to `/dev/sda1`, and reboot the Tile Processor, use this command:

   ```
   $ tile-monitor --net addr --hvx TLR_ROOT=/dev/sda1 -- quit
   ```

During this reboot (and all future reboots), the hypervisor, `hvconfig` file, and `vmlinux` will come from the SROM. Then Linux will "pivot root" to the disk partition created above, causing the files on the disk partition to be used as the standard file system.

For information about `TLR_ROOT`, see Table 3-3, "Environment Variables for /etc/rc Startup Script for Initramfs Boot Operations," on page 55.

1.8.6 Booting with a Root Filesystem Available over NFS

The `tile-monitor` command supports booting with a root filesystem available over NFS. The initramfs file `initramfs-diskless-nfs.cpio.xz` includes name server hostname resolution and support for using DHCP to determine the IP address of an NFS server machine from which the root filesystem is made available.

To boot a TILE-Gx device with the root filesystem available over NFS, use a `tile-monitor` command like this:

```
tile-monitor \
--hvx TLR_ROOT=10.200.0.160:/export/nfsroot423 \
--hvx TLR_ROOT_TYPE=nfs \
--hvx TLR_ROOT_NETWORK=eth0 \
--hvx TLR_ROOT_NETADDR=dhcp \
--hvx TLR_ROOT_OPTIONS=proto=tcp \
--initramfs $TILERA_ROOT/tile/boot/initramfs-diskless-nfs.cpio.xz
```

Note: The TILE-Gx device must have `no_root_squash` access to the NFS filesystem from which you mount the roofs. In this example, add an entry like this to the `/etc/exports` file on the NFS server from which you are exporting the rootfs:

```
/export/nfsroot423      *(rw,async,no_root_squash)
```

You must then re-export the NFS filesystem by executing (as root) the command `exportfs -a`.

1.9 SROM Boot Image Management (sbim)

The SROM Boot Image Management (`sbim` or `tile-sbim`) program manages the contents of a bootable SROM. It allows installation and management of multiple bootable images within one ROM. Bootable images can be the Tilera stand-alone boot loader, `mboot`, one of the standard Linux images distributed by Tilera, or a custom bootable image constructed to meet specific customer requirements.

Once an appropriate bootable image has been installed in an SROM, a Tile Processor can then be booted from it. (The system card must be reconfigured by means of DIP switch settings to boot from its SROM; see Section 1.8.3 "Booting from SROM" on page 7 for details.)

Generally, Tilera recommends using `sbim` on the target Tile Processor. This provides the most reliable operation, because `sbim` can automatically sense SROM characteristics, and can modify only the necessary areas of the SROM for each operation. However, this is not always possible. For example, you might want to prepare a simulated SROM to use with the simulator (`tile-sim`), or you might need to prepare a prototype image to be transferred to many target SROMs as part of a manufacturing process. In this case, you would use the `tile-sbim` command, which runs on the host system.

You could create a new boot image (`myboot.bootrom`) with a customized hypervisor configuration file (`myconfig.hvc`) and then install it in a previously-unused SROM on a Tile Processor card. The image should be installed twice. For robustness, it is recommended that you have at least two valid bootable images in your SROM whenever possible.

Once a booter has been installed in an SROM, it consists of a number of image slots, each of which can hold one bootable image. Image slots are numbered from zero to one less than the total number of images. Commands that act on a particular image slot have a location as a required or optional argument; a location specifies a slot number, either directly, or indirectly.

The valid formats for a location are:

`@n`	Specifies an absolute image slot number.
`0`	Specifies the image with the largest generation number.
`-n`	Specifies the nth most recently installed image. Thus, -1 specifies the image installed just before the most recently installed one; -2 specifies the image installed just before that; and so on.

For more information, see the `sbim` man page.

1.9.1 Boot Policy

When a Tile Processor card is booted from its SROM, the first code executed is the SROM booter. The SROM booter is responsible for verifying the remaining content of the SROM, choosing one of the set of valid bootable images, and then actually loading that image into the processor to complete the boot process.

The policy used by the default SROM booter provided with the MDE is to pick the image within the SROM that was most recently installed or promoted. This determination is done by examining each image's generation number; the image with the largest number is the most recent. An image with a lower generation number can be used if the one with the largest number is found to be corrupt.

Note that if there are two valid images, you can ask that the older image, rather than the newer one, be used for a boot, by typing `a` at the boot prompt:

```
Type t in three seconds to run thorough POST tests, q to run quick tests, or a to
boot alternate image.
```

1.10 Power-On Self Test (POST) Tests

This section describes the individual tests that are performed as part of the Power-On Self Test (POST) that runs when the Tile Processor boots.

See also Section 2.8.1 "Power-on Self-Test Configuration Options" on page 29.

1.10.1 Enabling Thorough Testing Mode

When booting, you are prompted for input to determine whether the system should run a quick or thorough POST. If the system does not receive an answer within 3 seconds, it runs the thorough POST. An example of the prompt looks like the following:

```
Type t in 3 seconds to run thorough POST tests, q to run quick tests...
```

You can control the POST mode using a hypervisor configuration option when you build the bootrom. See the `post=mode` option in Section 2.3.1 The options Command.

You can also control the POST mode by using the `POST_MODE` compile-time option for the Hypervisor. See Section 2.8.1 “Power-on Self-Test Configuration Options” on page 29.

1.10.2 Quick DRAM Test

Before any DRAM (dynamic random access memory) is used, a quick test runs that is designed to test the interconnect between the Tile Processor and the DRAM chips. The address and data lines of each memory controller are covered with *walking 1* and *walking 0* patterns.

This test always runs automatically during boot.

If a memory location fails, an error message is printed indicating the memory controller, address, and expected and actual data. The memory controller of the failing location is disabled, and testing continues.

As long as one of the memory controllers passes the test, boot continues.

To modify the default behavior on failure, edit the case for the code `POST_ERR_QUICK_DRAM` in the `src/sys/hv/boot_error.c` file.

1.10.3 Thorough Hypervisor DRAM Test

Before the hypervisor is loaded into memory, the memory range it will occupy is thoroughly tested. Multiple passes are made through every location of memory in the range to ensure that every bit can store a one and a zero, and to test for interference between adjacent bits.

This test only runs in thorough mode.

If a memory location fails, an error message is printed indicating the memory controller, address, and expected and actual data. The memory controller of the failing location is disabled, and the memory of the next controller is tested.

As long as one of the memory controllers passes the test, boot continues.

To modify the default behavior on failure, edit the case for the code `POST_ERR_HV_RAM` in the `src/sys/hv/boot_error.c` file.

1.10.4 Thorough Hypervisor File System DRAM Test

Before the hypervisor file system is initialized, the memory range it will occupy is thoroughly tested. Multiple passes are made through every location of memory in the range to ensure every bit can store a one and a zero, and to test for interference between adjacent bits.

This test only runs in thorough mode.

If a memory location fails, an error message is printed indicating the memory controller, address, and expected and actual data. Then the boot process halts.

To modify the default behavior on failure, edit the case for the code `POST_ERR_HV_RAM_FS` in the `src/sys/hv/boot_error.c` file.

1.10.5 Thorough Test for the Rest of DRAM

Before the hypervisor loads the client operating system, all memory not occupied by the hypervisor is thoroughly tested. Multiple passes are made through every location of memory in the range to ensure that every bit can store a one and a zero, and to test for interference between adjacent bits.

This test only runs in thorough mode.

If a memory location fails, an error message is printed indicating the memory controller, address, and expected and actual data. The memory controller of the failing location is disabled, and the boot process continues.

1.10.6 memtest

This test stresses the DDR3 DRAM modules in order to detect electrical signal integrity problems. It tests all available free memory after Linux has booted. Also, all available tiles are writing to and reading from memory in order to maximize the memory traffic.

This test does not run automatically. You can run the test by typing `memtest` at the Linux prompt. You can add an optional parameter `--passes n`, where `n` is the number of times to loop through the available memory. Each pass takes a few seconds, depending on how much memory is installed in the system.

If a failure is detected, a message is printed indicating the address and expected and actual data. Then testing continues until the desired number of passes are finished.

CHAPTER 2 HYPERVISOR CUSTOMIZATION

This chapter describes:

See also Appendix A— Hypervisor Console Escape Commands.

2.1 Overview

The Tilera® hypervisor abstracts some hardware details of the Tilera chip on which it is running from the supervisor, managing communication between tiles, from tiles to I/O controllers, and providing a low-level virtual-memory system. For information about the services the hypervisor provides, see Section 1.3 “Hypervisor” on page 4.

You can customize the hypervisor configuration file. This chapter tells you how.

2.2 Hypervisor Configuration File

The hypervisor supports a configuration file that allows you to control properties such as enabling and disabling I/O devices, allocating tiles to specific uses, and specifying the behavior and operation of the supervisor. The hypervisor configuration (`.hvc`) file is contained within the hypervisor file system.

See Section 2.4 “Understanding a Hypervisor Configuration File” on page 25.

To get information about the hypervisor while running Tile Linux, see these files in `/sys/hypervisor/`:

- `hvconfig`: Contains information about the hypervisor configuration (which devices are configured, which tiles they use, and so forth). The data in this file is intended to be used by humans, and not by programs. Its format is currently similar to that of the hypervisor configuration file, but this might change in the future.
- `version`: Contains information about the version of the hypervisor running on the processor.

- `config_version`: Contains information about when and by whom the bootrom was created.

2.2.1 Hypervisor Configuration File Format

The hypervisor configuration file contains commands and their associated, optional subcommands. Commands must start in column 1; subcommands must be preceded by at least one space. Each line can contain only one command or subcommand. Long command lines can be split across multiple lines with a backslash (`\`) at the end of each line.

Numeric command arguments can be in decimal, octal (with a leading 0), or hexadecimal (with a leading 0x or 0X). If a number is immediately followed by a `k`, `m`, or `g`, it is multiplied by 2^{10}, 2^{20}, or 2^{30}, respectively.

The hypervisor configuration file can contain comments, which begin with a pound (#) sign and go to the end of the line. To get a literal # character, use `\#`.

2.2.2 Preprocessor Syntax for Hypervisor Configuration Files

Hypervisor configuration files support a preprocessor syntax that allows their behavior to be changed without editing the file. Preprocessing allows one `.hvc` file to work for several different board configurations, and also allows easy customization of boot and driver parameters.

The preprocessor syntax is reminiscent of that used by C, but has some important differences:

- Preprocessor directives do not start with #.
- Preprocessor variables are expanded using `$VARIABLE` or `${VARIABLE}` syntax.
- Integer expressions can be evaluated using the `$(...)` syntax.
- To get a literal `$` character, use `\$`.

2.2.3 Preprocessor Directives for Hypervisor Configuration Files

If the first word on a line is one of the following directives, that line is processed by the preprocessor.

`define VARIABLE VALUE`

This defines a variable with the value `VALUE`, which can be empty. The variable can later be referenced as either `$VARIABLE` or `${VARIABLE}`, or checked with `defined(VARIABLE)`. Any `$` expressions in `VALUE` are evaluated immediately. Variables can be redefined, including in terms of their previous value. This is similar to C's `#define`.

`undef VARIABLE`

This removes the definition for `VARIABLE`, or does nothing if `VARIABLE` is not defined. This is similar to C's `#undef`.

`include FILEPATH`

This textually includes the contents of the specified file. Unlike C, `PATH` is not enclosed in double quotes or angle brackets. If `FILEPATH` is absolute, it specifies the file location. Otherwise the file is searched for in the following locations in order:

1. The directory containing the file that contains the `include` directive.
2. Any directories specified with `--hvi` arguments to `tile-monitor` or `tile-mkboot`.
3. The default `.hvc` directory: `$TILERA_ROOT/tile/etc/hvc`, or `/etc/hvc` when running natively.

This is similar to C's `#include`.

`error MESSAGE`

This prints out the specified error message and terminates `.hvc` processing with an error exit code. This is similar to C's `#error`.

`if PREDICATE`

`PREDICATE` is evaluated as a numeric expression, as if it were inside `$(...)`. If nonzero, the following text up until the next else, `elif` or `endif` is processed normally. Otherwise it is discarded. It must be matched with a closing `#endif`. This is similar to C's `#if`.

For example,

```
if $WIDTH < 8
something
else
something_else
endif
```

will be preprocessed into the line `something` if `$WIDTH` is less than 8, otherwise `something_else`.

`ifdef VARIABLE`

Equivalent to if `defined(VARIABLE)`. This is similar to C's `#ifdef`.

`ifndef VARIABLE`

Equivalent to if `!defined(VARIABLE)`. This is similar to C's `#ifndef`.

`elif PREDICATE`

If some earlier predicate in the `if/elif` chain already matched, `PREDICATE` is treated as zero. Otherwise, `PREDICATE` is evaluated as a numeric expression, as if it were inside `$(...)`. If nonzero, and all preceding `if` and `elif` predicates are zero, the following text up until the next else, `elif` or `endif` is processed normally. Otherwise it is discarded. This is similar to C's `#elif`.

`else`

If all preceding `if` and `elif` predicates are zero, the following text up until the following `endif` is processed normally. Otherwise it is discarded. This is similar to C's `#else`.

`endif`

This terminates an `if`, `ifdef`, or `ifndef` directive. This is similar to C's `#endif`.

2.2.4 Preprocessor Expressions for Hypervisor Configuration Files

`$VARIABLE, ${VARIABLE}`

These expressions are replaced with the value of `VARIABLE` from the most recent definition, which is either a `define` directive or a `--hvd VARIABLE=VALUE` argument to `tile-monitor` or `tile-mkboot`, or, in the special case of the `XARGS` variable, text provided by the `--hvx VALUE` command. Variable substitutions can be used in expression contexts or inside normal text.

Expanding a variable which is not defined is an error.

The curly brace form ${VARIABLE} is needed when the result of the variable expansion must be adjacent to text that might otherwise parse as more characters in the variable's name.

For example:

```
define NUM_TILES 4
device xgbe/0 xgbe_ipp${NUM_TILES}_epp
```

$(...)

Integer expressions can be evaluated by wrapping them inside $(). For example, $($WIDTH - 1) would expand the $WIDTH variable and then subtract one from it. The result of a $(...) expression must be an integer. Integer constants can be specified in decimal or hexadecimal.

Most C infix operators are allowed, as well as the defined(VARIABLE) syntax. Specifically, expressions can use the (,), !, ~, *, /, %, +, -, >>, <<, >, <, <=, >=, ==, !=, &, ^, |, &&, and || operators. They operate only on integers and have the same semantics, associativity and operator precedence as in C. All expressions are treated as signed integers.

The && and || operators are short-circuiting (again, as in C) so it is legal to write:

```
if defined(VARIABLE) && $VARIABLE
```

even though the $VARIABLE expression would normally produce an error if VARIABLE is not defined.

In addition to the above infix operators, the expression defined(VARIABLE) is replaced with 0 if VARIABLE is undefined, or 1 if it is defined. It is similar to C's defined keyword.

Note: You must include the triple \ character because the $ character is a variable reference operator in the hypervisor configuration file language. The hypervisor processes escapes in a --hvx argument twice. You must also enclose the hypervisor option declaration in single quotes.

Normal Variables

XARGS

XARGS is a normal variable, except that text can be appended to it conveniently on the command line. The tile-monitor and tile-mkboot --hvx VALUE expression appends VALUE to the end of the XARGS variable, separated by a space if XARGS was already defined. By convention XARGS is used for Linux boot arguments, but of course a .hvc file could use it for something else.

CONFIG_VERSION

CONFIG_VERSION is a human-readable string describing the .hvc file, intended for diagnostic purposes such as identifying which .hvc file got booted on a running system.

2.3 Hypervisor Configuration File Commands

This section describes the commands that can be used in the hypervisor configuration file.

2.3.1 The options Command

The options command sets hypervisor options, and takes the following form:

Syntax

`options` *option* [*option* ...]

Options can be of the form *foo* or *foo=bar*. The currently available options are:

`cpu_speed=`*speed*

Sets the CPU clock frequency to *speed*, in megahertz. The valid CPU frequency range depends on the card used. For a given CPU speed, Tilera dynamically sets the minimum CPU core voltage required to run at the requested speed. The default speed depends on what kind of card is installed. For more information about frequency range, refer to the appropriate Tilera data sheet for your chip.

If you request a speed that is outside the range of the card's capabilities (see above), the hypervisor software generates a warning and the chip still runs at the board's default speed.

The Tile Processor chip cannot run at any arbitrary frequency, but runs at the closest available frequency that is less than or equal to that requested. To check the frequency at which a chip is running, examine the file `/proc/cpuinfo` once the board has booted.

See also Section 3.2.2 "The /proc Virtual File System" on page 40.

`console_debug=`*string*

Specifies the string that, when entered on the UART console, puts the hardware into protocol (debug) mode. This can enable use of low-level debug tools, and is particularly valuable when it is necessary to retrieve useful information from a hung system.

The console debug string can be no longer than 16 characters, and can contain the following escape codes:

```
\r                Carriage return
\n                Newline
\t                Tab
\<octal-digits>   Any arbitrary ASCII character
\x<hex-digits>    Any arbitrary ASCII character; for example,
                 \use \x5c to insert a backslash.
```

Note: When the debug string is entered at the console prompt, it is not passed to the client supervisor. It is thus advisable to pick debug strings that users are not likely to type at the console prompt.

If the `console_debug` option is not specified, the default string used to enter protocol mode is `"\x1e"`, or the control-caret character (^^).

This feature might not interact well with applications that need to read raw binary data from the console. For such applications, specifying the `console_debug` option without a string, or with an empty string, disables entry into debug console mode.

See also Appendix A— Hypervisor Console Escape Commands.

`default_shared=x,y`

Specifies using tile *x,y* as the shared tile for devices that need exactly one shared tile and whose `device` stanza does not explicitly specify one. If this option is not present, the hypervisor uses tile (0,0) as the default shared tile.

See also Section 3.2.2 "The /proc Virtual File System" on page 40.

`dvfs=`*mode*

Enables dynamic voltage and frequency scaling availability. Choose from among the following modes:

- `off`

  Do not allow the primary client to modify any frequencies during operation.

- `core`

  Allows the primary client to modify the core frequency during operation, but do not make any corresponding changes in core voltage. This is the default setting.

- `core_volt`

  Allows the primary client to modify the core frequency during operation, and make corresponding changes in core voltage. This setting may lead to instability in some system configurations.

When you enable DVFS, in either `core` or `core_volt` mode, you can set the processor core frequency by writing a new frequency value (expressed in kHz units) into this file:

```
/sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
```

Note: If you adjust voltage through this setting, the system accommodates the frequencies of the I/O shims as well as the core. To permit maximum voltage decrease at lower core frequencies, and thus maximum power savings, you might also reduce I/O shim frequencies to the minimum required for their application (via the speed subcommand in the relevant device stanzas in this file), and disable any devices that you are not using by omitting the device stanzas for those devices entirely.

`mem_error=`*`mode`*

Configures the hypervisor's response to non-fatal processor errors. *`mode`* must be one of the following:

- `silent`

  Do not issue any console messages for non-fatal errors.

- `warn`

  Issue console warning messages for non-fatal errors. This is the default if no mode is specified.

- `panic`

  Panic on non-fatal errors.

`mem_speed=`*`speed`*

Sets the DDRAM access speed to speed, in millions of transactions per second. The valid DDRAM access speeds and the default speed depend on the board. For more information about DDRAM speed, refer to the appropriate Tilera data sheet for your chip.

If you request a DDRAM speed that is above the range of the board's capabilities, the DDRAM runs at the board's maximum speed. The Tile Processor memory interface cannot run at any arbitrary speed, but runs at the closest available speed that is less than or equal to the speed that you request.

Note: You can disable a specific `mshim` by setting the speed for that shim to the value of -1. For example to set the speed of `mshim0` to 1600 and disable `mshim1`, use the following `tile-monitor` argument:

```
--hvd MEM_SPEED=1600,-1
```

To determine the frequency at which the memory is running, examine the file `/proc/tile/memory` once the board has booted. The `controller_`*`n`*`_speed` values in this file are in bytes per second, and the DDRAM interface is eight bytes wide. So for example, a `mem_speed` setting of 800 (MT/s) appears as 6400000000 (bytes/s) in the file `/proc/tile/memory`.

See also Section 3.2.2 "The /proc Virtual File System" on page 40.

`post=`*`mode`*

Sets the power-on self-test mode. Choose from among the following modes:

- `quick`

  Runs a quick POST.

- `query_quick`

  Asks on the console whether to run a thorough POST. If no response, then run the quick POST.

- `thorough`

  Runs a thorough POST.

- `query_thorough`

  Asks on the console whether to run a thorough POST. If there is no response, run a thorough POST.

You can use `tile-monitor` to boot a device and supply POST modes by specifying `--hvd POST=`*`mode`* on the command line.

Note: Disabling the POST query via the quick or thorough options also disables the ability to request booting from the alternate SROM image.

`stats`

Enable the collection of hypervisor statistics. When this option is present in the `config.hvh` file, the hypervisor counts and times various operations (interrupt handlers, hypervisor messages, hypervisor syscalls) and makes summary statistics available though various interfaces. For details, see Section 2.5 "Gathering Hypervisor Statistics" on page 25.

`stripe_memory[=`*`mode`*`]`

The `stripe_memory` option requests that the system be configured to stripe memory accesses across all possible memory controllers. When a set of controllers is striped, the client supervisor sees one unified physical address space rather than separate address spaces.

Modes

If *`mode`* is present, it specifies one of the following:

- `never`

  Do not stripe memory.

- `default`

  Stripe pairs or quartets of memory controllers that have equal amounts of memory. If no striping is possible, print a warning.

  This is the default if no mode is specified.

- `silent`

Stripe pairs or quartets of memory controllers that have equal amounts of memory. If no striping is possible, do not print a warning.

- `always`

  Stripe pairs or quartets of memory controllers that have memory attached, even if this means reducing the amount of memory made available (when striping, all controllers in the striped group must have the same size). If memory is lost by this, print a warning.

2.3.2 The client Command

The `client` command defines a client supervisor. The client command and any following associated subcommands make up a client stanza.

By default, the hypervisor supports only one client definition. To enable multiple client support, see Section 2.8.3 “Miscellaneous Option” on page 30.

Syntax

```
client exe-name [ wd x ht [ ulhcx , ulhcy ] ]
```

Arguments

The supported arguments to the `client` command are:

exe-name

The executable file to run as the supervisor program. This must be a file within the hypervisor file system. This argument is required. Conventionally, this is `vmlinux`.

wd and *ht*

These two arguments specify the width and height of the tile rectangle allocated to the client. These arguments are optional; if omitted, the entire chip is used.

ulhcx and *ulhcy*

These two arguments specify the origin of the tile rectangle. These arguments are optional; if omitted, the upper left-hand corner of the chip, (0,0), is used.

The args Subcommand

Linux uses the arguments provided to the `args` subcommand as its boot arguments. See Section 3.3 “Linux Boot Arguments” on page 44 for more information.

The memory Subcommand

The `memory` subcommand defines the amount of memory the client is allowed to use on each memory controller. The value is specified in bytes.

Syntax

```
memory size-ctl-0 [size-ctl-1 [size-ctl-2 [size-ctl-3]]]
```

If this line is omitted, or the value `default` is specified, the client is given a share on the respective controller equal to that of all other clients for which an explicit size is not specified. A value of 0 disables that memory controller for this client. Values specified for nonexistent memory controllers are ignored.

Memory sizes can take a trailing `k`, `m`, or `g` to signify kilobytes, megabytes, or gigabytes, respectively.

The hfh_tiles Subcommand

The `hfh_tiles` subcommand designates the set of tiles that will be used to cache data from pages designated as "hash-for-home."

For example, you could set `hfh_tiles` to be the same as the dataplane set specified in the Linux boot arguments and then use hashing for all the dataplane allocations. Or you could set `hfh_tiles` to be the non-dataplane tiles, and use only per-page caching for the dataplane tiles, to avoid evicting any cache lines on dataplane tiles.

All specifications are relative to the client's tile grid.

By default, all tiles accessible to the client are used for hash-for-home. After the set is computed, any tiles that are being used as dedicated driver tiles are silently removed from it.

Syntax

`hfh_tiles` *`tilespec`* [*`tilespec ...`*]

Arguments

`tilespec` is one of:

`x,y`

One tile.

`wxh`

A rectangle of tiles `w` wide by `h` high whose upper-left-hand corner is (0,0).

`wxh@x,y`

A rectangle of tiles `w` wide by `h` high whose upper-left-hand corner is (`x`,***`y`***).

`cpunum`

Tile number `cpunum`, using Linux's CPU number mapping scheme.

`cpunum0-cpunum1`

Tiles ranging from `cpunum0` through `cpunum1`, inclusive.

`^tilespec`

Subtract the named tiles from those previously specified. If the first `tilespec` is a subtraction, start with the default set of tiles, else start with none.

2.3.3 The device Command

The `device` command requests that a specific device be included in the configuration. The `device` command and any following associated subcommands make up a device stanza. No devices are included by default. Devices must be specified in this manner if they are to be used.

Syntax

`device` *`devname`* [*`drivername`*]

Arguments

`devname`

The name of the device. The `devname` argument is of the form *`name/instance #`*. It comprises a device name, a slash '/', followed by a device instance number (for example, `xgbe/0`). This argument is required.

`drivername`

The name of the driver to be used. This argument is optional; if omitted, the first driver in the driver list that knows how to handle the named device is selected.

For information on specific options available for each device, see

Chapter 6: I/O Devices and Drivers.

Subcommands

`shared` *`x0 , y0 [ x1, y1 [ x2 , y2 ] ]`*

The `shared` subcommand designates tiles to be used as *shared tiles*, which can process device requests from one or more drivers and can also run the hypervisor and supervisor. Usage of these tiles, and the required number, depends on the device driver in use. For information about shared tiles, see Section 2.6 "Using Dedicated and Shared Tiles" on page 27.

`dedicated` *`x0 , y0 [ x1, y1 [ x2 , y2 ] ]`*

The `dedicated` subcommand designates tiles to be used as *dedicated tiles*, which exclusively process device requests and do not run a client supervisor. The usage of these tiles, and the number required, depends on the device driver in use. For information about dedicated tiles, see Section 2.6 "Using Dedicated and Shared Tiles" on page 27.

`args` *`args`*

The `args` subcommand specifies arguments to be passed to the device driver. The maximum size of these arguments is 1024 bytes. Chapter 6: I/O Devices and Drivers specifies the available arguments for each driver.

2.3.4 The print Command

The `print` command writes a message to the hypervisor console during boot, and takes the following form:

`print` *`message`*

The message can be any sequence of up to 127 printable characters, and is written to the console without change; no interpretation of escape sequences is done. Spaces after the `print` command and before the first non-space character are omitted.

This command can be useful when using many different bootrom files, as it can give you a way to visually determine which image has been booted.

More than one `print` command can be used.

2.3.5 The config_version Command

`config_version` *`text`*

The `config_version` command defines a version string for this hypervisor configuration, which is printed on the console at boot time, and is retrievable by means of the hypervisor's `confstr` interface.

The command takes the following form:

`config_version` *text*

The string is intended for use by humans, not programs, and is uninterpreted; it might contain whitespace. Only the last such command within the configuration file has any effect. Note that Tilera's default configuration file supplies this command, with an automatically-generated version string.

2.3.6 Additional Parameters

These are additional parameters used in some hypervisor configuration files:

- `CPU_SPEED`

  Requests the specified CPU clock frequency, in MHz. (This basically sets the value of the hypervisor option `cpu_speed`. See Section 2.3.1 "The options Command" on page 18.)

- `OPTIONS`

  Optional. Can be used to specify additional hypervisor options. It is in the same format as described in Section 2.3.1 "The options Command" on page 18.

2.4 Understanding a Hypervisor Configuration File

This section provides an example of a hypervisor configuration file.

Example 2-1. Hypervisor Configuration File

```
options default_shared=2,5

# Configure an SROM device using the default shared tile for its I/O.
device srom/0 srom

# Run Linux as the supervisor, passing the specified options to the
# initialization script (/etc/rc) as environment variables when Linux
# boots.
# Run Linux as the supervisor.
client vmlinux
  args TLR_NETWORK=xgbe0
```

The above configuration file sets up the following devices:

- Tile (2,5) is specified as the default shared tile.
- One SROM device is specified.
- The client Linux system is allocated all tiles that have not been reserved for driver dedicated tiles.

The argument to the `client vmlinux` command tells Linux to bring up the Ethernet interface when it boots, using DHCP for network addressing information (`TLR_NETWORK=xgbe0`).

2.5 Gathering Hypervisor Statistics

Use the hypervisor statistics feature to obtain information on the number and duration of various hypervisor events: interrupts, inter-tile hypervisor-to-hypervisor requests (hypervisor messages) and client-to-hypervisor requests (hypervisor syscalls).

2.5.1 Enabling Hypervisor Statistics

The collection of statistics involves extra overhead on hypervisor operations; therefore, statistics are not collected by default. You must decide to collect hypervisor statistics when you boot the hypervisor.

To enable hypervisor statistics collection, either:

- Add this option to the `config.hvh` file:

  ```
  options stats
  ```

- Add this option to the `tile-monitor` command line:

  ```
  --hvd STATS
  ```

2.5.2 Retrieving Hypervisor Statistics

After you enable the collection of hypervisor statistics, you can retrieve statistics values using two methods:

- A `/sys` file
- From the console using an escape command.

The following sections detail those methods.

Viewing Hypervisor Statistics from a /sys File

The hypervisor creates a statistics file for each tile in the `/sys` filesystem:

```
/sys/devices/system/cpu/cpuN/hv_stats
```

In this filename, *N* refers the Linux CPU number.

The following shows the contents of the statistics file for CPU 0:

```
# cat /sys/devices/system/cpu/cpu0/hv_stats
# (0,0) Name                        Events   Mean cyc/evt    Max cyc/evt
       I DTLB_MISS                      12           1051           1714
       I IDN_AVAIL                      29           3361          30764
       I IPI_3                          59            980           3635
       M write_console                   4          15214          30059
       M read_console                   21            510           1805
       M flush_remote:tlb                3           2348           4162
       S dev_pread                       2           1318           2220
```

Viewing Hypervisor Statistics from the Console

You can dump hypervisor statistics for all tiles to the console using console escape commands. For details, see Section A.3 “Console Escape Commands Used with Hypervisor Statistics” on page 113.

2.5.3 Interpreting Statistics Details

Most statistics are primarily of interest to systems programmers doing low-level performance tuning, and fully understanding each event may require study of Linux or hypervisor source code.

There are three types of events measured by the hypervisor statistics feature:

- Interrupts sent to the target tile and handled by the hypervisor. These events have an `I` in the first column; the second column contains the name of the interrupt (see `<arch/interrupts.h>`).
- Hypervisor messages, which are requests sent from the hypervisor on one tile to the hypervisor on the target tile. These events have an `M` in the first column; the second column contains the name of the message (see `$TILERA_ROOT/src/sys/hv/msg.h`).
- Hypervisor system calls, which are requests made to the hypervisor by the client supervisor on the target tile. These events have an S in the first column; the second column contains the syscall name (see `$TILERA_ROOT/src/sys/hv/include/hv/hypervisor.h`).

For each hypervisor event, the statistics feature tracks these quantities:

- Number of events seen on the target tile since boot (or since the statistics were reset to zero).
- Mean duration of each event in processor clock cycles.
- Maximum duration of each event in processor clock cycles.

Provisos for Hypervisor Statistics Gathering

The hypervisor statistics gathering mechanism is quite accurate; however, it cannot include every single cycle of the event. The statistics code itself takes time to execute, and some of that code necessarily executes before or after it measures the event cycle counts. Thus, a hypervisor event that takes X cycles might well have interrupted the client supervisor or user application for slightly more than X cycles.

Most ITLB and DTLB misses are satisfied by high-performance handlers which use hardware acceleration to look up translations in a software-managed cache, called the TSB. (See the Hypervisor Theory of Operation technical note for more information on the TSB.) The hypervisor statistics mechanism does measure these ITLB and DTLB events, since that would significant impact performance. Instead, the hypervisor statistics gatherer logs statistics only in the event that a translation is not found in the TSB (a "TSB miss"). If you want to get a count of the total number of ITLB or DTLB misses, you can configure the performance counters to count them, using the `oprofile` tool.

For statistics purposes, the hypervisor statistics gatherer breaks the `flush_remote` hypervisor message into two categories:

- Requests to flush TLB entries, which do not include any cache flushes, are logged as `flush_remote:tlb`.
- Requests to flush caches, which may or may not also flush TLB entries, are logged as `flush_remote:cache`.

In some cases, tiles running the hypervisor can be interrupted by messages from other tiles, or by device interrupts; this is necessary to prevent deadlock, among other things. If two tiles each decided to send the other a `flush_remote` request, and refused to handle any simultaneous incoming messages until that request had been completed, a deadlock occurs. Time spent processing those message or handling those interrupts will be charged to those particular events, but will also be charged to whatever event caused us to enter the hypervisor in the first place.

The `IDN_AVAIL` and `INTCTRL_3` interrupts are used in the processing of incoming hypervisor messages; thus, there will be almost complete overlap between the time spent handling those interrupts and the time taken by all M events.

Many events happen during system startup, and not afterwards; some of these take significant amounts of time, and thus lead to large maximum cycle counts. When investigating performance issues, it may be wise to reset statistics counts before starting the target application, so that long delays experienced during startup do not dominate the results.

2.5.4 Resetting Hypervisor Statistics Gathering

You can reset all statistics by writing a NULL to a `hv_stats` file. For example:

```
# echo > /sys/devices/system/cpu/cpu0/hv_stats
```

2.6 Using Dedicated and Shared Tiles

Hypervisor device drivers can require that certain tiles be dedicated to their use. These tiles process I/O exclusively; they do not run the supervisor, and thus cannot run application programs. Very few of Tilera's standard drivers for devices in the TILE-Gx processor family require dedicated tiles.

Some drivers require a *shared tile* in place of, or in addition to, any *dedicated tiles* they might need. A shared tile provides a synchronization point through which I/O is performed. Unlike a dedicated tile, a shared tile can run the supervisor and therefore application programs. Several devices can use the same shared tile.

Tiles dedicated to I/O can in theory be located anywhere on the chip. For example, tiles (5,0), (5,4), (4,5), and (5,5), shown in gray in Figure 2-1, could be dedicated to I/O. In this case, the remaining tiles, shown in green, could all be used for the supervisor.

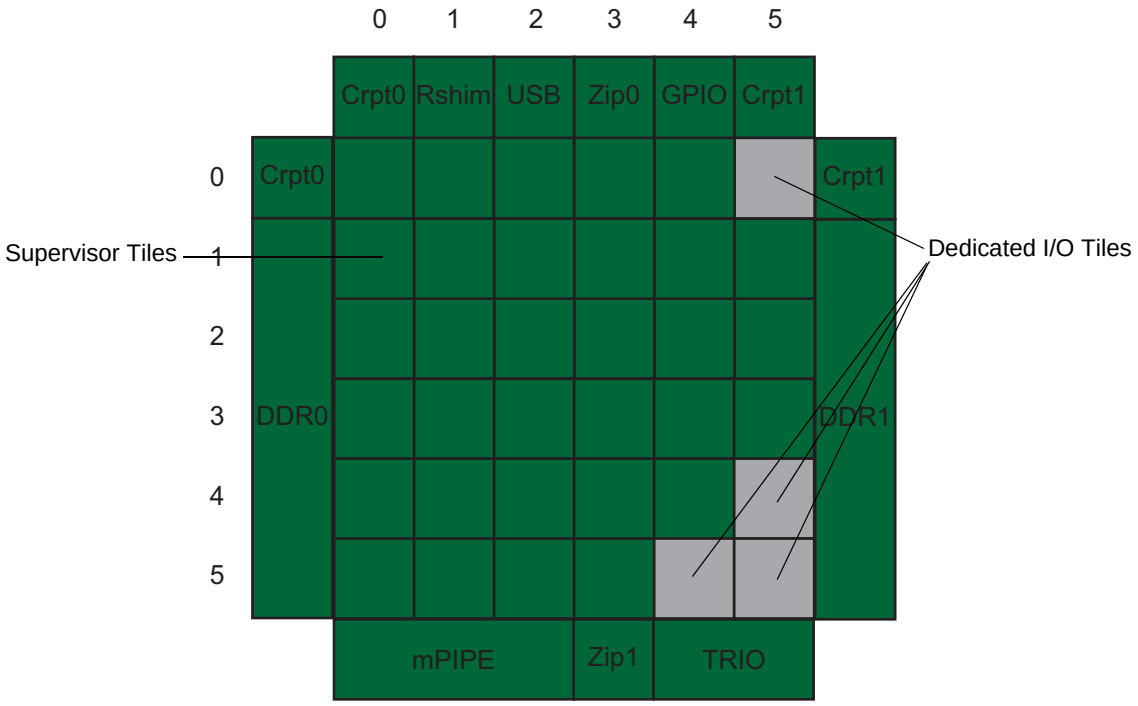

Figure 2-1: Tiles Allocated to a Supervisor

The full size of the hardware chip is specified in the `/sys/devices/system/cpu/chip_width` and `/sys/devices/system/cpu/chip_height` files. To obtain a complete picture of tiles available to Linux, you must combine the information in these two files with that in `/proc/cpuinfo`.

For more information, see Section 2.3.2 “The client Command” on page 22.

2.7 Controlling Memory Striping

By default, the Tile Processor implements *memory striping*, or memory accesses that can be automatically distributed among the memory controllers.

With memory striping, the booter and hypervisor arrange for memory to be striped across pairs or quartets of memory controllers. Linux will boot up believing it has a single memory controller that is larger than any of the actual physical memory controllers.

For applications that cannot easily segregate or load-balance memory traffic among the controllers, memory striping can provide a performance advantage.

To control memory striping, use the hypervisor’s `stripe_memory` option. See Section 2.3.1 “The options Command” on page 18.

2.8 Compiling Conditionally to Change Hypervisor Options

While most hypervisor options are enabled or disabled by means of the hypervisor configuration file, a few options are set as compile-time options. This section describes these options.

In general, these compile-time options are for items that must be configured very early in the boot process, before the hypervisor is able to parse the configuration file. The prebuilt hypervisor and booter that are distributed as part of the MDE use a set of compile-time options that have been chosen to meet the needs of most customers who are using Tilera's board and system products. However, if these options are not optimal for your application, you might need to change them.

To modify these options, you must rebuild the hypervisor from source, adding the appropriate `-D` options to the `EXTRA_DEFS` make variable.

For example, assume that you wanted to build a hypervisor that supported four client supervisors. To build a new hypervisor in the MDE tree and then run it on a Tilera PCIe card:

```
$ cd $TILERA_ROOT/src
$ make -C sys EXTRA_DEFS="-DMAX_CLIENTS=4"
$ tile-console
```

The various generated hypervisor components will be written to `sys/hv/`.

To boot using these new components, use this command in another window:

```
$ $TILERA_ROOT/bin/tile-monitor --hv-bin-dir $TILERA_ROOT/src/sys/hv
```

Note: These features and the compile-time options are subject to change in future releases.

2.8.1 Power-on Self-Test Configuration Options

The options in Table 2-1 affect the operation of the power-on self test (POST) operation that is performed during system boot.

Table 2-1. Hypervisor Compile-Time Options for POST Configuration

Option	Description
`POST_VERBOSE`	When this symbol is defined, POST prints informational messages (for example, when it starts to test a particular component). Normally, no output is produced unless POST detects an error.
`POST_MODE`	You can set this symbol to one of three values, which affect how the system runs POST tests at boot time: • `POST_QUICK` specifies that thorough tests will never be performed. • `POST_QUERY` specifies that the system prints a message on the serial console asking whether thorough tests should be performed or not; if you type q into the console within three seconds, the system performs quick tests, otherwise, thorough tests occur. • `POST_THOROUGH` specifies that the system always performs thorough tests. The default is `POST_QUERY`.

2.8.2 PCIe Configuration Options

The options in Table 2-2 affect how the Tile Processor's PCIe interfaces identify themselves to a host system. These interfaces typically use a built-in set of default identification information, which is made available to the host system by means of PCIe configuration space accesses; for example, the vendor ID is set to Tilera's PCI-SIG assigned vendor ID, 0x1A41. By using these options, the PCIe interfaces can be configured to supply a customized set of identification data.

For this to work, your system must boot the Tile Processor by means of an interface other than PCIe; for instance, the SROM. If the chip is booted by means of PCIe, the host system will sense the built-in PCI identification information before the booted hypervisor can change it.

For TILE-Gx36 devices, the custom PCIe device identification data is programmed into the pre-booter on the SROM. To generate the new pre-booter, run the following commands:

```
cd $TILERA_ROOT/src
make -C sys EXTRA_DEFS="-DPCIE_ID=0xdevice_id_vendor_id \
-DPCIE_SUBSYSTEM=0xsubsys_idsubsys_vendor_id"
```

Create the `PCIE_ID` and `PCIE_SUBSYSTEM` values as indicated in Table 2-2.

Table 2-2. Hypervisor Compile-Time Options for PCIe Configuration

Option	Description
`PCIE_ID`	When this symbol is set to a 32-bit numeric value, it overrides the default vendor ID and device ID. The resulting vendor ID comes from the low 16 bits of this value, while the device ID comes from the high 16 bits.
`PCIE_SUBSYSTEM`	When this symbol is set to a 32-bit numeric value, it overrides the default subsystem vendor and device IDs. The new subsystem vendor ID comes from the low 16 bits of this value, while the subsystem ID comes from the high 16 bits. **Note:** The Tilera PCIe drivers use the high 8 bits of the subsystem ID for use in communication between the tile-side driver and the host-side driver, so any value set there is overwritten. If you want to use Subsystem ID, please contact Tilera.

2.8.3 Miscellaneous Option

The option in Table 2-3 applies to Multiple Client Supervisor (MCS) support.

Table 2-3. Hypervisor Compile-Time Miscellaneous Option

Option	Description
`MAX_CLIENTS`	Specifies the number of clients you want to support for Multiple Client Supervisor (MCS) support. Because each client needs at least one tile, the maximum reasonable value for this option is 64.

2.9 Supporting Multiple Client Supervisors

Tilera's standard configuration files contain only one client. You can use the Multiple Client Supervisor (MCS) feature to enable multiple client support.

MCS allows more than one instance of a client supervisor (for example, Linux) to run simultaneously on a Tile Processor. Each instance has dedicated resources (tiles, memory, I/O devices) allocated to it, and has access to a shared console.

In order to enable MCS, you must build a new hypervisor from the source files provided with the release. (See Section 2.8 "Compiling Conditionally to Change Hypervisor Options" on page 29.)

As part of the build, you must set the `MAX_CLIENTS` macro to the number of clients you want to support. See Section 2.8.3 "Miscellaneous Option" on page 30.

See also Appendix A— Hypervisor Console Escape Commands.

2.9.1 Client Types

The first client specified in the hypervisor configuration is termed the primary client, and has certain special properties documented in the following sections.

2.9.2 Device Assignment

Currently, on-chip devices, such as an mPIPE or a TRIO or interface, can only be used by one client. (The console serial port is an exception; support for sharing it between clients is detailed in a later section.) The hypervisor configuration file controls which client owns which devices, by means of the `device` command within each client stanza.

The `device` command lists the name of one device (for example, `mpipe/0` or `trio/1`. The client in whose stanza the `device` command appears will be designated as that device's owner, and will have exclusive access to it. If a device is not named in any `device` commands, it will be owned by the primary client.

For more information, see Section 2.3.3 "The device Command" on page 23.

2.9.3 Memory Allocation

By default, each client is given an equal share of the available memory on each of the available memory controllers. (Due to Linux requirements, memory is given to clients in 16 MB chunks, so some memory might be lost because of rounding.) If this equal allocation is not desirable, you might want to use the `memory` command in the client stanza. See Section "The memory Subcommand" on page 22.

Each argument to the `memory` subcommand is the amount of memory, in bytes, to be allocated to a particular client on one of the chip's memory controllers (two controllers and thus two arguments on the TILE-Gx36).

The values might contain a trailing `k`, `m`, or `g` to mean kilobytes, megabytes, or gigabytes, respectively. Also, a value can be `default`, which means to use the normal equal allocation for this client on that controller. If some clients have explicit allocations for a certain controller and others do not, the explicit allocations are satisfied first, and the remaining memory is then divided among the rest of the clients.

If memory is striped, then pairs or quartets of memory controllers can be merged into a single logical controller, so there will be fewer arguments to the `memory` subcommand. Memory striping is only possible on processors with a uniform memory configuration. It is enabled by default in Tilera's distributed hypervisor configuration files, using `stripe_memory` with the `options` command. See Section 2.3.1 "The options Command" on page 18 for more information.

2.9.4 Shared Console

In an MCS configuration, the serial console is shared between all clients. The model for this is: at any one time, exactly one client is connected to the physical console. If that client emits console output, that output will be emitted from the serial port. And, if characters are input to the serial port, those characters will be provided as input to the currently connected client.

If a client that is not connected to the console emits output, that output is buffered until it is connected to the console, at which time all buffered output is emitted from the serial port. Each client's console output buffer is of finite size (currently 4 K bytes); if a client emits more than a buffer's worth of output while it is not connected to the console, the oldest output is lost.

You can change which client is connected to the console at any time, by means of a set of console escape commands. Note that output from the hypervisor itself, or from BME clients, is not buffered, and is always written immediately to the serial port.

2.9.5 Example Using MCS

This example configuration file, with 36 clients, demonstrates a near-maximum configuration.

Hypervisor file for this example

```
options stripe_memory=silent

options default_shared=5,5

client vmlinux 1x1 0,0
client vmlinux 1x1 1,0
client vmlinux 1x1 2,0
client vmlinux 1x1 3,0
client vmlinux 1x1 4,0
client vmlinux 1x1 5,0

client vmlinux 1x1 0,1
client vmlinux 1x1 1,1
client vmlinux 1x1 2,1
client vmlinux 1x1 3,1
client vmlinux 1x1 4,1
client vmlinux 1x1 5,1

client vmlinux 1x1 0,2
client vmlinux 1x1 1,2
client vmlinux 1x1 2,2
client vmlinux 1x1 3,2
client vmlinux 1x1 4,2
client vmlinux 1x1 5,2

client vmlinux 1x1 0,3
client vmlinux 1x1 1,3
client vmlinux 1x1 2,3
client vmlinux 1x1 3,3
client vmlinux 1x1 4,3
client vmlinux 1x1 5,3

client vmlinux 1x1 0,4
client vmlinux 1x1 1,4
client vmlinux 1x1 2,4
client vmlinux 1x1 3,4
client vmlinux 1x1 4,4
client vmlinux 1x1 5,4
```

```
client vmlinux 1x1 0,5
client vmlinux 1x1 1,5
client vmlinux 1x1 2,5
client vmlinux 1x1 3,5
client vmlinux 1x1 4,5
client vmlinux 1x1 5,5
```

2.10 Enabling the Second UART Device

You can access the UART1 device from the Linux kernel by enabling the `TILE_GXIO_UART` and `SERIAL_TILEGX` configurations in the Linux kernel. If you do not enable these configuration options, you can only access the UART1 device through user-space calls to the GXIO API. Tilera provides you with kernel object modules for the serial core and TILE-Gx serial drivers. By default, the Tilera kernel enables the `TILE_GXIO_UART` and configures the related `SERIAL_TILEGX` as a loadable module.

Note: In `menuconfig`, you reach the `TILE_GXIO_UART` configuration through "Tilera-specific configurations" > "Tilera Gx UART I/O support" and the `SERIAL_TILEGX` configuration through "Device Drivers" > "Character devices" > "Serial drivers" > "TILE-Gx on-chip serial port support".

These modules work in concert with the section in the Hypervisor configuration file `$TILERA_ROOT/tile/etc/hvc/devices.hvh`:

```
# Optionally enable the second on chip UART.
if defined(UART_1) && $UART_1
device uart/1
endif
```

The `uart1` device is only enabled when you define `UART_1` through a hypervisor definition.

To use the second UART, you load the modules as follows:

1. Load the serial core module, using this command:

   ```
   sh-4.1# insmod /lib/modules/version/kernel/drivers/tty/serial/serial_core.ko
   ```

2. Load the TILE-Gx serial driver, using this command:

   ```
   sh-4.1# insmod /lib/modules/version/kernel/drivers/tty/serial/tilegx.ko
   ```

3. To enable the UART1 device definition, use the `tile-monitor` command with the `--hvd UART_1` argument.

2.11 Building the Hypervisor

Any changes that you have made to the default hypervisor are not useful until you compile them and include them in a new bootrom for the TILE-Gx device. To build the hypervisor, follow these steps:

1. Change to the directory hypervisor build directory:

   ```
   $ cd $TILERA_ROOT/src/sys
   ```

Build the hypervisor, using the command:

```
$ make
```

This command creates a hypervisor binary as `$TILERA_ROOT/tile/boot/hv` and a new version of the `vmlinux` binary as `$TILERA_ROOT/tile/boot/vmlinux`. You can point the `tile-monitor` and `tile-mkboot` commands at these new files to create bootroms. See Chapter 4: Making a Bootrom.

CHAPTER 3 LINUX INTERFACES AND CUSTOMIZATION

This chapter describes:

3.1 Building Linux from Source

You can rebuild Linux from the source distribution included in the MDE.

Note: The `$TILERA_ROOT/src/` directory contains two `linux-*.get` scripts. The older version gets the source for the baseline supported kernel, and the newer version is provided on an experimental, unsupported basis.

In these directions, we assume the following conventions:

- `${LINUX_DIR}` is the Linux source directory, where the `.get` script places the source files.
- `${OBJ_DIR}` is the empty directory that you create in which you build the new Linux files. You call the `tile-prepare` script to run on the `${OBJ_DIR}` directory.

To start building Linux, follow these steps:

1. If you are cross-compiling from an Intel host, make sure you have the `TILERA_ROOT` environment variable properly set to the correct architecture directory of your MDE distribution (the directory containing, for example, the `tile/` and `bin/` subdirectories). You can use the `tile-env` script to set this environment variable. If you are building Linux natively on a TILE-Gx system, you do not need to set `TILERA_ROOT`.
2. Create a new, empty directory to be the parent of `${LINUX_DIR}`, for instance:

   ```
   mkdir /opt/tilera/projects/my_linux_src
   ```
3. Use `cd` to change to that directory:

   ```
   cd my_linux_src
   ```
4. Run the script `$TILERA_ROOT/src/linux-3.10.42.get`.

By default, the script downloads the appropriate kernel sources from `kernel.org` and creates the directory `linux-3.10.42`, which is `${LINUX_DIR}`. If you already have the kernel sources, then you can bypass this step by setting the environment variable `DOWNLOAD_CACHE` to point to a directory containing `linux-3.10.42.tar.xz`. In either case, the script then applies Tilera-specific patches to the kernel sources. In this example, the `$TILERA_ROOT/src/linux-3.10.42.get` script places the downloaded Linux source file in the directory `my_linux_src/linux-3.10.42`.

5. Create a new, empty directory to be the `${OBJ_DIR}`, for instance:

```
cd ${LINUX_DIR}
make ARCH=tilegx O=${OBJ_DIR} defconfig
```

Note: It is particularly important to specify the `ARCH=` variable when cross-compiling, in order to build for the correct target platform.

6. Initialize the build directory using the default configuration for the target. For example, assuming you're building Linux for TILE-Gx:

```
cd /opt/tilera/projects/new_linux
```

You can invoke your preferred Linux option configuration tool to modify the Linux configuration. For example, you can invoke any of the make commands shown below to update the build's `.config` file:

```
cd ${LINUX_DIR}
make ARCH=tilegx O=${OBJ_DIR} menuconfig   # CLI menu-based configuration
make ARCH=tilegx O=${OBJ_DIR} gconfig      # GTK-based GUI configuration
make ARCH=tilegx O=${OBJ_DIR} xconfig      # QT-based GUI configuration
```

For more information, see the section "CONFIGURING the kernel:" in the `${LINUX_DIR}/README` file.

7. To run the Linux build, use the following commands:

```
cd ${LINUX_DIR}
make O=${OBJ_DIR}
```

Note: It is not necessary to specify `ARCH=`, since the configuration process has already defined the architecture in the build makefile.

8. The result of the build process is a `vmlinux` output file, which is found here:

`${OBJ_DIR}/vmlinux.`

If you later decide you need to rebuild Linux with different options, for example to enable a new driver, you can do that by repeating steps 6 and 7, that is, re-run the configuration tool and then re-run the Linux build.

Warning! If you change configuration options and rebuild Linux, you should make sure updated kernel modules are also available at boot time. For example, if you are using the `tile-monitor --rootfs` option or equivalent to boot into a disk-based filesystem, you might want to update the kernel modules on disk prior to rebooting.

3.1.1 Ensuring Equivalency Between Linux Kernel and Modules

If you are building a loadable kernel module, in order to successfully load the module into Linux, you must ensure that the version string for the module is the same as the Linux from which you boot the TILE host. The simplest way to ensure this is to configure/build the Linux source from the same MDE that you use to boot the TILE host.

To determine if your modules have the same Linux version string (sometimes referred to as the *version magic string*) as the Linux running on the TILE host, you could use commands such as this:

```
# cat /proc/version
Linux version 3.10.32-MDE-4.3.beta2.176174 (support@tilera.com) (gcc version
4.4.7 20131017 (Tilera) (GCC) ) #1 SMP Sat May 24 16:22:45 EDT 2014

# modinfo ./tile/lib/modules/3.10.32-MDE-4.3.beta2.176174/kernel/net/sctp/
sctp.ko | grep vermagic
vermagic:    3.10.32-MDE-4.3.beta2.176174 SMP mod_unload tilegx 64KB
```

3.1.2 Building Third-Party Packages from Source

We generally provide third-party packages by using `*.get` scripts, which, when run in a given working directory, download the open-source tarball (or SRPM) for the particular package (if needed), extract it into a source subdirectory with a similar name, and then apply the patch that is contained in the `*.get` script. You can build the resulting sources by running `configure` and `make` commands.

If you need to re-build some of the other source code that we distribute (such as `binutils`, `busybox`, `gcc`, `gdb`, `glibc`, `newlib` and `ptpd`), do the following:

1. Create a directory to hold the source code (or use `$TILERA_ROOT/src`).
2. Run the `*.get` script provided for the third-party package in the `$TILERA_ROOT/src` directory.
3. Unpack and patch the source code. See whatever `README` is available for details.
4. Create an empty directory and `cd` into it.
5. Run the `tile-prepare` script from the third-party package.
6. To re-configure the third-party package prior to building, run the `make menuconfig` command.
7. Run the `make` command the build from source.

For further information, see the file `$TILERA_ROOT/src/README`.

3.1.3 Cross-Compiling the Linux Kernel with Modules

In order build the Linux kernel for Tile processors natively, you must run the following commands to include the new kernel modules:

```
make
make modules
make install
make modules_install
```

If you are cross-compiling the Linux kernel for Tile processors, you must build the `modules_install` target by pointing to the `$TILERA_ROOT/tile` directory. To do that, use the following commands:

```
make
make modules
make install
INSTALL_MOD_PATH=$TILERA_ROOT/tile make modules_install
```

3.1.4 Using a Re-Built Linux Kernel with tile-monitor

To use the new `vmlinux`, you must do one of the following:

- Boot your custom Linux image using `tile-monitor` with `--vmlinux`:

  ```
  $ tile-monitor --vmlinux $OBJ_DIR/vmlinux
  ```

  The default Linux image is found in the MDE at `$TILERA_ROOT/tile/boot/vmlinux`.
- Create a new `bootrom` file so that the new `vmlinux` can be booted onto the chip. For example, in the sample `tile-mkboot` script in Section 4.4 "Creating a Temporary Bootrom File Using tile-monitor" on page 67, replace `tile/boot/vmlinux` with the full path to your newly-built `vmlinux` image.
- If you are using `mboot`, boot directly from your custom Linux image. See Chapter 5: Using the mboot Boot Loader.

Note: You can build the `mboot` Linux (shipped in the MDE as `tile/boot/vmlinux_mboot`) using a similar version of this process. Build the Linux image with an embedded initramfs, as discussed in Section 3.4.2 "Adding User Files to the Initramfs Boot Image" on page 58. Build the initramfs from the files in the Tile file hierarchy in `/usr/lib/initramfs/mboot/contents.txt` (under the `$TILERA_ROOT/tile/` directory in the cross-build environment). The `busybox` found in `/usr/lib/*` is built from the `mboot/tile-defconfig` file in the `busybox` source code and can be rebuilt as is normal for `busybox`. See `$TILERA_ROOT/src/README` for more information on rebuilding `mboot` itself.

3.1.5 Using Cache Packing with Linux

As discussed in the feedback optimization chapter of the *Multicore Development Environment Optimization Guide* (UG515), it is possible to improve performance by passing run-time information back to the compiler and linker. You can use this technique when building Linux as well.

1. The first step is to gather the run-time information. To do so, set `CONFIG_FEEDBACK_COLLECT=y` and `CONFIG_FEEDBACK_USE="`*`feedback_file`*`"` in your Linux configuration file, `make clean` and `make`.

   The resulting `vmlinux` binary collects run-time information while it is running (though at some cost in performance).

2. Boot the Linux you have just built and run appropriate training data. A Linux built with feedback collection enabled has a `/proc/tile/feedback` file that allows on-demand access to the run-time information gathered by the running kernel.

3. When you have run your application long enough to gather the run-time information you want, download the `feedback` file (`/proc/tile/feedback`) to a suitable file on your Intel host, for example `feedback.raw`, and then convert it to a proper feedback file with, for example:

   ```
   tile-convert-feedback -o /path/to/vmlinux.fb -f feedback.raw
   ```

   If compiling natively, you can copy `/proc/tile/feedback` to a local `feedback.raw` file, and run `tile-convert-feedback` on it just as you would if cross-compiling.

4. Now, rebuild Linux using the feedback file you have just generated. In your Linux configuration file, comment out `CONFIG_FEEDBACK_COLLECT` again, then set `CONFIG_FEEDBACK_USE="/path/to/vmlinux.fb"`. Rebuild Linux by means of `make clean` and `make`.

   The compilation of the source files makes use of the run-time feedback information for basic block ordering, unrolling, branch prediction, and the like. In addition, the link step is augmented with a cache-packing phrase. This phrase places all the functions at memory addresses that should cause as few conflicts in the cache as possible.

Note: Some compiler warnings about "Feedback data inconsistency" are normal.

3.2 Configuring Linux

In general, Tilera supports only the distributed kernel configuration, but some configuration options are more likely to work than others, and some are known not to be supported.

The purely architecture-independent options are likely to work. These include file system types, networking configuration options (IPv6, alternate protocols on top of Ethernet, and so forth), I/O scheduler types, cryptography, security, and so forth.

Tile Processor architecture-specific options that are configured in the `arch/tile/` tree generally should not be modified, because the kernel might not work when they are disabled.

The `usr/contents.txt` file in the build directory can be updated with other files to include in the initial `ramfs` as needed.

3.2.1 Devices and Drivers

The default configuration includes the following devices:

- `/dev/bme/mem` — communication with BME applications over shared memory.
- `/dev/console` — system console
- `/dev/fuse` — access to the host file system by means of the `shepherd` and `tile-monitor`.
- `/dev/hda/*` — compact flash drive if enabled and installed
- `/dev/tilegxpci`*`N`*`/*` — The following PCIe driver interfaces include the following:
  - `0-3` — PCIe bidirectional character stream devices on the Linux x86 host machine in which a TILEncore-Gx Intelligent Application Adapter is resident, including 4 individual channels multiplexed over the PCIe link to the PCIe host, numbered 0 to 3.
  - `barmem` — PCIe Base Address Register 1 (BAR1) mapping interface
  - `boot` — PCIe boot interface
  - `ctl` — IOCTL interface
  - `h2t/*` — host-to-tile zero-copy command queues
    - `info` — PCIe link information
  - `lock` — per-process lock for boot
  - `packet_queue/*` — zero-copy packet queues
  - `rshim/*` — Rshim access interface
  - `t2h/*` — tile-to-host zero-copy command queues
- `/dev/trio`*`N`*`-mac`*`M`*`/*` — The following PCIe devices exist under `/dev/trio`*`N`*`-mac`*`M`* on a TILEncore-Gx Intelligent Application Adapter:
  - `0-3` — PCIe bidirectional character stream devices on the Tile side, including 4 individual channels multiplexed over the PCIe link to the PCIe host, numbered 0 to 3.
  - `barmem` — PCIe Base Address Register 1 (BAR1) mapping interface
  - `host_nic/*` — PCIe host NIC interface
  - `host_pq/*` -- zero-copy packet queues for channel status management
  - `h2t/*` — host-to-tile zero-copy command queues
  - `t2h/*` — tile-to-host zero-copy command queues

- /dev/hugepages/* — file system for creating huge page memory areas
- /dev/null — the null device, typically to discard data written to it
- /dev/ptmx — pseudo-terminal master
- /dev/pts/* — pseudo-terminal slave devices
- /dev/sda/* — SATA disk, if installed
- /dev/srom/* — partitions on the SROM device
- /dev/tty — controlling terminal of a process
- /dev/urandom — kernel random number source device
- /dev/zero — reads return zero-valued data, writes are discarded

3.2.2 The /proc Virtual File System

The /proc virtual file system provides access to a wide variety of system information, including information about individual processes.

Tile Linux provides all standard Linux 3.4 /proc entries and also provides these additional directories:

- /proc/driver/
- /proc/tile/
- /proc/sys/tile/

/proc/driver/ directory

The /proc/driver/ includes the following Tilera-specific virtual files:

- gxpci_rc_major_mac_link_domain — This file is available on a TILEmpower-Gx appliance operating as a PCIe root complex node, and reports the major device number, the TRIO instance number *N*, and the PCIe port number *M*, the link index in the PCI domain to which this port belongs, and the PCI domain number.
- gxpci_ep_major_mac_link — This file is available on a TILEncore-Gx Intelligent Application Adapter operating as a PCIe endpoint node, and reports the major device number, the TRIO instance number *N*, and the PCIe port number *M*, and the link index in the PCI domain to which this port belongs.
- rtc — Reports the values for the Real Time Clock driver, including rtc_time, rtc_date, and 24hr mode (yes or no).

/proc/tile/ directory

The /proc/tile/ includes the following Tilera-specific virtual files:

- environment — Reports the approximate chip and board temperatures in Celsius. Note that not all platforms have the required support for this file, and some platforms might have unique mechanisms regarding chip and board temperatures.
- hardwall — Directory containing entries for each iMesh hardwall currently allocated on the system. Normally, the Tilera run-time libraries handle this internally. See the manpage for hardwall for details.
- interrupts — Displays the count of interrupts, by type, that have been delivered to each tile since boot.

  The various interrupt types that are tracked in this file are:

  - `timer` — The number of hardware timer interrupts. These are caused primarily by process scheduling and high-resolution timer interrupts. Idle tiles and dataplane tiles generally do not see timer interrupts.
  - `syscall` — The number of system calls made by user processes on this CPU. The atomic instruction emulation code uses "fast syscalls," which are not counted by this mechanism because they do not cause any significant amount of kernel code to be run.
  - `resched` — The number of reschedule interrupts delivered by other CPUs to this CPU, to cause rescheduling so that waiting tasks can wake up when appropriate.
  - `hvflush` — The number of "hypervisor flush" interrupts delivered by other CPUs to this CPU to cause TLB or cache flushing.
  - `SMPcall` — The number of generic *remote procedure call* interrupts sent by other CPUs to this CPU to handle running code on multiple CPUs for kernel purposes.
  - `hvmsg` — The number of hypervisor message interrupts delivered to this CPU. These are used by some Linux drivers.
  - `devintr` — The number of hypervisor device interrupts delivered to this CPU. These are used by most Linux devices to implement the standard Linux "IRQ" model for device interrupts.
- `memory` — Reports the memory speed of each controller in bytes per second, and the mapping between Linux NUMA memory nodes and the physical memory controllers.
- `memprof` — Memory profiling interface.

/proc/sys/tile/ directory

The `/proc/sys/tile/` includes the following Tilera-specific virtual files:

- `hash_default` — This file is only present when `mmap()` and other memory-allocating functions return hash-for-home pages by default. This is true when the `ucache_hash` (or `cache_hash)` boot option is set to `all` or `allbutstack`, as is true by default. See the `ucache_hash` boot option in Table 3-1.
- `userspace_perf_counters` -- Sets Multiprogramming Level (MPL) for `PERF_COUNT`. To provide user-space access to performance counters, set the entry to `1` (thus setting the MPL to user protection level). To run `perf_event` or `oprofile`, set the entry to `0` (thus setting the MPL to Linux kernel protection level). Default is `0`.
- `unaligned_fixups/count` — The number of unaligned "fixups" performed by the kernel since boot.
- `unaligned_fixups/enabled` — Whether unaligned fixups are enabled (1), disabled but still intercepted by the kernel to provide the unaligned address information with the SIGBUS signal (0), or completely disabled so that no SIGBUS address information is provided to the user (-1). See also the `unaligned_fixup` boot option in Table 3-1.
- `unaligned_fixups/printk` — 0 or 1 to indicate whether each unaligned fixup should cause a message to be displayed on the console. See also the `unaligned_printk` boot option in Table 3-1.
- `homecache/migrated_mapped_pages` — The number of memory pages that are mapped into user memory whose home cache has been changed.
- `homecache/migrated_tasks` — The number of tasks that have been migrated from one CPU to another.

- homecache/migrated_unmapped_pages — The number of memory pages that are not mapped into user memory whose home cache has been changed.
- homecache/sequestered_pages_at_free — The number of pages that have been sequestered at kernel memory allocation time (typically due to dataplane restrictions).
- homecache/coherent_sequestered_purges — The number of times the list of sequestered pages has been partially drained by returning all coherently cached pages to the generic Linux allocator.
- homecache/incoherent_finv_pages — The number of times the OS had to do a flush-and-invalidate sequence on a page of memory, for every core, rather than the normal once through the page.
- homecache/sequestered_purges — The number of times the list of sequestered pages has been drained by doing a cache flush on every CPU.

/proc/sys/debug/exception-trace file

The /proc/sys/debug/exception-trace file is enabled for the TILE Architecture. This replaces /proc/sys/tile/crashinfo from previous versions of the MDE.

A value of 1 (the default) generates a one-line kernel message when user processes crash.

A value of 0 disables all messages.

A value of 2 generates more substantial debug information for each user process crash.

3.2.3 The /sys Virtual File System

The /sys virtual file system provides access to a wide variety of system information, including information about individual processes.

Tile Linux provides all standard Linux 3.4 /sys entries and also provides these additional directories:

- /sys/devices/system/comp/
- /sys/devices/system/cpu/
- /sys/hypervisor/

/sys/devices/system/comp directory

The /sys/devices/system/comp directory includes the following Tilera-specific virtual files:

- comp0 — The primary MiCA compression shim. This directory includes as reset file. Writing a 1 to this device file causes the hypervisor to reset the MiCA compression shim. For details, see Section 3.8.4 “Resetting MiCA Compression Shims Using sysfs” on page 62.

/sys/devices/system/cpu directory

The /sys/devices/system/cpu directory includes the following Tilera-specific virtual files:

- chip_height — The number of tiles high that the TILE-Gx processor has in it’s matrix.
- chip_width — The number of tiles wide that the TILE-Gx processor has in it’s matrix.
- chip_serial — The serial number of the TILE-Gx processor.
- chip_revision — The revision number of the TILE-Gx processor.
- dataplane — Which tiles are currently configured as dataplane tiles and thus running ZOL on the TILE-Gx processor.

/sys/devices/system/cpu/cpu*N*/cpufreq directory

The `/sys/devices/system/cpu/cpu`*N*`/cpufreq` directory, (where *N* can be 0 through 35 for a TILE-Gx36™ processor and 0 through 71 for a TILE-Gx72™ processor), includes the following Tilera-specific virtual files:

- `cpuinfo_cur_freq` — The currently set chip frequency on the TILE-Gx processor, in kHz.
- `cpuinfo_max_freq` — The maximum possible chip frequency on the TILE-Gx processor, in kHz.
- `cpuinfo_min_freq` — The minimum possible chip frequency on the TILE-Gx processor, in kHz.
- `scaling_setspeed` — If you configure the hypervisor to use Dynamic Voltage and Frequency Scaling (DVFS), you can set the processor core frequency by writing a new frequency value (expressed in kHz units) into this file.

  Note: Since there is only one clock speed for the entire TILE-Gx chip, changing the speed for any CPU changes the speed of all CPUs.

For more information about enabling DVFS, see Table 3-1.

/sys/hypervisor directory

The `/sys/hypervisor` directory includes the following Tilera-specific virtual files:

- `config_version` — Information about the current build of the Hypervisor (who built it, when, from where, using what parameters/options)
- `hvconfig` — The current configuration settings for the running Hypervisor.
- `type` — The type of hypervisor that you are running. The default is `tilera`.
- `version` — A version number for the hypervisor that you are running.

/sys/hypervisor/board directory

The `/sys/hypervisor/board` directory includes the following Tilera-specific virtual files:

- `board_description` — A short description of the board.
- `board_part` — The part number for the board.
- `board_revision` — The revision number for the board.
- `board_serial` — The serial number for the board.
- `cpumod_description` — A short description of the CPU module installed on a board.
- `cpumod_part` — The part number for the CPU module installed on a board.
- `cpumod_revision` — The revision number for the CPU module installed on a board.
- `cpumod_serial` — The serial number for the CPU module installed on a board.
- `mezz_description` — A short description of the mezzanine board which is by default is a Tilera 4x10G I/O Module.
- `mezz_part` — The part number for the I/O module.
- `mezz_revision` — The revision number for the I/O module.
- `mezz_serial` — The serial number for the I/O module.

- `switch_control` — On a platform which is equipped with a Marvell switch, this file contains something like `control: mdio gbe/0` which indicates the Management Data Input/Output (MDIO) interface for that switch. This file is empty on any platform which does not contain a Marvell switch.

3.2.4 Other Facilities

Tilera provides NUMA APIs for controlling access to the multiple memory controllers that can be attached to the chip. In particular, this means up to four NUMA regions can be defined and accessed with `mbind()`, `set_mempolicy()`, and so on. By default, the memory controllers are "striped" to provide a single virtual memory controller. For information on how to manage memory striping, see Section 2.7 "Controlling Memory Striping" on page 28.

Tilera also provides standard SMP APIs for controlling scheduling to different cores. As is the case for Linux in general, the kernel automatically schedules processes and threads to run on idle cores. However, you might use the SMP APIs, such as `sched_setaffinity()`, in the normal manner to control how Linux assigns processes to run on specific cores. The TMC user library can be used to simplify binding processors to cores, using APIs such as `tmc_cpus_set_my_cpu()`.

Note: The Linux man page for the `sched_setaffinity` function implies that the set of available CPUs is contiguous starting with CPU ID 0. That is incorrect; no assumptions should be made about particular CPUs being available, or the set of CPUs being contiguous, because CPUs can be taken offline dynamically or be otherwise absent. In Tile Linux, dedicated hypervisor and bare metal environment (BME) tiles are always unavailable.

3.3 Linux Boot Arguments

When booting Linux, you can provide arguments on the Linux command line that are interpreted to provide particular kernel functionality.

The hypervisor configuration file normally allows a `client` stanza, which specifies a particular supervisor (kernel) to boot. If you choose a Linux kernel, typically with something like `client vmlinux`, an indented `args` line can follow, which specifies the boot arguments.

When booting the `tile-monitor`, you can specify multiple `-hvx ARG` arguments to `tile-monitor`. The hypervisor passes the `ARG` arguments to the booted Linux as boot arguments. For example, adding `-hvx debug` adds the standard `debug` boot argument to Linux.

Boot arguments are divided into the following primary types:

- Kernel parameters

  The kernel interprets these parameters.

- Initial environment variables

  All other options of the form `x=y` are presumed to be environment variables, and are available to the `/etc/rc` script, which runs at boot time.

- Command-line arguments for `init`

  `init` accepts the command-line argument `single` to boot in single-user mode; other arguments are largely ignored.

3.3.1 Tilera-Specific Linux Boot Arguments

The current set of valid Linux boot arguments that are specific to the Tile Processor™ are listed in Table 3-1.

Table 3-1. Tile-Specific Linux Boot Arguments

Argument	Values	Description
`cache_hash`		This option takes the same set of values as the `kcache_hash` option and the `ucache_hash` option. It has the effect of setting both of those two options to the specified value. For example, `cache_hash=allbut-stack` is equivalent to `kcache_hash=allbutstack ucache_hash=allbutstack`.
`crashinfo`		By default, a one-line log message for any user application crash is produced. To get a full stack trace, you can use the `crashinfo` option (either `crashinfo` without a value or `crash-info=2`). This option requests that the kernel dump information about crashing user programs to the console. The information includes details of the signal that caused the crash, the stack trace at the time of crash, shared-object information on userspace stack traces and, if appropriate, a dump of the region of memory near the crash address. If you boot with `crashinfo=1`, you get the default behavior (one-line message). If you boot with `crashinfo=0`, the one-line message is suppressed. Another way to get full crash information is by running the following: `sysctl -w debug.exception-trace=2`
`dataplane`	A set of tiles, for example: `17-22,25-30`. This value can be a single CPU or a range. (Note: spaces are not allowed.)	This option enables Zero Overhead Linux (ZOL) mode on a set of tiles for use by dedicated applications that wish to run with low latency, such as networking applications that process packets at 10 Gbps. This option isolates the specific tiles, similar to the `isolcpus` option, and arranges for Linux to protect a process running on that tile from Linux bookkeeping interrupts. For example, to configure all but the first CPU on a TILE-Gx8036 processor as `dataplane` CPUs, you can add `dataplane=1-35` to your Linux boot arguments. See Section 3.6 “Enabling Zero Overhead Linux (ZOL) With the dataplane Boot Argument” on page 60.

Table 3-1. Tile-Specific Linux Boot Arguments (continued)

Argument	Values	Description
`disabled_cpus`	A set of tiles, for example: `0-7`. This value can be a single CPU or a range.	This option takes an argument of a list of tiles to be considered “disabled” from Linux's point of view. Normally Linux treats any dedicated hypervisor tiles that are within its configured tile rectangle as “disabled”; this argument can be used to disable additional tiles if desired. For example, `disabled_cpus=0-7,16,32` would disable ten tiles on the grid.
`gxpci_endp.net_macs`	`gxpci_endp.net_macs=1`	This argument specifies the number of the TRIO interface to use with the PCIe host point-to-point driver.
`hardlockup`	`hardlockup=1`	This argument enables the hardlockup detection code. By default, the Linux kernel does not provide an argument to disable the hardlockup watchdog. To enable hardlockup detection, set the `hardlockup=1` boot option. The default value is 0 (disabled).
`isolnodes`		This option allows a user application to decide how each memory controller is used by the kernel. The argument is a comma-separated list of memory controllers and/or a hyphen-separated range of memory controller numbers. “Isolated” nodes are used only minimally by the kernel (typically around 1 megabyte) and are not the “default” memory controller for any tile. As a result, user applications using the `mbind()` routine to request these memory controllers can use almost all of the memory.
`kcache_hash`		This option allows the user to tell the kernel how to configure its own code, data, heap, and stack areas. Possible values are: `all` - Use hash-for-home for kernel memory. `allbutstack` - Use hash-for-home for all kernel memory except the kernel stack associated with each task, which is homed locally on the task's current CPU. `static` - Use hash-for-home for all kernel code and static data. `ro` - Use hash-for-home only for kernel code. `none` - Do not use hash-for-home by default at all. The default setting for this value is `allbutstack`. For more information on how to use this boot option, see *Programming the TILE-Gx Processor* (UG505).

Table 3-1. Tile-Specific Linux Boot Arguments (continued)

Argument	Values	Description
kdata		This option controls how the kernel maps kernel data.
	kdata=huge	Use huge pages to map the kernel data, using hash-for-home. This is the default. If you use kcache_hash to disable hash-for-home caching of the heap (specifying none, ro, or static), that setting disables huge pages. The kernel automatically converts kernel huge page mappings to small pages as needed.
	kdata=small	Use small pages to map the kernel data.
	kdata=CPULIST	In some cases the kernel caches each page of kernel data on a separate CPU's cache, effectively “striping” across those CPUs. This is true if booting with kcache_hash=none or kcache_hash=ro. In that case, by default the kernel uses all the available Linux tiles (minus dataplane tiles, if any) for striping. If you don’t want to use striping on all CPUs, use this option to specify the set of CPUs to use to stripe the kernel's static data. Specify the CPUs in a comma-separated list and/or hyphen-separated ranges. You can specify the small option with CPULIST, which both enables small pages and specifies the kernel data homecache CPUs. For example, to use small pages home-cached on CPUs 0 through 3, specify kdata=small,0-3.
ktext		This option controls how the kernel text (that is, the code) is cached. The default setting is huge. The kcache_hash option controls if the kernel code is cached by means of hash-for-home, which is the case by default. In this mode, only ktext=nocache can be used; the other options are only relevant if kcache_hash=none is specified.
	ktext=huge	Kernel text is cached directly from RAM using one huge page.
	ktext=local	Kernel text is cached directly from RAM using many small pages (not generally useful).
	ktext=all	Kernel text is cached both locally on the tiles, and also distributed in the caches of tiles on the chip. All tiles will be used to hold kernel text.
	ktext=CPULIST	For example, ktext=1,2,3,8-10 In this case, kernel text is cached locally and is also distributed in the caches of the specified tiles.
	ktext=nocache	If nocache is specified before the other ktext option, then no local caching is performed. For example, for kernel-intensive operations, a useful setting can be ktext=nocache,all. This caches only in the distributed set of tiles, and not in the local L2.

Table 3-1. Tile-Specific Linux Boot Arguments (continued)

Argument	Values	Description
`lockup_dumpall`	`lockup_dumpall=1`	Enables Linux to perform a backtrace on all tiles when a softlockup or hardlockup occurs. The default kernel condition provides only a backtrace dump for a single tile in the case of a softlockup or hardlockup. To provide more comprehensive backtrace dump information (from all tiles), set the `lockup_dumpall=1` kernel boot option. The default value is 0 (disabled).
`maxmem`		This argument arbitrarily limits the amount of RAM used by Linux to the specified value, with the unit value set by using a suffix of `k` for kilobytes, `m` for megabytes, or `g` for gigabytes. The cutoff is applied as memory controllers are examined; controllers past the limit are ignored, and if a controller's memory straddles the limit, only some of its RAM is used. For example, in a system with four 512MB controllers, setting `maxmem=1280m` results in using the first two controllers, and only 256 MB from the third controller.
`maxnodemem`		This provides a per-controller limit on the amount of RAM used. The value of the argument specifies a controller number followed by a colon and a memory size, with the unit value set by using a suffix of `k` for kilobytes, `m` for megabytes, or `g` for gigabytes. For example, specifying `maxnodemem=0:256m` tells Linux to only use 256 MB on controller 0. You can specify this boot argument multiple times, each time referencing a different controller.
`memmap`	An amount of memory and a starting physical address, for example: `'memmap=2G$6G'`.	Specifies that the kernel reserve the amount of memory starting at a particular starting physical address, using the format *`amount`*`$`*`start_addr`*, with the unit value set by using a suffix of `k` for kilobytes, `m` for megabytes, or `g` for gigabytes. The memory is reserved through `bootmem` during startup, and the TILE-Gx processor creates a suitable structure resource marked as `Reserved` so you can see the range reported in the file `/proc/iomem`. You can reserve up to 64 such regions on the boot command line. For example, to reserve a 1 GB range starting at Physical Address 5GB, use the `tile-monitor --hvx` hypervisor argument as follows: `--hvx 'memmap=1G\\\$5G'` You must include the triple “\” character because the “$” character is a variable reference operator in the hypervisor configuration file language. The hypervisor processes escapes in a --hvx argument twice. You must also enclose the hypervisor option declaration in single quotes. This Linux configuration option does not support syntax values with `@` or `#` directives, or the `exactmap` argument.

Table 3-1. Tile-Specific Linux Boot Arguments (continued)

Argument	Values	Description
noallocl2		The kernel will normally cache data locally on the tile that is accessing it, which has the potential of evicting data that some other tile has a copy of, thus reducing the overall amount of on-chip address space that can be cached. Specify this option to eliminate local caching, thus increasing somewhat the overall caching capacity of the chip. This is at the cost of somewhat higher latency for accesses that would otherwise be locally cached. Note: this option does not take a value.
nodma		This option prohibits user code from using the tile DMA facilities.
nosmp		This option is taken as shorthand for disabling all tiles except the boot tile.
noudn		This argument prevents the Linux kernel from accessing the UDN or allowing any user processes to use the UDN.
pcie_host_pq_h2t_queues	pcie_host_pq_h2t_queues=*T*,*P*,*M* *T* specifies the TRIO instance (0 is only legal value on TILE-Gx36 and TILE-Gx16). *P* specifies the port number (0 is only legal value on TILE-Gx36 and TILE-Gx16) *M* specifies the is the number of host-NIC interfaces. The default is 8. The range is 1-8.	This argument specifies the number of Host-to-Tile Packet Queue interfaces to use on the x86 Linux host which has a TILEncore-Gx Intelligent Application Adapter installed. To use these Host-to-Tile packet queues on the Tile-side, you must install the host driver tilegxpci using the pq_h2t_queues= argument, using the same values for *T*, *P* and *M* as for the pcie_host_pq_h2t_queues= Tile-side Linux boot argument. By default, there are 8 Tile-to-Host interfaces and 8 Host-to-Tile interfaces. You can configure an asymmetrical number of H2T and T2H Packet Queue interfaces. Due to TILE-Gx PCIe hardware constraints, the sum of Tile-to-Host queues and Host-to-Tile queues must not exceed 16.
pcie_host_pq_t2h_queues	pcie_host_pq_t2h_queues=*T*,*P*,*N* *T* specifies the TRIO instance (0 is only legal value on TILE-Gx36 and TILE-Gx16). *P* specifies the port number (0 is only legal value on TILE-Gx36 and TILE-Gx16) *N* specifies the is the number of Tile-to-Host packet queues. The default is 8. The range is 1-8.	This argument specifies the number of Tile-to-Host Packet Queue interfaces to use on the x86 Linux host which has a TILEncore-Gx Intelligent Application Adapter installed. To use these Tile-to-Host packet queues on the Tile-side, you must install the host driver tilegxpci using the pq_t2h_queues= argument, and specifying the same values for *T*, *P* and *Q* as for this pcie_host_pq_t2h_queues= Tile-side Linux boot argument. By default, there are 8 Tile-to-Host interfaces and 8 Host-to-Tile interfaces. You can configure an asymmetrical number of H2T and T2H Packet Queue interfaces. Due to TILE-Gx PCIe hardware constraints, the sum of Tile-to-Host queues and Host-to-Tile queues must not exceed 16.

Table 3-1. Tile-Specific Linux Boot Arguments (continued)

Argument	Values	Description
pcie_host_vnic_ports	pcie_host_vnic_ports=*T*,*P*,*M* *T* specifies the TRIO instance (0 is only legal value on TILE-Gx36 and TILE-Gx16). *P* specifies the port number (0 is only legal value on TILE-Gx36 and TILE-Gx16) *M* specifies the is the number of host-NIC interfaces. The default is 4. The range is 1-4.	This argument specifies the number of host-NIC ports to use as Ethernet ports on the x86 Linux host which has a TILEncore-Gx Intelligent Application Adapter installed. Default is 4 ports. To use these host-NIC ports on the Tile-side, you must install the host driver tilegxpci using the nic_ports= argument, using the same values for *T*, *P* and *M* as for the pcie_host_vnic_ports= Tile-side Linux boot argument.
pcie_host_vnic_rx_queues	pcie_host_vnic_rx_queues=*T*,*P*,*Q* *T* specifies the TRIO instance (0 is only legal value on TILE-Gx36 and TILE-Gx16). *P* specifies the port number (0 is only legal value on TILE-Gx36 and TILE-Gx16) *Q* specifies the is the number of receiver packet queues. The default is 1. The range is 1-10.	This argument specifies the number of receiver packet queues to use for each host-NIC port on the x86 Linux host which has a TILEncore-Gx Intelligent Application Adapter installed. For the default 4 host-NIC interfaces, you can set 1 or 2 receiver packet queues. If you configure only 1 host-NIC port, you may configure up to 10 receiver packet queues. To use these receiver packet queues on the Tile-side, you must install the host driver tilegxpci using the nic_rx_queues= argument, and specifying the same values for *T*, *P* and *Q* as for this pcie_host_vnic_rx_queues= Tile-side Linux boot argument. You must match the number of transmitter and receiver and packet queues.
pcie_host_vnic_tx_queues	pcie_host_vnic_tx_queues=*T*,*P*,*Q* *T* specifies the TRIO instance (0 is only legal value on TILE-Gx36 and TILE-Gx16). *P* specifies the port number (0 is only legal value on TILE-Gx36 and TILE-Gx16) *Q* specifies the is the number of transmitter packet queues. The default is 1. The range is 1-10.	This argument specifies the number of transmitter packet queues to use for each host-NIC port on the x86 Linux host which has a TILEncore-Gx Intelligent Application Adapter installed. For the default 4 host-NIC interfaces, you can set 1 or 2 transmitter packet queues. If you configure only 1 host-NIC port, you may configure up to 10 transmitter packet queues. To use these transmitter packet queues on the Tile-side, you must install the host driver tilegxpci using the nic_tx_queues= argument, and specifying the same values for *T*, *P* and *Q* as for this pcie_host_vnic_tx_queues= Tile-side Linux boot argument. You must match the number of transmitter and receiver and packet queues.

Table 3-1. Tile-Specific Linux Boot Arguments (continued)

Argument	Values	Description
`pcie_rc_delay`	pcie_rc_delay=*x*,*y*,*z* *x* specifies the TRIO instance (0 is only legal value on TILE-Gx36 and TILE-Gx16). *y* specifies the MAC instance (0,1,2) *z* specifies the bring-up delay in seconds. The default for *z* is 10 seconds.	This argument specifies a delay value that must elapse before the specified MAC within a specified TRIO instance brings up the PCIe Root Complex (RC) link. This argument has no effect if the MAC is configured as an End Point (EP) link. Use this argument to bring up the interconnecting links for the nodes on a TILExtreme-Gx platform. The setting `pcie_rc_delay=0,0` is equivalent to `pcie_rc_delay=0,0,10`.
`rootwait`		This option indicates that the specified device may not yet be ready when the kernel has finished its early initialization, and that the kernel should wait indefinitely for the device to become valid to complete booting. Use this option with the kernel independent `root=`*`partition`* option.
`tile_net.cpus`	A set of tiles, for example: `2-11`. This value can be a single CPU or a range.	This option restricts Linux network device interrupt processing to specific tiles. For example, to enable 16 network CPUs, you might add `tile_net.cpus=16-31` to configure CPUs number 16 through 31 as responsible for network interrupts. See Section 3.7 “Optimizing Linux Network Stack Performance With the tile_net.cpus Boot Argument” on page 60.
`tile_net.jumbo`	A number of jumbo buffers to create.	This option allows the user to specify the number of jumbo buffers to create at boot time. Each such buffer consumes 32KB of system memory. For example, to create 100 jumbo buffers at boot time, use `tile_net.jumbo=100`.

Table 3-1. Tile-Specific Linux Boot Arguments (continued)

Argument	Values	Description
ucache_hash		This option allows the user to tell the kernel how to configure the default use of hash-for-home caching for user processes' code, data, heap, and stack areas. Possible values are: all- Use hash-for-home by default for all types of user memory. allbutstack - Use hash-for-home by default for all types of user memory except the stack associated with each task, which is homed locally on the task's current CPU. static - Use hash-for-home by default for all user code and static data. ro - Use hash-for-home by default only for user code and read-only data. none - Do not use hash-for-home by default at all. The default setting for this value is allbutstack. The defaults can be overridden by means of the LD_CACHE_HASH environment variable or by specifying caching attributes explicitly. For more information on how to use this boot option, see *Programming the TILE-Gx Processor* (UG505).
unaligned_fixup		This option controls the use of kernel-managed unaligned access fix-ups. By default, unaligned fix-ups are performed: the kernel traps and handles memory access for loads and stores not aligned to the size of the load or store operand. If this option is set to 0, unaligned fixups are not performed; however, the kernel still traps and specially handles unaligned accesses, so it can provide the actual address of the unaligned access. Booting with this option set to -1 causes the kernel to perform no special handling at all, in which case the address of the unaligned access is not passed to the user code. This option can be changed after boot by means of the /proc/sys/tile/unaligned_fixups/enabled file.
unaligned_printk		Set this option to 1 to specify that every unaligned fix-up triggers a kernel console message. By default (or if you specify 0), only the first unaligned fix-up triggers a kernel console message. This option can be changed after boot by means of the /proc/sys/tile/unaligned_fixups/printk file.

3.3.2 Platform-Independent Linux Boot Arguments

Linux provides many platform-independent boot arguments, most of which should function on the Tile Processor as well.

A few particularly relevant boot arguments are listed in Table 3-2. Many other arguments exist for generic Linux, most of which are not documented here.

Table 3-2. Platform-Independent Linux Boot Arguments

Argument	Description
`default_hugepagesz=n`	`default_hugepagesz=n` sets the default hugepage size for the system (as used, for example, by the `tmc_alloc_set_huge()` routine), with the unit value set by using a suffix of `k` for kilobytes, `m` for megabytes, or `g` for gigabytes. The default hugepage size is 16MB. The default Linux kernel hugepage sizes are 1MB and 16MB. To use a hugepage size which is not one of those default sizes, you must specify that size using both the `hugepagesz=` and `default_hugepagesz=` options. For more information on how to use this boot option, see *Programming the TILE-Gx Processor* (UG505).
`earlyprintk=1`	This argument is primarily useful for debugging early kernel boot, and causes kernel console output to be emitted directly to the console, even before the usual kernel console mechanisms have been initialized. Any option value specified here enables this option, but you must specify an '`=`' and then any arbitrary text. By convention, one uses `earlyprintk=1`.

`hugepages`
 	
Possible formats:
`hugepages=n`
`hugepages=n,n,n,n`
`hugepages="n n n n"`
 	
Note: use commas on the kernel boot command line; use spaces for the `/proc` API.
	`hugepages=n` reserves a fixed number of huge (by default, 16MB) pages for use by applications or kernel drivers with the `/dev/hugepages` file system after booting has completed. You can modify this number by writing to the file `/proc/sys/vm/nr_hugepages`. To check the current number, you can read that same file (or `/proc/meminfo`).
	
	`hugepages=n,n,n,n` reserves a fixed number of huge pages on specific controllers. For example, `hugepages=0,4,4,0` reserves 4 huge pages on controllers 1 and 2, and none on controllers 0 and 3. You can omit trailing arguments, and the omitted nodes are simply initialized with 0 huge pages. A trailing comma is allowed.
	
	For more information using this boot option, see *Programming the TILE-Gx Processor* (UG505).

`hugepagesz=n`	`hugepagesz=n` sets the size of hugepages to reserve with the `hugepages=` parameter, with the unit value set by using a suffix of `k` for kilobytes, `m` for megabytes, or `g` for gigabytes. The default Linux kernel hugepage sizes are 1MB and 16MB. To use a default hugepage size which is not one of those sizes, you must specify that default size using both the `hugepagesz=` and `default_hugepagesz=` options. For more information on how to use this boot option, see *Programming the TILE-Gx Processor* (UG505).

Table 3-2. Platform-Independent Linux Boot Arguments (continued)

Argument	Description

`isolcpus`
 	
Possible formats:
`isolcpus=n`
`isolcpus=m,n`
`isolcpus=m-n`
	This argument allows tiles to be isolated from the Linux scheduler, which prevents Linux from moving any process or thread onto those tiles unless they explicitly demand it.
	
	For example, setting `isolcpus=1,2` would isolate a couple of tiles; `isolcpus=0-55` would isolate all tiles except those on the bottom row of the mesh. The only way to move a process onto or off of an isolated tile is by means of the `sched_setaffinity()` system call or a Tilera-specific wrapper such as `tmc_cpus_set_my_cpu()`.
	
	The `isolcpus` argument prevents random processes from interfering with CPUs on which the user has carefully placed important processes or threads. CPUs included in `isolcpus` will not run any process unless that process is affinitized to that exact CPU.
	
	This argument is similar to the `dataplane` option, but is preferred for code that makes use of kernel calls.

`root=`*`partition`*	The `root=` argument allows you to specify the root filesystem that the kernel should mount before starting userspace. For example, `root=/dev/sda1` makes `/dev/sda1` the "root" partition. The `tile-monitor --rootfs` option can be used to set this option when booting.

3.3.3 Shepherd Arguments

Note: For information about what the `shepherd` process is and what it does, see the section on the "Shepherd Process" in the "Using tile-monitor" chapter in the *Multicore Development Environment User Guide* (UG501).

The list below shows the possible arguments provided to the `shepherd` process for both initramfs and disk-based boots in the initialization scripts:

- `--net` tells the `shepherd` process to listen on network port 963.
- `--tmfifo` tells the `shepherd` process to listen on the TMFIFO interface.

By default, `--net` is always provided. If appropriate, `--tmfifo` is also provided.

For initramfs boots, you can change the arguments provided to the `shepherd` process by means of the `TLR_SHEPHERD_ARGS` boot-time environment variable.

If the arguments contain a space, then you must wrap the arguments in double quotes and you must also wrap the entire boot-time environment variable assignment in single quotes.

For example, to tell the `shepherd` process to use TMFIFO but not the network:

```
--hvx 'TLR_SHEPHERD_ARGS="--tmfifo"'
```

Note: The single quotes get stripped off by the shell, correctly yielding: `TLR_SHEPHERD_ARGS="--tmfifo"`

Note: For initramfs boots, you can disable the `shepherd` process completely by adding `TLR_SHEPHERD=n` to your `.hvc` file.

3.4 Controlling the Linux Initramfs Boot Process

When booting a Tile device without persistent storage, you must provide the initial file system as part of the bootrom image itself, using the Linux 3.4 `initramfs` scheme.

The initial `ramfs` file holds the list of system components necessary for booting the system up to the point where it can communicate with the Tile Processor. This list includes `busybox`, `procps`, and the Tilera `shepherd` executable. Tilera system software mounts various file systems at startup, including `/proc`, `/sys`, `/dev/hugepages`, `/dev/shm`, and `/dev/pts`.

The default initramfs used when booting Linux this way is shipped in the `/boot/initramfs.cpio.xz` file in the Tile file system hierarchy (`$TILERA_ROOT/tile/` in the cross-build environment). You can modify this file if you wish, by copying (or modifying) the files in `/usr/lib/initramfs/` in the Tile filesystem hierarchy and running the `tile-gen-initramfs` program to create a new `initramfs.cpio.xz` file. See Section 3.4.2 "Adding User Files to the Initramfs Boot Image" on page 58 for more information.

3.4.1 Environment Variables for /etc/rc Startup Script for Initramfs Boots

You can pass the boot arguments shown in Table 3-3 to the `/etc/rc` startup script to control its behavior. These arguments are for initramfs boots only.

Unless otherwise specified, all boot-time environment variables should be set either to `y` (for example, `TLR_TELNETD=y`) or `n` (for example, `TLR_TEMPMOND=n`). If a variable is unset, or if it is set to an unsupported value, it will normally default to `n`, unless otherwise noted in the text. Some variables (for example, `TLR_NETADDR` and `TLR_NETWORK`) can take additional values beyond `y` or `n`; this is noted in the text.

Note: We recommend that you specify these environment variables using the `--hvx` flag instead of modifying the client arguments in the `.hvc` file.

Table 3-3. Environment Variables for /etc/rc Startup Script for Initramfs Boot Operations

Argument in Initramfs Environment	Description in Initramfs Environment
`TLR_AFFINITY`	Sets the default affinity for all processes created out of the `/etc/rc` file; by default, all CPUs are available. A comma-separated list of CPUs can be specified, including CPU ranges separated by hyphen. Example: `TLR_AFFINITY=0-7`.
`TLR_AVOID_CONSOLE`	If set to `y`, removes the `/dev/console` device and redirects all the `/etc/rc` output to `/dev/null`. This makes it easier for user processes to use the serial console for I/O, though kernel messages might still be interspersed.
`TLR_COREDUMPS`	Enables creation of core dumps on fatal signals. By default, Tilera Linux does not enable core dump creation, because the Tilera debugging infrastructure generally works by using `ptrace` to debug fatal signals as they happen. However, if core dumps are desired, set `TLR_COREDUMPS=y`.

Table 3-3. Environment Variables for /etc/rc Startup Script for Initramfs Boot Operations (continued)

Argument in Initramfs Environment	Description in Initramfs Environment
TLR_INTERACTIVE=y	Starts a root shell on the console. The /etc/rc script enables the switch by default. If TLR_SHEPHERD=y is also set, both shepherd and console shells are available. See Section 3.3.3 "Shepherd Arguments" on page 54 for shepherd arguments.
TLR_NETADDR=x.y.z.q TLR_NETADDR=none	Sets a manual IP address. If this variable is set, you can also specify TLR_NETMASK to set a network mask. If neither is set, but a network interface is enabled by means of TLR_NETWORK, the software attempts to configure the IP address by means of DHCP. If this variable is set, you can also specify TLR_NETGW to indicate the default gateway to use. If this variable is set to none, the software does not attempt to set the IP address.
TLR_NETGW=x.y.z.q	Sets a default gateway on the network interface that can be used to route off the local network. TLR_NETADDR must also be set to use this variable.
TLR_NETMASK=x.y.z.q	Sets a network mask. TLR_NETADDR must also be set to use this variable.
TLR_NETWORK=auto TLR_NETWORK=*interface* TLR_NETWORK=pci-net TLR_NETWORK=none	Specifies the network interface that the /etc/rc script brings up when it runs. If set to auto, attempts to bring up the first I/O interface that works with DHCP (eth0, eth1, xgbe0, xgbe1, gbe0, gbe1), and starts DHCP on it. Note: When using the vmlinux.hvc file, TLR_NETWORK=auto is the default if no other vmlinux ARGS have been specified. If set to *interface*, brings up Linux networking on the specified interface. Here, *interface* is the name of an I/O device to use for networking, such as xgbe0, xgbe1, gbe0, gbe1. If set to pci-net, brings up the TILE side of a pci-net connection, in order to provide a networking tunnel over the PCI link of a PCIe card to a companion pci-net process running on the x86 Linux host. If set to none, Linux does not attempt to bring up any of the network interfaces.

Table 3-3. Environment Variables for /etc/rc Startup Script for Initramfs Boot Operations (continued)

Argument in Initramfs Environment	Description in Initramfs Environment
TLR_QUIET=y	Suppresses messages. If this variable is set at boot time, the normal startup messages from the /etc/rc script and associated daemons are suppressed. Combined with the standard Linux quiet boot option, this can remove almost all of the normal startup messages from the Linux boot process.
TLR_ROOT=root	Block device holding a filesystem, or tmpfs to copy the rootfs to a tmpfs, or none to use the rootfs. To use the Compact Flash and Solid-State Disk (CF/SSD) for the new root, use/dev/hda1 or similar. To use a SATA disk, use /dev/sda1 or similar. By default, the rootfs is copied to a tmpfs whose size limit is half of total memory. It is generally better to use the root= kernel boot option rather than including an initramfs just for the purpose of using TLR_ROOT, but this option can be useful. Using the TLR_ROOT_SUBDIR, and so forth options can provide functionality not available with the regular root= boot option.
TLR_ROOT_INIT	File to run in the new root (/sbin/init by default).
TLR_ROOT_OPTIONS	Options for mount. For example: TLR_ROOT_OPTIONS=size=10m to specify the maximum size of a tmpfs. By default, no extra options are passed to mount. Note that many mount options, including size, can also be set after boot. For example: mount -o remount,size=100m / or similar.
TLR_ROOT_SUBDIR=*path*	Subdirectory on the target filesystem specified by TLR_ROOT; by default the root filesystem is the top of the filesystem on that partition. Specifies a subdirectory on the partition to become the new root filesystem. For example, to use the demo/root subdirectory on /dev/sda1 as the root filesystem, use: TLR_ROOT=/dev/sda1 TLR_ROOT_SUBDIR=demo/root
TLR_ROOT_TYPE	Type of filesystem, or empty to auto-detect. If TLR_ROOT is tmpfs, TLR_ROOT_TYPE is forced to tmpfs.
TLR_SHEPHERD=y	Specifies that you want to run the shepherd process for communication with the Tile Processor. This argument is enabled by default in the /etc/rc script.
TLR_SHEPHERD_ARGS=[*args*]	Allows you to specify the precise set of command-line arguments for the shepherd process. See Section 3.3.3 “Shepherd Arguments” on page 54.

Table 3-3. Environment Variables for /etc/rc Startup Script for Initramfs Boot Operations (continued)

Argument in Initramfs Environment	Description in Initramfs Environment
TLR_SHEPHERD_PCIE_MAC	Sets the PCIe MAC number [0, 2] that the Shepherd opens and starts monitoring. If you do not specify this variable, the Shepherd monitors the first MAC of all the active MAC ports, starting from 0.
TLR_SSHD=y	Specifies that you want to run the Dropbear `ssh` daemon for communication with the Tile Processor. This argument is enabled by default in the `/etc/rc` script.
TLR_SSHD_ARGS=[args]	Allows you to specify the precise set of command-line arguments for the Dropbear `ssh` daemon. If not specified, the default value of this variable is `-e`, which allows login to accounts with empty passwords.
TLR_TELNETD=y	Makes a telnet service available on that network. `y` is the default boot argument. If this variable is set, a `telnet` daemon is started. By default, the `root` account can be specified when connecting to the daemon and logins with no password are accepted.
TLR_TEMPMOND=y	Starts the temperature monitoring daemon, `tempmond`. The `tempmond` daemon is normally started by default if the board provides temperature information. You can force `tempmond` to be disabled by setting this variable to `n`.

3.4.2 Adding User Files to the Initramfs Boot Image

The default Linux initramfs comes from the `initramfs.cpio.xz` file in the Tile file system hierarchy (located in `/boot` in the native build environment and `$TILERA_ROOT/tile/boot/` in the cross-build environment). To create a customized initramfs file, copy the initramfs directories (`/usr/lib/initramfs`) into a working directory, change the `contents.txt` file to reflect your changes to the initramfs, make your changes to the initramfs files and run the `tile-gen-initramfs` command to create an `initramfs.cpio` file from your working directory.

The `contents.txt` file specifies which files are included, and under what names; it also lists directories, symbolic links, and device nodes that should be present in the `initramfs` file system.

To create a customized `initramfs` file, follow this procedure:

1. Copy the contents of the original initramfs directories into a working directory:

   ```
   cp -a /usr/lib/initramfs /proj/scratch_initramfs
   ```

2. Change to the default directory:

   ```
   cd /proj/scratch_initramfs/default
   ```

3. Edit the contents.txt file to add, remove or change files that you want to include in the initramfs.

To include a new binary file, copy one of the file options lines in the section:

```
#
# Executables
#
```

The format of an initramfs file definition in `contents.txt` is as follows:

```
file initramfs_file source_file permissions uid gid
```

The *`initramfs_file`* is the pathname for the new file in the initramfs filesystem, and *`source_file`* is the path to the file that you wish to add to the initramfs.

You can use two special keywords in the *`source_file`* pathnames:

- `ROOT`

  Specifies the path relative to `$TILERA_ROOT/tile`

- `DIR`

  Specifies the path relative to the directory which contains the `contents.txt` file

For example, the line which includes the `xzdec` decompression binary into the `initramfs` is:

```
file /usr/bin/xzdec ROOT/usr/lib/initramfs/bin/xzdec 755 0 0
```

4. From your working directory, run the `tile-gen-initramfs` command to create the create a new initramfs file:

   ```
   tile-gen-initramfs contents.txt new_initramfs.cpio.xz
   ```

5. You can use tile-monitor to boot tile hardware using a custom vmlinux file and the new initramfs file as shown below:

   ```
   $(TILERA_ROOT)/bin/tile-monitor --vmlinux /path_to_custom_vmlinux  \
   --initramfs /proj/scratch_initramfs/default/new_initramfs.cpio.xz
   ```

   To embed the new `initramfs` file into a `vmlinux` image, rebuild the kernel after specifying the new `initramfs` file path as a parameter in the `CONFIG_INITRAMFS_SOURCE=` kernel configuration option. You can configure this option by directly modifying the generated configuration file or running the command `make menuconfig` to reconfigure the `vmlinux` image. The following is an example of setting this option:

   ```
   CONFIG_INITRAMFS_SOURCE="/myproj/initramfs.cpio"
   ```

3.5 Linux Security Configuration

By default, Linux is shipped in a development-friendly but insecure configuration, primarily because it boots from a fixed image and does not rely on any persistent state in the board when it boots.

By default, if you enable network access (for example, by means of the `TLR_NETWORK` boot environment variable), several insecure servers will be exposed:

- Both `telnet` and `ssh` servers run by default, and allow logins to the `root` account without a password.
- The `shepherd` process runs as `root` by default, and allows a range of `rpc` commands to be requested without any authentication.

In the initramfs environment, to disable these three daemons, you can add the following to your `boot` command arguments:

```
TLR_TELNETD=n TLR_SSHD=n TLR_SHEPHERD=n
```

If booting into a persistent file system using the disk-based initialization model, you should comment out the `telnetd`, `dropbear`, and `shepherd` lines from `/etc/init/tilera.conf`.

By default, the `shepherd` process runs with`--net`, which allows it to accept commands from the network. Add the following to your `boot` command arguments to allow network access:

```
TLR_SHEPHERD_ARGS=--net
```

And finally, if the board serial connection is not secure, you might want to disable automatically running a shell prompt on the console. To change the console to run a login prompt instead of a shell, the tile `/etc/inittab` file can be modified to require a login by running `/bin/login` instead of `-/bin/sh` on the console (see the comments in that file). If you do this, you should also provide a password for the `root` account, by running the `passwd` command or by copying the encrypted password field in `/etc/shadow` from another Linux system. These actions require modifying the default `root` filesystem and cannot be requested by `boot` command arguments.

3.6 Enabling Zero Overhead Linux (ZOL) With the dataplane Boot Argument

The `dataplane` boot argument allows specifying dataplane tiles for use by low-latency applications. See Table 3-1, "Tile-Specific Linux Boot Arguments," on page 45.

Dataplane tiles run in Zero Overhead Linux (ZOL) mode. This means that when a single user process (or thread) is running on the tile, Linux protects that tile from any cross-tile interrupts, and will disable delivering scheduling *ticks* to the process while it is running. The tile runs only its own user-space instructions and should maintain a very low level of "jitter" or timing unpredictability.

In addition to protecting the processes running on the dataplane tiles, Linux isolates the tiles from the normal kernel scheduler, and does not attempt to load-balance processes automatically onto those tiles. This protects any processes running on those tiles, but also means that the application must explicitly handle any load balancing, for example by using the TMC library's `tmc_cpus_set_my_cpu()` API.

In addition, this option arranges for the default process affinity to be the non-dataplane tiles, so that `init`, system daemons, `tile-monitor`, the `shepherd` process, and the like are constrained to run only on the non-dataplane tiles.

The set of dataplane tiles can be displayed after boot by means of `/sys/devices/system/cpu/dataplane`. This file is a standard Linux CPU list, which means that it lists ranges of CPUs with a dash, and individual CPUs, all separated by commas.

For more information, see the "Zero Overhead Linux Applications" chapter in *Programming the TILE-Gx Processor* (UG505).

3.7 Optimizing Linux Network Stack Performance With the tile_net.cpus Boot Argument

The `tile_net.cpus` boot argument applies only when using native Linux networking with the `gbe` or `xgbe` devices. It does not apply to other devices using other drivers, such as the `eth0` and `eth1` devices on TILEmpower, which use the PCIe root complex driver. See Table 3-1, "Tile-Specific Linux Boot Arguments," on page 45.

In some situations, it is beneficial to restrict Linux network device interrupt processing to specific tiles. By default, network device interrupts are sent to all Linux tiles. The main function performed by network device interrupts is ingress processing (that is, handling incoming network packets). The `tile_net.cpus` boot argument limits network device interrupts to the specified set of tiles. Typically, you segregate the `tile_net.cpus` tiles from application tiles (that is, assign them to disjoint tiles). This allows tiles to focus on separate activities: ingress processing on the one hand, and application functionality (and egress processing) on the other hand.

Note: Dataplane tiles do not receive device interrupts, so the `tile_net.cpus` option is not needed to protect dataplane tiles from those interrupts. In fact, any dataplane tiles are removed from the `tile_net.cpus` set, if both are specified. A ramification of this is that at least one non-dataplane Linux tile is required in order to run Linux networking.

3.8 Configuring and Using Dynamic Voltage and Frequency Scaling

You can configure Dynamic Voltage and Frequency Scaling (DVFS), using the `dvfs=` hypervisor configuration, with modes to enable frequency changes for just the core but not voltage changes (default), enable frequency changes for just the core with voltage changes, or to forbid DVFS.

Add a line into the hypervisor configuration file (usually `vmlinux.hvc`) as follows:

```
options dvfs=mode
```

Set *`mode`* as follows:

- `off` – Do not allow the primary client to modify any frequencies during operation.
- `core` – Allow the primary client to modify the core frequency during operation, but do not make any corresponding changes in core voltage. This is the default setting.
- `core_volt` – Allow the primary client to modify the core frequency during operation, and make corresponding changes in core voltage.

Note: The `core_volt` setting may lead to instability in some system configurations.

When you enable DVFS, in either `core` or `core_volt` mode, you can set the processor core frequency by writing a new frequency value (expressed in kHz units) into this file:

```
/sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
```

Note: If you adjust voltage using this method, the system accommodates the frequencies of the I/O shims as well as the core. To permit maximum voltage decrease at lower core frequencies, and thus maximum power savings, you might also reduce I/O shim frequencies to the minimum required for their application (using the speed subcommand in the relevant device stanzas in the `vmlinux.hvc` file), and also disable any devices that you are not using by omitting the device stanzas for those devices entirely.

3.8.1 Determining the Current TILE-Gx Chip Frequency

To determine the current chip frequency, examine the virtual device file:

```
/sys/devices/system/cpu/cpu0/cpufreq/cpuinfo_cur_freq
```

The frequency units are in kHz.

3.8.2 Determining the TILE-Gx Chip Frequency Ranges

The maximum and minimum chip frequencies, in kHz, are in these files:

```
/sys/devices/system/cpu/cpu0/cpufreq/cpuinfo_min_freq
/sys/devices/system/cpu/cpu0/cpufreq/cpuinfo_max_freq
```

3.8.3 Modifying the TILE-Gx Chip Frequency

To modify the frequency, write the new frequency, in kHz, into this file:

```
/sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
```

So, for instance, to run at 1.2 GHz, you could do:

```
echo 1200000 >/sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
```

To set the minimum frequency, use this command:

```
cat /sys/devices/system/cpu/cpu0/cpufreq/cpuinfo_min_freq  \
>/sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
```

For more information about the hypervisor options related to DVFS, see Section 2.3.1 “The options Command” on page 18.

3.8.4 Resetting MiCA Compression Shims Using sysfs

To reset the MiCA compression shims through the `sysfs` interface, use the command:

```
# echo 1 >/sys/devices/system/comp/comp0/reset
```

This command causes the hypervisor driver to disable the shims, save the registers, reset the compression shims, restore the contents of the configuration registers and restart any pending contexts.

The hypervisor puts an error code in the Extra Data register for any contexts that were hung.

3.9 Troubleshooting

If the Linux kernel encounters trouble, it attempts to dump any relevant panic messages or stack back traces, or both, to the console. The console is normally connected by means of a serial connection from the PCI board. If any output is generated and the kernel appears to be misbehaving, the console output provides important debugging information for Tilera.

You can also use the boot variable, `TLR_COREDUMPS`, which enables the creation of core dumps when it encounters fatal signals. If you need to receive core dumps, set `TLR_COREDUMPS=y`. For more information, see Table 3-3, “Environment Variables for /etc/rc Startup Script for Initramfs Boot Operations,” on page 55.

3.9.1 Console Serial Device Settings

The console is normally available over a serial connection from the Tilera board, running at 115200 baud, 8 bits, no parity, and one stop bit. No flow control is supported (either hardware or software) so flow control must be disabled as well.

Typically, the Tilera board serial port is connected to a serial port on another (host) Linux box. Various Linux commands can be run on the host Linux to monitor the Tilera console serial connection, for example `cu`, `kermit`, or even `cat`.

Tilera provides two commands, `tile-console` and `tile-monitor`, that take care of setting up the proper serial device settings, and are generally easier to use than generic Linux commands.

For more information on `tile-console`, see "Tile Processor Console" (`tile-console.html`) in the "Target Platform Tools" section of the MDE Document Index or the man page at `man tile-console`. (These are identical.)

For more information on `tile-monitor`, see "Tile Processor Monitor" (`tile-monitor.html`) in the "Target Platform Tools" section of the MDE Document Index or the man page at `man tile-monitor`. (These are identical.)

CHAPTER 4 MAKING A BOOTROM

This chapter describes:

4.1 Overview

When you make changes to the default hypervisor or to the vmlinux configuration, you must create a new bootrom to make those changes available on the Tilera hardware. This chapter tells you how.

4.2 Creating a Bootrom

To run the hypervisor with a new configuration, you must create a bootrom containing that configuration. A bootrom file is a stream of executable code and data that can be used to boot a Tile Processor. The bootrom consists of the booter and hypervisor code, followed by a hypervisor file system image containing files that you specify. Normally these files consist of the hypervisor configuration file and the client operating system image (for example, `vmlinux`).

There are two basic ways to create a bootrom file:

- Use `tile-mkboot` (see Section 4.3 “Creating a Bootrom Using the tile-mkboot Command” on page 66)
- Use `tile-monitor` to invoke `tile-mkboot` (see Section 4.4 “Creating a Temporary Bootrom File Using tile-monitor” on page 67 and Section 4.5 “Creating a TILEmpower-Gx Bootrom Using tile-monitor” on page 67)

4.3 Creating a Bootrom Using the tile-mkboot Command

The `tile-mkboot` command creates a bootrom file.

Some of the options available for `tile-mkboot` are:

`-b, --boot` *`bootfile`*

If the `--boot` option is specified, the booter and hypervisor code are taken from the named boot file; otherwise, the default boot file, `$TILERA_ROOT/tile/boot/boot.bin`, is used.

`-o, --output` *`outfile`*

Specifies the file name of the bootrom to be output.

`[`*`outname`*`=]`*`infile`*

Specifies a file to be included in the hypervisor file system image. If the `outfile=` prefix is included, the input file `infile` is added to the file system under the name `outfile`. Otherwise, the name in the output file system is identical to the input file name.

The hypervisor expects its configuration file to be called `config`, and is not able to complete the boot process if a file by that name is not included in the bootrom. The name of the client operating system can be specified in the configuration file, but is conventionally `vmlinux`.

For other options, see "Tile Processor Boot Image Creator" (`tile-mkboot.html`) in the "Development Tools" section of the MDE Document Index or the man page at `tile-man tile-mkboot`. (These are identical.)

4.3.1 Creating the mboot.bootrom File with tile-mkboot

You can reproduce the `mboot` bootrom file that ships as `$TILERA_ROOT/tile/usr/lib/boot/mboot.bootrom` with the following `tile-mkboot` command:

```
$ tile-mkboot --output mboot.bootrom \
   --compress --hvc vmlinux.hvc \
   classifier=$TILERA_ROOT/tile/boot/classifier \
   vmlinux=$TILERA_ROOT/tile/boot/vmlinux_mboot
```

4.3.2 Invoking tile-mkboot Using the tile-monitor --hvc Option

For convenience, you can use `tile-monitor`'s `--hvc` option to invoke `tile-mkboot` and to boot the resulting bootrom file. For example:

```
$ tile-monitor --hvc myboot.hvc
```

If `tile-monitor` finds that a TILEncore-Gx Intelligent Application Adapter is installed in the host PC, it will implicitly define certain `--hvd` parameters appropriate to the card. It creates these parameters from information in the file `/dev/tilegxpciN/info` (for the *N*th PCIe card). This file is created by the host PCIe driver from information that it extracts from the TILEncore-Gx Intelligent Application Adapter.

4.4 Creating a Temporary Bootrom File Using tile-monitor

The `tile-monitor --create-bootrom` option creates a bootrom without booting a chip. Other useful `tile-monitor` options relating to the hypervisor are `--hvc`, `--hvd`, `--hvx`, `--hv-bin-dir`, and `--vmlinux`.

When you invoke `tile-monitor` on a host PC with an installed TILEncore-Gx adapter, `tile-monitor` invokes the `tile-mkboot` command to create a temporary bootrom file, which it then uses to boot the adapter. You can create the contents of this temporary bootrom file using the following `tile-mmonitor` command:

```
$ tile-monitor --create-bootrom vmlinux.bootrom \
--hvd BOARD_HAS_PLX=1 \
--hvc $TILERA_ROOT/tile/etc/hvc/vmlinux.hvc \
--vmlinux $TILERA_ROOT/tile/boot/vmlinux
```

You can create the contents of this temporary bootrom file using the following `tile-mkboot` command:

```
$ tile-mkboot --output vmlinux.bootrom --hvd BOARD_HAS_PLX=1 \
--hvc $TILERA_ROOT/tile/etc/hvc/vmlinux.hvc \
vmlinux=$TILERA_ROOT/tile/boot/vmlinux
```

4.5 Creating a TILEmpower-Gx Bootrom Using tile-monitor

Use the `tile-monitor` command to invoke `tile-mkboot` to create a bootrom file using the `--create-bootrom` argument. The following sections show various methods for doing this.

4.5.1 Creating the Default Bootrom for a TILEmpower-Gx Appliance

TILEmpower-Gx appliances contain a default bootrom burned into the on-board SROM flash memory device. This file can be found at `$TILERA_ROOT/tile/usr/lib/boot/tilempower.bootrom`. You can reproduce that bootrom file using the following `tile-monitor` command:

```
$ tile-monitor --no-dev --create-bootrom tilempower.bootrom \
    --rootfs /dev/sda1 --hvx rootwait
```

Note: The `--no-dev` argument specifies that `tile-monitor` does not wait for any device, which is useful when you are just creating a bootrom or a preprocessed hypervisor configuration file. The `--rootfs` argument passes a boot argument to Linux of `root=block-device`, and suppresses the default inclusion of an `initramfs.cpio.xz` file.

4.5.2 Creating a Bootrom for both Hypervisor and Linux Images

If you have modified he hypervisor and re-configured or customized Linux, you must build the images and then create a new bootrom, pointing to the new images from the `tile-monitor` command. You can produce a new bootrom file with the following `tile-monitor` command:

```
$ tile-monitor --create-bootrom tilempower.bootrom \
    --hvc vmlinux.hvc --hv-bin-dir $TILERA_ROOT/tile/boot \
    --vmlinux $TILERA_ROOT/tile/boot/vmlinux
```

4.5.3 Creating a Bootrom with the Default initramfs

To create bootroom for a TILEmpower-Gx platform using the a custom `vmlinux` image and the default initramfs file, use the following `tile-monitor` command:

```
$ tile-monitor --create-bootrom tilempower.bootrom \
   --hvc vmlinux.hvc
```

4.5.4 Creating a Bootrom Using a Custom initramfs

Customize the files, directories, symlinks, and device nodes to be included in the root RAM filesystem of the booted kernel in your custom initramfs filesystem (`my-initramfs.cpio.xz`) by copying the `$TILERA_ROOT/tile/usr/initramfs/default/contents.txt` file and editing that file using the standard `gen_init_cpio` format.

Use the `tile-gen-initramfs` command to create a custom `initramfs` file by pointing to modified `contents.txt` and specifying an `initramfs` output filename:

```
$ tile-gen-initramfs /initramfs/default/my-contents.txt my-initramfs.cpio.xz
```

To generate a bootrom using that custom `initramfs`, use this `tile-monitor` command:

```
$ tile-monitor --create-bootrom tilempower.bootrom \
   --hvc vmlinux.hvc --initramfs my-initramfs.cpio.xz
```

4.5.5 Creating a Bootrom Specifying a Root Filesystem

To create bootroom for a TILEmpower-Gx platform using the a custom `vmlinux` image and ignoring the default `initramfs` file and using a root filesystem which already exists on the TILEmpower-Gx platform SSD (`/dev/sda1`), use the following `tile-monitor` command:

```
$ tile-monitor --create-bootrom tilempower.bootrom \
   --hvc vmlinux.hvc --rootfs /dev/sda1
```

CHAPTER 5 USING THE MBOOT BOOT LOADER

This chapter describes:

5.1 What is mboot?

Often with embedded systems, there is no host system on which to run `tile-monitor`. Instead, embedded systems typically boot either from a hard disk, from the network, or from compact flash. To facilitate booting from these devices, the MDE includes a boot loader, called `mboot`.

`mboot` typically executes from SROM. It also has the ability to locate other devices that might contain a final Linux kernel, to select a Linux kernel either interactively or automatically, and to begin executing that Linux kernel on the Tile Processor.

`mboot` provides an extended shell environment with the most commonly used commands for accessing file systems (such as `mount`, `cat`, and `dd`), as well as network configuration commands (such as `ifconfig` and `route`). `mboot` allows an administrator to select configuration parameters to be applied automatically in subsequent booting.

When booting the Tile Processor from `mboot`, the boot process is modified as follows:

1. The Tile Processor loads and executes the first kernel from SROM or from an accompanying agent on a host PC.

   This first kernel is the boot loader, `mboot`. The `mboot` kernel is a customized version of Tile Linux. It has access to all of the same devices to which the Tile Processor has access.

2. After the `mboot` kernel has started running, it invokes setup code and then runs a program with a simple command-line interface (CLI), which you can use for configuration or booting.

3. The `mboot` kernel loads the final kernel (over TFTP, FTP, or from a local device). Although the final kernel could be Linux, there is no requirement that it must be.

Warning! The final kernel must be completely self-contained; in particular, you cannot use `mboot` to load a `vmlinux` and an `initramfs` file separately. If you require an `initramfs`, you must embed it in the `vmlinux` image. See Section 3.4.2 “Adding User Files to the Initramfs Boot Image” on page 58.

Warning! The final kernel should be based on the same MDE version as `mboot` and the hypervisor. A kernel based on other MDE versions might not be able to boot up, and even if it boots, it might not run without errors.

5.2 mboot Flash Configuration

In a typical configuration, a Linux kernel image is stored on a compact flash card or SATA disk. The configuration process usually involves configuring the network, downloading the kernel image, setting the boot parameters, then rebooting to get the new kernel.

Configuring the network can be done either with a static address or by means of DHCP.

To assign a static address, use the `ifconfig` command. For example:

```
mboot# ifconfig xgbe0 192.168.1.27 -mask 255.255.255.0 -up
```

This command assigns an UP address of 192.168.1.27, with a netmask for a class C network, and enables the interface.

To obtain an IP address by means of DHCP, use the `dhcp` command. For example:

```
mboot# dhcp xgbe0 yes
```

This causes an IP address to be obtained by means of DHCP the next time the card is booted.

You can download the kernel with either `tftp` or `ftp`.

To download the kernel using TFTP, use the `tftp` command. For example, this command downloads the kernel image from the TFTP server on 192.168.1.100 and stores it to the hard drive:

```
mboot# tftp 192.168.1.100 vmlinux -hd
```

To configure the boot device and Linux image file name, use the `bootparam` command. For example, this command configures `mboot` to boot an image named `vmlinux` from the hard drive:

```
mboot# bootparam -dev hd -img vmlinux
```

You can view the current configuration with the `showcfg` command. This command shows the current configuration, but the new configuration does not take effect until the next time the card is booted.

5.2.1 Passing Network Configuration Settings to the Booted Kernel

`mboot`'s network configuration settings are not automatically passed to the booted kernel. Specifically, `USER` configurations created or saved by `mboot`'s `ifconfig` and `dhcp` commands are applied only to `mboot`, not to the booted kernel.

For example, after the following is invoked:

```
# ifconfig gbe0 x.x.x.x -up
```

`gbe0` settings will be saved as `USER` configurations. To apply the same settings to the booted kernel, you must explicitly pass them by means of `mboot`'s `bootparam` command as:

```
# bootparam TLR_NETWORK=gbe0 TLR_NETADDR=x.x.x.x
```

See Table 3-3, "Environment Variables for /etc/rc Startup Script for Initramfs Boot Operations," on page 55 for more environment variables that you can pass to the kernel, in particular `TLR_NETGW`, `TLR_NETMASK`, and `TLR_NETWORK`.

5.3 mboot Command-Line Interface

The `mboot` command-line interface (CLI) provides the following capabilities:

- Display of files in the boot logic disk zone (the `/boot/` directory) for hard disk drives.

- Management of Ethernet devices, including setting and showing parameter values, and setting:
  - MAC address
  - IP address, subnet mask, and gateway
- Downloading of a kernel image to hard disk or memory.
- Setting and retrieving the following boot parameters, and storing them in the data flash segment:
  - OS kernel file name
  - Kernel file location
  - Boolean value for downloading directly to memory and running
- Setting and retrieving Trivial File Transfer Protocol (TFTP) parameters, and storing them in the data flash segment.
- Boot command within the boot loader to load the kernel image from a specific device.
- Explicit configuration command to set a value in flash specifying the default location from which to boot the kernel image (that is, when the user does not go into the `mboot` CLI).

For detailed information on `mboot` commands, see Appendix B— mboot CLI Commands.

CHAPTER 6 I/O DEVICES AND DRIVERS

This chapter describes:

6.1 Hypervisor Devices and their Drivers

The Tile Processor™ includes on-chip I/O controllers, including DDR3 memory, XAUI and GbE Ethernet, PCIe, and other assorted I/O. Separate I/O shims on the edges of the Tile Processor manage each actual hardware interface, and have one or more connections to the iMesh™ interconnect. Commands and responses flow over the processor's memory networks, which connect all tiles and I/O shims.

This chapter describes MDE-compatible I/O devices and their drivers. Each subsequent section in this chapter focuses on software support for one particular I/O shim, and generally contains the following information:

- A brief description of the device.
- An explanation of how the device is configured in the hypervisor configuration file, including differences between the different drivers that can drive the device; shared and dedicated tile requirements; and driver options.
- A description of the user and/or application programming interface to the device.

The hypervisor uses two kinds of tiles to aid in I/O processing: *dedicated* and *shared tiles*.

- Dedicated tiles only process I/O device requests; they cannot run a client supervisor.
- Shared tiles also process device requests, but unlike dedicated tiles they can run the hypervisor, supervisor, and application programs. Shared tiles provide a synchronization point through which I/O is performed. Several devices can use the same shared tile. One tile designated as the *default shared tile* is used if no shared tile is explicitly specified for a device that requires one.

Dedicated Tiles

The following device requires dedicated tiles:

- Memory profiling

Shared Tiles

The following devices require shared tiles:

- SROM
- mPIPE
- EEPROM
- I^2C master
- TMFIFO
- Watchdog

Tilera ships a default hypervisor configuration file (`vmlinux.hvc`) which references `device?` statements in a `devices.hvh` file. If you create your own `.hvc` file, you must have `device?` statements somewhere in your configuration file.

For information on developing and optimizing your own hypervisor drivers, see Chapter 7: Developing Hypervisor Drivers.

6.2 mPIPE Driver

Depending on the capabilities of the specific processor, the mPIPE driver provides access to Gigabit Ethernet, 10 Gigabit Ethernet, or Interlaken links. The driver supports both the native Linux Ethernet driver and direct application access by means of the `gxio_mpipe_` routines in the GXIO library. See the *MDE mPIPE Programmer's Guide* (UG506) and the *Application Libraries Reference Manual* (UG527) for information about mPIPE application programming.

Drivers

In order to access mPIPE, you must specify `device? mpipe/0` in the `vmlinux.hvc` hypervisor configuration file. You configure all device options by means of the Linux API.

Tiles

The mPIPE driver requires one shared tile, which is used for link management tasks, including the servicing of link state change interrupts.

6.2.1 Modifying mPIPE Timestamp Counter Tick Rate

The hypervisor option `timestamp-is-cycle` modifies the rate at which mPIPE's timestamp counter ticks, and its initial value, so that it is synchronized with each tile's cycle counter. The timestamp counter is the source for the timestamps supplied with incoming packets. By default, the timestamp counter runs at a rate of one tick per nanosecond, and is not synchronized with any on-chip or off-chip counter.

Packet timestamps are delivered in the `time_stamp_sec` and `time_stamp_ns` members of the mPIPE packet descriptor (`gxio_mpipe_idesc_t`). When the `timestamp-is-cycle` option is in effect, the value:

```
descriptor.time_stamp_sec * (uint64_t) 1000000000 + descriptor.time_stamp_ns
```

is equal to the value of the tile's cycle count register at the time the packet arrived at mPIPE.

To use the `timestamp-is-cycle` option, add the following device stanza to the file `$TILERA_ROOT/tile/etc/hvc/devices.hvh`:

```
device? mpipe/0
  args timestamp-is-cycle
```

For more details on packet time-stamping, see the `gxio` section of the *Application Libraries Reference Manual* (UG527) and/or the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404). For details on the cycle counter and APIs for reading it, see the Tile Architecture Headers section of the *Application Libraries Reference Manual* (UG527).

Caveats

This option may be useful for certain types of performance measurement; however, its use is not generally recommended, since it causes packet timestamps to be inconsistent with the hardware documentation and thus possibly inconsistent with the expectations of some applications. Use of this option is inconsistent with, and may disable, certain enhancements to mPIPE timestamp support (for instance, synchronization of mPIPE timestamps with an external time source), should Tilera ever choose to provide such enhancements in a future version of the MDE.

6.3 TRIO Driver

The TRIO driver provides access to PCIe, in both root complex and endpoint modes.

The driver supports several Linux drivers, as well as direct application access by means of the `gxpci_` routines in the GXIO library. See the *MDE TRIO Programmer's Guide* (UG503) and the *Application Libraries Reference Manual* (UG527) for information about TRIO application programming.

In order to access TRIO, you must specify `device? trio/0` in the hypervisor configuration file.

You configure all device options by means of the `gxpci` API.

The Tile Processor includes one or more TRIO shims, each of which contains multiple PCIe ports. Each on-chip PCIe port is capable of running in either of two modes:

- *Endpoint mode* allows the Tile Processor card to work as an add-in card that allows communication between the Linux host machine and the Tile Processor card. See Section 6.3.1 "Accessing PCIe Endpoint Mode" on page 75.
- *Root complex mode* allows the Tile Processor to act as a host, so that system developers can connect other PCIe devices (for example, a hard disk controller) directly to the Tile Processor. See Section 6.3.4 "Accessing PCIe Root Complex Mode" on page 77.

6.3.1 Accessing PCIe Endpoint Mode

The Tile Processor card installs into a x8 or x16 PCIe slot in a host machine. The `tilegxpci` driver allows processes running on the host machine to boot the Tile Processor chip and to communicate with processes running under Tile Linux. Host machine processes communicate with Tile Linux processes through several communication mechanisms, including:

- **Character streams:** -- a set of 4 socket-like communication streams, named:
  - `/dev/tilegxpci`*N*`/`*X* on the host. For example, writing to the special file `/dev/tilegxpci`*N*`/3` on the host machine transmits data to `/dev/trio`*N*`-mac`*M*`/3` on the Tile Processor, which any Tile Linux process can then read.
  - `/dev/trio`*N*`-mac`*M*`/`*X* on Tile Linux. For example, data written to `/dev/trio`*N*`-mac`*M*`/3` is readable from the host machine's special file `/dev/tilegxpci`*N*`/3`.

Note: On the Tile side, you access the PCIe links via the `/dev/trio`*N*`-mac`*M*`/` directory, where *N* is the TRIO instance number (always 0 for TILE-Gx36) and *M* is the PCIe port number on this TRIO instance.

- **Zero-copy packet queue:** For high performance and low latency applications, your host applications can transfer data to and from the TILE-Gx endpoint ports by opening devices `/dev/tilegxpci`*N*`/packet_queue/h2t/`*M* and `/dev/tilegxpci`*N*`/packet_queue/t2h/`*M*, respectively. On the TILE-Gx side, the GXPCI library supports the packet queue interfaces. For more information, see the *Application Libraries Reference Manual* (UG527).
- **Zero-copy command queue:** Applications can issue buffer commands directly to the driver through `/dev/tilegxpci`*N*`/h2t/`*X* on the host and `/dev/trio`*N*`-mac`*M*`/t2h/`*X* on Tile Linux.

  If you intend to copy data from the Tile-side to the Host-side, use APIs to open a T2H channel on both the Tile and host sides, with the Tile-side preparing a send buffer and the Host-side preparing a receive buffer.

  If you intend to copy data from the Host-side to the Tile-side, use APIs to open a H2T channel on both the Tile and host sides, with the Host-side preparing a send buffer and the Tile-side preparing a receive buffer.

  If your applications needs to both read and write, have it open both a H2T channel and a T2H channel.

  For a pair of processes between two TILE-Gx PCIe ports operating in either endpoint or root complex mode, the processes can exchange data buffers through zero-copy command queues using the GXPCI library. For more information, see the *Application Libraries Reference Manual* (UG527).

6.3.2 Getting the Host Link Index Information

Each TILE-Gx endpoint port has a unique link index number in a PCI domain, with number 0 reserved for the root-complex port. You can get a unique link index (N + 1) of each TILEncore-Gx Intelligent Application Adapter via `HOST_LINK_INDEX` from the file `/dev/tilegxpci`*N*`/info`.

The local EP port link index is the number in the last column of the file `/proc/driver/gxpci_ep_major_mac_link`.

To get this information, use this command:

```
#cat /proc/driver/gxpci_ep_major_mac_link
247 trio0-mac0 1
```

The information in the file is organized this way:

- The number in the first column is the major device number.
- The second column includes the TRIO instance number *N*, and the PCIe port number *M*.
- The last column is the link index in the PCI domain to which this port belongs.

6.3.3 Supporting Multiple PCIe Endpoint Ports

The TILE-Gx PCIe host driver supports boards with multiple PCIe Endpoint ports from a single TILE-Gx processor, supporting up to 6 endpoint ports per chip.

For TILEncore-Gx72 cards, the PCIe host driver supports host device names in the form of:

- `/dev/tilegxpci`*N*, where *N* starts at 0, for the first port of the board *N*
- `/dev/tilegxpci`*N*`-link`*M*, where *M* starts at 1, for other ports on the board

Note: The PCIe host driver also establishes `/dev/tilegxpci`*N*`-link`*M* as soft links to the `/dev/tilegxpci_`*port_major#* devices.

6.3.4 Accessing PCIe Root Complex Mode

The TILE-Gx Processor runs as a PCIe hosted device by default. However, with root complex mode enabled, the TILE-Gx Processor can run as the system host. For example, the TILE-Gx Processor can detect other PCIe cards, such as a hard disk controller, and use those devices as any Linux system might. It can also run as a root complex on one PCIe interface and as an endpoint on another. In other words, you can use either or both the root complex and endpoint modes simultaneously.

Tile Linux automatically scans the PCI bus at boot time and loads device drivers as needed. It might be necessary to compile and upload additional Linux drivers to support any particular device.

6.3.5 Configuration Settings for PCIe Port Operating Mode

You can configure the port operating mode to either the endpoint or root complex mode using these methods:

- In the board information block (BIB). For more information, see the *Tilera Board Information Block Format* (AN040).
- With a port strapping pin, which overrides the BIB entry.

6.3.6 Configuring Asymmetrical Packet Queues

By default, the x86 Linux host driver (`tilegxpci.ko`) provides you with four Tile-to-Host interfaces and four Host-to-Tile interfaces. You can configure an asymmetrical number of H2T and T2H Packet Queue interfaces.

Note: In a packet capture application, it is likely that the number of Tile-to-Host queues is larger than the number of Host-to-Tile queues.

You can open an asymmetrical number of Packet Queue interfaces by doing the both of the following:

1. Install the x86 Linux host driver (`tilegxpci.ko`), adding:

   ```
   pq_h2t_queues=M pq_t2h_queues=N
   ```

   to the host driver module argument list, where *N* is the number of Tile-to-Host queues and `M` is the number of Host-to-Tile queues.

2. Boot Tile Linux on a TILEncore-Gx PCIe card, adding the `tile-monitor` options:

   ```
   --hvx pcie_host_pq_h2t_queues=T,P,M
   --hvx pcie_host_pq_t2h_queues=T,P,N
   ```

   where *T* is the TRIO instance number, *P* is the PCIe port number, *N* is the number of Tile-to-Host queues and *M* is the number of Host-to-Tile queues.

Note: You must configure the x86 Linux host driver and the Tile Linux kernel to use the same settings for the numbers of Tile-to-Host queues (*N*) and Host-to-Tile queues (*M*).

For example, boot Tile Linux and install the default x86 Linux host driver to support five Tile-to-Host queues (*N*) queues and three Host-to-Tile queues (*M*), do the following:

1. Install the x86 Linux host driver using this command:

```
# $TILERA_ROOT/lib/modules/tilepci-install pq_t2h_queues=5 \
pq_h2t_queues=3
```

2. Boot the TILEncore-Gx PCIe Card using this `tile-monitor` command:

```
$ tile-monitor --hvx pcie_host_pq_t2h_queues=0,0,5 \
--hvx pcie_host_pq_h2t_queues=0,0,3
```

Note: Due to TILE-Gx PCIe hardware constraints, the sum of Tile-to-Host queues and Host-to-Tile queues must not exceed eight.

6.3.7 Configuring the Host Ring Buffer Size

The host ring buffer that is available in the x86 Linux host driver (`tilegxpci.ko`) is composed of one or more discreet segments. Each segment is a chunk of physically contiguous memory that is divided into individual packet buffers of equal sizes. By default, the host ring buffer has a single segment of size 4MB, consisting of 1024 entries of 4KB packet buffers. The maximum segment size is determined by the size of maximum physically contiguous memory that can be allocated on the host system. On a generic x86 Linux host system, 4MB is the maximum physically contiguous buffer that can be allocated, without reconfiguring the host kernel.

To request a host ring buffer size larger than 4MB, use the x86 Linux host driver parameters `pq_t2h_mems=` and `pq_h2t_mems=` to specify the total ring buffer sizes of the tile-to-host and host-to-tile packet queue interfaces, respectively.

The parameters take the following format:

`[`*`size`*`][,`*`numa_node`*`],[`*`share_ring`*`][:[`*`size`*`][,`*`numa_node`*`]],[`*`share_ring`*`]`

Each parameter triplet specifies the following:

`size:` Total ring buffer size, with a default of 4MB. You can set sizes using a suffix of K for kilobytes, M for megabytes and G for gigabytes.

`numa_node:` NUMA zone of the buffer, with a default of 0.

`share_ring:` If the ring buffer is shared between the T2H and H2T interfaces with the same interface number, with 1 indicating a shared buffer. By default, this parameter is 0 since each unidirectional interface has its own ring buffer. Using a shared ring buffer is preferred in loopback-like applications because such a shared buffer can avoid the expensive memory copy operation when queuing an ingress (T2H) packet to the egress (H2T) queue.

The relative position of the triplet in the parameter string indicates the Packet Queue interface number.

The following is an example of a host driver installation command setting the buffers to 2MB using shared ring buffers:

```
tilepci-install pq_t2h_mems=2M,0,1 pq_h2t_mems=2M,0,1
```

Note: The triplet setting for the H2T/T2H queues with the same interface number must be identical.

You define the maximum number of segments in a host ring buffer using the macro `HOST_PQ_SEGMENT_MAX_NUM`. The maximum allowed value is 32, which means that you can request up to 128MB for the ring buffers. Use the macro `GXPCI_HOST_PQ_SEGMENT_ENTRIES` to define the number of buffer entries in a segment.

Note: The value of GXPCI_HOST_PQ_SEGMENT_ENTRIES is set to 1024 by default. We strongly suggest that you do not change this value. A 128MB ring buffer provides 32*1024 packet buffers. You can set the individual packet buffer size to any value that is smaller than 4096-byte and is a multiple of two.

6.3.8 Configuring gxpci Ethernet Devices in the Host Driver

By default, when you run the tilepci-install command, you install the tilegxpci.ko driver. The host driver provides you with gxpci*N-M* Ethernet devices. On the x86 Linux host, both the tilegxpci and tilegxpci_nic host drivers provide you by default with 4 Ethernet devices named gxpci1-0, gxpci1-1, gxpci1-2 and gxpci1-3.

The naming convention is gxpci*N-M*, where *N* is the link number of the TILEncore-Gx PCIe adapter (which is always started at 1), and *M* spans from 0 to the total number of host-NIC ports defined for the tilegxpci driver that you install on the x86 Linux Host. The default number of Host NIC ports is 4. These devices correspond to the four XGbE ports on the TILEncore-Gx Intelligent Application Adapter.

Note: To install the tilegxpci_nic host driver, run the tilepci-install command with the netlib argument.

You can change the configuration of the gxpci*N-M* Ethernet interfaces using these host driver arguments:

- nic_ports=*M*.

  Sets the number of host-NIC ports, which corresponds to the variable *M* in the Ethernet devices named gxpci*N-M*. The default is 4. The range of possible values spans from 1-4.

  To use the host-NIC ports on the Tile-side, you must use the pcie_host_vnic_ports= Tile-side Linux boot argument to configure the number of SDNIC ports in vmlinux, using the same values for T,P and M as for the nic_ports= x86 Linux host driver argument.

- nic_rx_queues=*Q*.

  Sets the number of receiver packet queues for to create for each host-NIC port. The default is 1. For the default 4 host-NIC interfaces, you can set 1 or 2 receiver packet queues. If you configure only 1 host-NIC port, you may configure up to 10 receiver packet queues.

  To use these receiver packet queues on the Tile-side, you must use the pcie_host_vnic_rx_queues= Tile-side Linux boot argument to configure the number of receiver packet queues in vmlinux, using the same values for T,P and Q as for this nic_rx-_queues= x86 Linux host driver argument.

- nic_tx_queues=*Q*.

  Sets the number of transmitter packet queues for to create for each host-NIC port. The default is 1. For the default 4 host-NIC interfaces, you can set 1 or 2 transmitter packet queues. If you configure only 1 host-NIC port, you may configure up to 10 transmitter packet queues.

  To use these transmitter packet queues on the Tile-side, you must use the pcie_host_vnic_tx_queues= Tile-side Linux boot argument to configure the number of transmitter packet queues in vmlinux, using the same values for T,P and Q as for this nic_tx_queues= x86 Linux host driver argument.

  Note: You must match the number of transmitter and receiver packet queues.

6.3.9 Using the gxpci Ethernet Devices

The PCIe host driver's host-NIC component (hereafter referred to as host-NIC driver) abstracts each host-NIC interface using the Linux kernel `net_device` structure, enabling the same kernel/driver API to the host-NIC interface as is available to regular Ethernet NIC devices.

For details about configuring host-NIC channels, see the `host-NIC driver` options in Section 6.3.8 "Configuring gxpci Ethernet Devices in the Host Driver" on page 79. For details about using the Point-to-Point driver, see Section 6.3.11 "Using a PCIe Point-to-Point Connection" on page 84.

You can assign IP addresses to the devices using the `ifconfig` command, such as:

```
ifconfig gxpci1-0 192.168.300.101 up
```

By default, issuing the `ifconfig -a` command on an x86 Linux host which has TILEncore-Gx Intelligent Application Adapter installed results in messages like the following:

```
ifconfig -a
eth0      ...
gxpci1-0  Link encap:Ethernet  HWaddr E2:9E:55:63:DB:80
          BROADCAST NOARP  MTU:4000  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 b)  TX bytes:0 (0.0 b)
          Interrupt:17

gxpci1-1  Link encap:Ethernet  HWaddr DA:86:8F:A7:10:E0
          BROADCAST NOARP  MTU:4000  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 b)  TX bytes:0 (0.0 b)
          Interrupt:17

gxpci1-2  Link encap:Ethernet  HWaddr 52:1F:5B:40:F7:62
          BROADCAST NOARP  MTU:4000  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 b)  TX bytes:0 (0.0 b)
          Interrupt:17

gxpci1-3  Link encap:Ethernet  HWaddr DA:76:7E:BF:74:24
          BROADCAST NOARP  MTU:4000  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 b)  TX bytes:0 (0.0 b)
          Interrupt:17

tilep2p1  Link encap:Ethernet  HWaddr 8A:C4:5D:AE:8B:D9
          BROADCAST MULTICAST  MTU:4000  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 b)  TX bytes:0 (0.0 b)
          Interrupt:17
```

6.3.10 Tunneling Network Traffic over PCIe

This section shows how to establish network connectivity between a TILEncore-Gx Intelligent Application Adapter and the x86 Linux host's network over PCIe. The basis for this solution is to create an IP tunnel over a PCIe zero-copy channel to deliver IP traffic between the TILEncore-Gx Intelligent Application Adapter and the host.

Note: The zero-copy channels provided by this tunnel cannot be used by other PCIe zero-copy applications.

To establish an IP tunnel across a PCIe zero-copy channel, run the utility program `/usr/sbin/pci-net` on the TILEncore-Gx Intelligent Application Adapter and `$TILERA_ROOT/bin/tile-pci-net` on the x86 Linux host. The utility configures a point-to-point interface and then runs in an infinite loop, forwarding packets between the PCIe device and the Linux kernel TUN device.

You must boot the TILEncore-Gx PCIe card with Linux configured for network over PCIe, by specifying a `TLR_NETWORK` boot option, for example:

```
$ tile-monitor --dev gxpci0 --root --hvx TLR_NETWORK=pci-net
```

Alternatively, you may use the `tile-monitor` option `--upload-tile /usr/sbin` instead of `--root` in order to be able to quit from `tile-monitor` without introducing issues with accessing FUSE-mounted directories that are no longer available.

Using the tile-pci-net and pci-net Programs

You can run the `tile-pci-net` host-side and `pci-net` tile-side programs using similar and related options. Use the `pci-net` (and the `tile-pci-net`) utility as follows:

```
$ [tile-]pci-net [-c pcie-card-number] [-s ZC-channel-number]
[-l local-IP-address] [-r remote-IP-address]
```

The arguments to `pci-net` are all optional, and are as follows:

- *`pcie-card-number`* — Specifies the desired PCIe card number: 0 (default) or 1.
- *`ZC-channel-number`* — Specifies the desired PCIe zero-copy channel number, with 1 (default) being the only option.
- *`local-IP-address`* — Specifies the IP address for the local interface of the P-2-P link.
- *`remote-IP-address`* — Specifies the IP address for the remote interface.

Note: To establish a particular connection, use the same *`pcie-card-number`* and *`ZC-channel-number`* arguments must be the same for `tile-pci-net` and `pci-net` commands on the host and the tile system, respectively. You swap the *`local-IP-address`* and *`remote-IP-address`* for the Host and Tile-side commands.

If you do not provide either of the IP address arguments, the utility derives a default address for a given *`pcie-card-number N`* and *`ZC-channel-number M`*, as follows:

- Host side: 192.168.100+*N*.100+*M*
- Tile side: 192.168.100+*N*.200+*M*

For example, if you do not provide IP address arguments, the host-side interface IP address is 192.168.100.101, and the tile-side interface IP address is 192.168.100.201.

Note: You must invoke the Tile-side `pci-net` and Host-side `tile-pci-net` commands within 90 seconds of each other, otherwise the pipeline times out.

Establishing Route and Gateway Settings for the PCIe Tunnel

To establish routing and gateway settings, follow these steps (assuming that you have read Section "Using the tile-pci-net and pci-net Programs" on page 81):

1. Run `/usr/sbin/pci-net` on the TILEncore-Gx Intelligent Application Adapter.
2. Run `$TILERA_ROOT/bin/tile-pci-net` on the host.
3. Use the `route` command on any remote machines to specify the TILEncore-Gx Intelligent Application Adapter's host as the gateway for the TILEncore-Gx Intelligent Application Adapter's IP address:

   ```
   $ route add card_IP_address gw host_IP_address
   ```

   For example, assuming that the x86 Linux host is on the sub-net 172.16.15.*x* and that the LAN-facing address for the x86 Linux host is 172.16.15.14, use this route command:

   ```
   $ route add 192.168.100.201 gw 172.16.15.14
   ```

4. On the shell or console for the TILEncore-Gx Intelligent Application Adapter, use the `route` command to let the card list the host as its default gateway or gateway for a specific remote machine:

   ```
   # route add default gw host_pci_net_IP_address
   ```

   or:

   ```
   # route add remote_IP_address gw host_pci_net_IP_address
   ```

   For example, to let the card list the x86 Linux host as its default gateway, use this command:

   ```
   # route add default gw 192.168.100.101
   ```

   To let the card list the gateway for a specific remote machine with the LAN address of 172.16.15.60, use this `route` command:

   ```
   # route add 172.16.15.60 gw 192.168.100.101
   ```

5. On the x86 Linux host, turn on IP forwarding for the host, using this command:

   ```
   $ echo 1 > /proc/sys/net/ipv4/ip_forward
   ```

   Or to enable when the host boots, edit the file `/etc/sysct1.conf` to contain:

   ```
   net.ipv4.ip_forward = 1
   ```

Figure 6-1 illustrates the various IP addresses involved in this Tunneling route and gateway example.

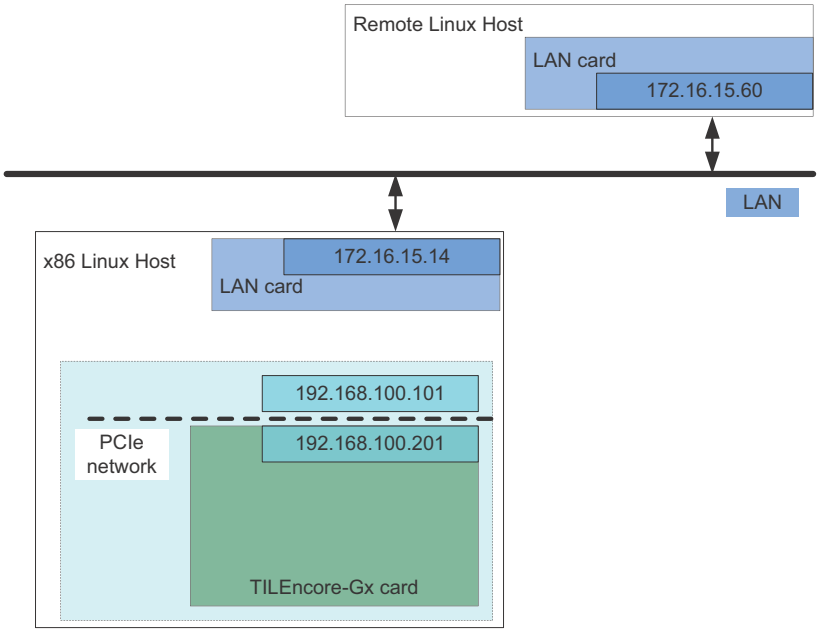

Figure 6-1: IP Address Examples for Tunneling Traffic Over PCIe

Configuring the PCIe Tunnel Interface on the x86 Linux Host

On the TILEncore-Gx Intelligent Application Adapter, the default command `pci-net` configures the interface as a P-2-P interface with local IP address 192.168.100.201.

The `ifconfig` command shows the following:

```
tun0    Link encap:UNSPEC HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
        inet addr:192.168.100.201 P-t-P:192.168.100.101
        Mask:255.255.255.255
        UP POINTOPOINT RUNNING NOARP MULTICAST MTU:1500 Metric:1
        RX packets:0 errors:0 dropped:0 overruns:0 frame:0
        TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:500
        RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

You can invoke the `pci-net` command for the TILE-Gx device using these methods:

- Manually, in the `tile-console` window
- By supplying `--hvx TLR_NETWORK=pci-net` to the `tile-mkboot` or `tile-monitor` commands so that the TILEncore-Gx Intelligent Application Adapter runs the command `pci-net &` automatically on the tile after booting.

Configuring the PCIe Tunnel Interface on Boot

On the host, you must manually run the command:

```
$TILERA_ROOT/tile-pci-net &
```

You must be logged in as `root` to run this command, as configuring a network interface is a privileged operation.

Once you invoke `tile-pci-net` command, an `ifconfig` command shows the following:

```
tun0    Link encap:UNSPEC HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
        inet addr:192.168.100.101 P-t-P:192.168.100.201
        Mask:255.255.255.255
        UP POINTOPOINT RUNNING NOARP MULTICAST MTU:1500 Metric:1
        RX packets:0 errors:0 dropped:0 overruns:0 frame:0
        TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:10
        RX bytes:0 (0.0 b) TX bytes:0 (0.0 b)
```

Now, you can `telnet` into the tile system by running the command:

```
$ telnet 192.168.100.201
```

6.3.11 Using a PCIe Point-to-Point Connection

The MDE provides network-over-PCIe connectivity between the x86 Linux host and a TILE-Gx endpoint using the in the x86 Linux host driver (`tilegxpci.ko`) through a PCIe point-to-point NIC port called `tilep2p`*N*.

To use the Ethernet interfaces that the PCIe point-to-point host driver provides, do the following:

1. On the TILEncore-Gx Intelligent Application Adapter, configure an IP address for the `p2p` port, using the `ifconfig` command, for instance:

   ```
   ifconfig tilep2p1 192.168.0.2 up
   ```

2. On the x86 Linux host, open the `p2p` interface by using the `ifconfig` command, for instance:

   ```
   ifconfig tilep2p1 192.168.0.1 up
   ```

3. Demonstrate connectivity using `ping`.

   a. From the x86 Linux host, run this command:

      ```
      ping 192.168.0.2
      ```

   b. From the TILEncore-Gx Intelligent Application Adapter:

      ```
      ping 192.168.0.1
      ```

Note: These instructions assume that you run these commands on the first TILEncore-Gx adapter in your x86 Linux host. If you have two TILEncore-Gx adapters installed, and use the second card for these tests, use the `gxpci` port number Linux `tilep2p2`.

6.3.12 Setting BIOS Memory for SR-IOV Virtual Functions

Each TILE-Gx PCIe Virtual Function consumes at least 33MB of Base Address Register (BAR) space, which is scarce resource. Multiplying that with the number of Virtual Functions per port, and the number of ports, could exceed the limit set by the BIOS on some Linux x86 host machines.

If your Linux x86 host machine fails to initialize SR-IOV, you might be running out of Base Address Register (BAR) space.

The host kernel may display this message in failing the SR-IOV initialization:

```
tilegxpci 0000:08:00.0: not enough MMIO resources for SR-IOV
```

A potential fix is to modify a PCI memory mapping setting in BIOS on that host. To allow enough BAR space, try changing the BIOS settings of Memory Mapping, which is most often available as:

BIOS > Advanced > PCI Configuration > Set Memory Mapped I/O above 4 GB to Enabled

6.3.13 Preserving TILEncore-Gx MAC Links During Reset

For the `tilegxpci` Linux PCIe host driver, the default value for the `gxpci_reset_all` option is 1, which has the effect that when a host initiates a TILE-Gx chip reset via PCIe, this action resets all chip components including the PCIe MACs. The resulting PCIe link down event can trigger fatal reactions on some hosts. To avoid the problem on these hosts, install the PCIe host driver with the option `gxpci_reset_all=0`, which causes the TILE-Gx chip reset to not reset the PCIe MACs, keeping the PCIe links up.

To accomplish this, you can either:

- Supply the `gxpci_reset_all=0` argument to the `tilepci-install` command when installing the PCIe host driver as root on the Linux x86 host machine in which the TILEncore-Gx adapter is installed:

  ```
  # tilepci-install gxpci_reset_all=0
  ```

- Use this command on the Linux x86 host machine in which the TILEncore-Gx adapter is installed:

  ```
  # echo 0 > /sys/module/tilegxpci/parameters/gxpci_reset_all
  ```

- Add the following configuration options to the `/etc/modprobe.d/tilegxpci.conf` file on the Linux x86 host machine in which the TILEncore-Gx adapter is installed:

  ```
  options tilegxpci gxpci_reset_all=0
  ```

6.4 MiCA Driver

The MiCA driver provides access to cryptography and compression/decompression acceleration. The driver supports direct application access by means of the `gxcr_` routines in the GXCR library. See the *MDE MiCA Programmer's Guide* (UG507) and the *Application Libraries Reference Manual* (UG527) for information about MiCA application programming.

In order to access MiCA, you must specify the following definitions in the hypervisor configuration file:

```
device? comp/0
device? comp/1
device? crypto/0
device? crypto/1
```

You configure all device options by means of the `gxcr` and `gxio` APIs.

6.4.1 Enabling User-Space PKA Driver Access to PKA Shims

This release provides a user-space PKA driver, which can access PKA registers from user-space. By default, the Linux PKA and TRNG drivers take over the PKA hardware. The Linux PKA driver allocates both crypto shims on a TILE-Gx36™ processor, and the TRNG driver uses one of those two. The Linux PKA and TRNG drivers can share a shim with each other, but shims cannot be shared between Linux and user-space. To use the user-space PKA driver, you must configure the Linux PKA driver to give up one or both shims. Either remove the PKA driver from the Linux configuration or use a `tile-monitor` command to set a boot-time parameter to tell the Linux PKA driver to use no shim or only 1 shim (the default is to use 2 shims). To instruct both the PKA and TRNG drivers to not use the shims from `tile-monitor`, use this command:

```
tile-monitor --hvx pka.num_pka_accels=0 --hvx tile-rng.num_hw_trngs=0
```

6.5 GPIO Driver

The GPIO driver provides access to GPIO pins.

The driver supports direct application access by means of the `gxio_gpio_*` routines in the GXIO library. See the *Application Libraries Reference Manual* (UG527) for information on GPIO application programming.

In order to access GPIO, you must specify `device? gpio/0` in the hypervisor configuration file. You configure all device options by means of the `gxio` API.

Note that the set of available GPIO pins, and the devices they are connected to, are properties of the specific board or platform being used. Consult the appropriate documentation to find out which pins are available.

6.6 Tile-side USB Host Driver

Tilera provides a native (tile-side) host driver for the TILEmpower-Gx and TILExtreme-Gx appliances. This enables tile-side USB host driver support, allowing you to connect USB devices to TILE-Gx and have them show up under Tile Linux.

You may enable the drivers on all platforms, but some platforms may not connect all possible USB host interfaces. TILEmpower-Gx and TILExtreme-Gx appliances provide one host interface (`usb_host/1`). There is no host interface on the TILEncore-Gx Intelligent Application Adapter.

Driver

To enable the native (tile-side) host driver for the TILEmpower-Gx and TILExtreme-Gx appliances, you must specify `device? usb_host/1` in the hypervisor configuration file.

Note: There is no `gxio` API for the USB host shim. Use the standard Linux USB stack to access USB host devices.

6.7 SROM Driver

The SROM driver provides access to a Serial Peripheral Interconnect Read-Only Memory (SPI ROM) or flash ROM device.

The normal purpose of the SROM device is to hold software used to boot the Tile Processor when it runs standalone. It can also be used to hold system or application data that must be specific to a particular card and must be persistent across power cycles.

6.7.1 Hypervisor Configuration

This section describes hypervisor configuration settings for SROM devices.

Drivers

The SROM device is named `srom`, with one instance: `srom/0`. There is only one driver for the `srom` device, also named `srom`.

Tiles

The `srom` driver requires one shared tile, and does not require any dedicated tiles. The purpose of the shared tile is to perform programmed I/O operations to the SROM hardware as necessary to accomplish the requested read or write operations. The shared tile caches data recently read from the ROM, to reduce the latency of subsequent reads. It also caches data that is destined to be written to the ROM; this allows data to be written in larger chunks, improving write performance.

The SROM interface is fairly slow and, as noted above, is accessed through programmed I/O, not DMA. Therefore, large read or write operations can tie up both the shared driver tile and the initiating tile for relatively long periods of time. Because of this, it is inadvisable to run a time-critical process on the initiating tile.

Driver Arguments

There are no driver arguments for the `srom` driver.

6.7.2 Usage

A set of character device files in `/dev/srom/` provide access to the SROM from Linux. Tilera currently configures three such devices:

- `/dev/srom/bootimage` is used by the system to modify the bootable system software contained within the SROM. Typically, you would not access this partition; instead, use the `sbim` utility to manage the various bootable images contained in the SROM.

  See Chapter 5: Using the mboot Boot Loader. See Chapter 4: Making a Bootrom for information on how to set up a Linux image.

- `/dev/srom/bootparam` is used to hold configuration data for `mboot`.
- `/dev/srom/userparam` can be used to hold application data.

The sizes of these files will vary, depending on the size of the SROM. On the TILEmpower-Gx, `/dev/srom/bootparam` and `/dev/srom/userparam` are each 512 KB, and `/dev/srom/bootimage` is approximately 14.5 MB.

As these files are character devices, standard Linux command-line tools, like `cat` or `dd`, can read from or write to them. Applications can access them directly using either system calls, like `read()` or `write()`, or C library functions, like `fread()` or `fwrite()`.

If you do write applications that directly access the ROM, keep the following in mind:

- As mentioned above, the device shared tile does cache write data to improve performance. To ensure that all data written has actually been written to non-volatile storage, the Linux file associated with the ROM must be closed. When the `close()` call returns, the data written will have been flushed to the ROM. Applications should be careful not to alter the `bootimage` and `bootparam` partitions, or the system could be rendered unbootable from SROM.
- The SROM technology requires that data be written in fairly large chunks, called sectors. If a power failure or system crash were to occur while one part of a sector was being written, it is possible that other data within that sector might become corrupted. Because of this, if you want to store critical data in the ROM, and want it to be available even after such a failure, Tilera recommends that you write multiple copies of the data to different sectors, and protect your data with some sort of checksum. The sector size will vary, depending on the ROM used. On the TILEmpower-Gx card, it is 256 KB.

6.7.3 Examples

The following is an example of the hypervisor configuration file content for an SROM device:

```
device srom/0
```

6.8 memprof Driver

The memory profiling driver is a pseudo-device that provides information about the throughput and latency of the memory system.

6.8.1 Hypervisor Configuration

This section describes hypervisor configuration settings for the memory profiling device.

Drivers

The memory profiling device is named *memprof*, with two instances: `memprof` which is the dedicated mode instance, and `memprof_shared`, which is the shared mode instance.

The `devices.hvh` file contains the following memory profiling settings:

```
ifdef MEMPROF_TILE
# Optionally enable memprof using a given dedicated tile.
device memprof memprof
  dedicated $MEMPROF_TILE
elif defined(MEMPROF)
# Optionally enable memprof using the default shared tile.
device memprof memprof_shared
endif
```

Tiles

The `memprof` driver requires one shared tile, but can run on a single dedicated tile. The purpose of the tile is to collect statistics from each memory controller at very frequent, regular intervals, so that profiling data is complete and up to date.

Driver Arguments

There are no arguments for the `memprof` driver.

6.8.2 Usage

The `mcstat` command is the recommended user interface to the memory profiling facility (see the `mcstat` manpage for a description of its usage and options). See the *Multicore Development Environment Optimization Guide* (UG515) for background information on the `mcstat` program. For a description of the Linux API to the profiling driver itself, see the `memprof` manpage.

6.8.3 Running memprof in Shared Mode

To turn on the memprof process in shared mode on Tile 0,0, invoke the `tile-monitor` command with the following argument:

```
--hvd MEMPROF
```

6.8.4 Running memprof in Dedicated Mode

To turn on the memprof process in dedicated mode on Tile 3,5, invoke the `tile-monitor` command with the following argument:

```
--hvd MEMPROF_TILE=3,5
```

The following is an example of configuring the `memprof` driver to use one dedicated tile:

```
device memprof
   dedicated $MEMPROF_TILE
```

6.9 I^2C Driver

The I^2C adapter driver provides access to the I^2C devices connected to the processor's I^2C Master interface.

6.9.1 Hypervisor Configuration

This section describes hypervisor configuration settings for the I^2C Master device.

Drivers

There is only one driver, `i2cm`, for the I²C Master device, named `i2cm/0`.

Tiles

The `i2cm` driver should be assigned to one shared tile in the `.hvc` file. For example:

```
device i2cm/0 i2cm
   shared 0,3
```

If no shared tile is specified here, the `i2cm` driver uses the default shared tile.

Driver Arguments

There are no arguments for the `i2cm` driver.

Board Information Block (BIB) Setting

The hypervisor controls which I²C devices can be accessed by Linux. To enable access to I²C devices, the system Board Information Block (BIB) must include one `i2c_dev_cfg` description entry for each I²C slave device.

I²C devices that do not have the corresponding description entries in the BIB are not accessible to Linux. One example of these invisible devices is the EEPROM chip that contains the BIB itself.

For more information, see *Tilera Board Information Block Format* (AN040).

6.9.2 Linux Client Driver Configuration

The Linux client driver must be provided in order to access the corresponding I²C devices through the I²C adapter driver within the framework of the Linux kernel I²C stack.

I²C Adapter Driver

By default, the Tilera Linux I²C adapter driver is configured into the kernel by means of the `CONFIG_I2C_TILE` configuration option.

I²C Slave Driver

An I²C slave driver is described by a kernel `i2c_driver` structure that includes an `i2c_device_id` structure. To attach an I²C slave device to its device driver, the driver must set the `name` field of the `i2c_device_id` structure to a name that is identical to the character string that is embedded in the device's `i2c_dev_cfg` BIB entry.

6.9.3 Usage

This section describes two Linux user interfaces for the I²C slave devices.

/dev Interface

The I²C devices can be accessed through the Linux special file `/dev/i2c-0`. Refer to the example C programs, which can be found in the Linux source documentation file `Documentation/i2c/dev-interface`.

/sys Interface

The I²C devices can also be accessed through the `/sys` interface. For example, an EEPROM device with address 0x52 can be accessed by issuing file I/O requests against the following file: `/sys/devices/platform/tile-i2c/i2c-adapter/i2c-0/0-0052/eeprom`.

For example:

```
dd if=/sys/devices/platform/tile-i2c/i2c-adapter/i2c-0/0-0052/
\eeprom of=/tmp/eeprom
```

6.10 Watchdog Driver

The watchdog driver provides access to the TILE-Gx Rshim watchdog device, (the first of the three independent 48-bit TILE-Gx down-counters). For more information, see Chapter 14.4 "Down-Counters and Watchdog" in *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404).

6.10.1 Hypervisor Configuration

This section describes hypervisor configuration settings for the watchdog devices.

Driver

There is only one driver, `watchdog`, for the watchdog device, named `watchdog`.

Tiles

The watchdog driver is disabled by default. For more information, see the `devices.hvh` file.

Driver Arguments

There are no arguments for the `watchdog` driver.

Board Information Block Setting

To enable the access to the watchdog device, the system Board Information Block (BIB) must include the corresponding watchdog device entry. For more information, see the *Tilera Board Information Block Format* (AN040).

6.10.2 Linux Client Driver Configuration

In order to access the watchdog device, you must configure the Linux client driver with the kernel configuration option `CONFIG_WATCHDOG_TILE`.

6.10.3 Usage

You can access the watchdog device through the Linux special file `/dev/watchdog` with the Linux watchdog driver API. For more information, see the Linux kernel document `Documentation/watchdog/watchdog-api.txt`.

6.10.4 Example

To turn on the watchdog process, invoke the `tile-monitor` command with the following argument: `--hvd WATCHDOG=1`

The console for that device displays:

```
Tilera watchdog found. Default timeout=20 sec, nowayout=y
```

6.11 TMFIFO Driver

The TMFIFO driver provides access to the rshim's `tile-monitor` FIFO, which is used to support data transfer between the Tile Processor and a host computer connected by means of USB.

The TMFIFO driver is automatically configured and need not be specified in the hypervisor configuration file.

The driver requires one shared tile, which is currently unused, but which in the future might service hardware interrupts.

CHAPTER 7 DEVELOPING HYPERVISOR DRIVERS

This chapter describes:

- Section 7.1 Hypervisor Drivers in MDE 4.x
- Section 7.2 Getting Started with Hypervisor Drivers
- Section 7.3 Optimizing Device Drivers
- Section 7.4 Building a Modified Hypervisor

See also Appendix A— Hypervisor Console Escape Commands.

7.1 Hypervisor Drivers in MDE 4.x

The role of hypervisor drivers has changed substantially in MDE 4.*x*, compared with previous releases.

On TILE*Pro* processors, the hypervisor was responsible for device configuration and management, but was also involved in every device I/O operation. For example, an incoming Ethernet frame was processed by one or more IPP tiles, doing tasks like header parsing and buffer management, before an arrival notification was delivered to its destination tile by means of the IDN. Even when the notification reached that tile, a small bit of hypervisor code was run there before the packet was available to an application. Similarly, a PCIe register read or write that might have been generated by the Linux root complex driver was relayed to one or more dedicated tiles in the hypervisor PCIe driver, which was responsible for formatting the actual PCIe transaction and sending it out to the I/O shim.

On processors in the TILE-Gx family, the hypervisor is still responsible for initial device configuration, memory mapping management, and some device management functions (for example, managing Ethernet PHYs). However, it no longer performs the time-critical functions that happen on every I/O operation. Functions such as packet parsing, buffer management, and transaction generation are performed directly in hardware, or by dedicated processors like the mPIPE classifier. Other functions are performed by supervisor device drivers (like the native Linux Ethernet driver) or by applications.

This new software division of labor is made possible by the memory-mapped I/O (MMIO) architecture of the TILE-Gx I/O shims. This architecture allows direct access to those shims from Linux kernel code or even from applications running in user space. It is aided by new virtualization and protection features in the I/O shims that allow the hypervisor to delegate responsibility for some, but not all, I/O operations to lower protection levels.

This change in responsibility means that for most users, there should be far less need to write new hypervisor drivers or even customize the drivers provided by Tilera, compared with previous MDE versions. However, to the extent that such customizations are still useful, this chapter describes the general principles of device driver operation. Significant modifications to drivers will, as always, require study of the hypervisor source code.

7.2 Getting Started with Hypervisor Drivers

The hypervisor includes drivers for most I/O shims on the Tile Processor™. These drivers are:

- mPIPE driver, which controls the mPIPE network interface shims. This driver is at `$TILERA_ROOT/src/sys/hv/tilegx/drivers/mpipe`. See Section 6.2 “mPIPE Driver” on page 74.
- TRIO driver, which controls the TRIO PCIe interface shims. This driver is at `$TILERA_ROOT/src/sys/hv/tilegx/drivers/trio`. See Section 6.3 “TRIO Driver” on page 75.
- MiCA driver, which controls the MiCA cryptography and compress/decompression acceleration shims. This driver is at `$TILERA_ROOT/src/sys/hv/tilegx/drivers/mica`. See Section 6.4 “MiCA Driver” on page 85.
- GPIO driver, which controls the general-purpose I/O (GPIO) pins. This driver is at `$TILERA_ROOT/src/sys/hv/tilegx/drivers/gpio`. See Section 6.5 “GPIO Driver” on page 85.
- Serial Peripheral Interface (SPI). This driver is at `$TILERA_ROOT/src/sys/hv/drivers/srom`. See Section 6.7 “SROM Driver” on page 86.
- Memprof driver (`sys/hv/drivers/memprof/`), which collects memory-performance statistics from the four DDR controllers. This driver is at `$TILERA_ROOT/src/sys/hv/drivers/memprof`. See Section 6.8 “memprof Driver” on page 87.
- Inter-Integrated-Circuit (I2C) driver, a low-speed two-wire serial interface. On Tilera’s board and platform products, this is used for controlling card features, although no driver interface is exported. This driver is at `$TILERA_ROOT/src/sys/hv/drivers/i2cm`. See Section 6.9 “I2C Driver” on page 88.
- Watchdog driver, provides access to the watchdog device connected to the processor’s I^2C Master interface. See Section 6.10 “Watchdog Driver” on page 90.
- TMFIFO driver, which supports communication between the Tile Processor and a host computer system. This driver is at `$TILERA_ROOT/src/sys/hv/tilegx/drivers/tmfifo`. See Section 6.11 “TMFIFO Driver” on page 90.
- Mshim driver, which probes the DDR controllers, but exports no other operations. This driver is at `$TILERA_ROOT/src/sys/hv/tilegx/drivers/mshim`.

7.2.1 Other Drivers

The Tilera® Linux kernel device drivers (in `sys/linux/arch/tile/drivers/`) use hypervisor drivers for off-chip I/O. Linux drivers include GbE Ethernet drivers and the `memprof` driver (`sys/linux/arch/tile/drivers/memprof.c`).

Another driver included with the MDE is a Linux host PCIe driver. This driver runs on the host computer and communicates with the chip’s PCIe driver in its endpoint mode. On the TILExpress card, it manages bootload and host support. Similarly, a Linux host USB driver manages communication with systems, such as the TILEmpower-Gx, that are accessed by means of USB.

7.2.2 Controlling and Extending Drivers

To support the Tile Processor™ in a variety of applications, some of the hypervisor drivers have control interfaces that modify their behavior during operation. Some hypervisor drivers can be configured at startup with device argument subcommands in the hypervisor configuration file, discussed in Section 2.2.1 “Hypervisor Configuration File Format” on page 16. Each device driver implements its own set of subcommands.

7.2.3 Driver Interfaces

Device drivers run at an elevated privilege level, have access to physical memory, and use the Internal Dynamic Network (IDN) to communicate. Device drivers can use dedicated tiles for better performance. The hypervisor uses some drivers internally, while most drivers offer simplified device access to a client operating system such as Linux. Hypervisor device drivers run as part of the hypervisor, whereas native Linux drivers run as part of the supervisor.

The client operating system opens drivers using a driver name and calls them using a handle. The hypervisor forwards driver calls through a standard `drv_ops` operations vector `$TILERA_ROOT/doc/html/hv/structdrv__ops.html`, which includes operations to open and close a device, to read and write addresses in the device, and polling and service calls. A device can also support specialized hypervisor fast calls.

The hypervisor runs on all active tiles. A hypervisor driver can run on all tiles, on shared tiles, or on dedicated tiles. The driver can be called from any tile in the client operating system. If there is no need for shared state or atomic operations outside the I/O device, the driver code can simply run on the calling tile.

More complex drivers need to maintain consistency across all tiles. These drivers forward operations to a remote tile that manages a consistent state, with a stub on each tile called by the client. The hypervisor provides RPC versions of driver calls that forward operations over the IDN. IDN messages are limited in size; thus, reads and writes are broken into 256-byte transfers. Atomic operations should be defined to fit in a single transfer.

If a consistent driver state is all that is required, a remote driver tile can be shared with other drivers.

Drivers allocate local memory with `drv_state_zalloc()`. Memory allocated with `drv_state_zalloc()` is not magically shared between tiles. Shared memory can be allocated with `drv_shared_state_alloc()`; many of the new drivers for TILE-Gx shims use shared memory to manage their state.

Drivers with strict performance requirements can use dedicated tiles, which are not shared. Drivers with very high performance requirements might need to dedicate several tiles.

7.2.4 Building Drivers

The file `sys/hv/Makefile` lists all hypervisor sources, including drivers. Adding a source file to the makefile's `HV_SOURCES` variable includes it in the hypervisor. This can be done on the make command line without modifying the makefile by defining the make variable `EXTRA_SOURCES`.

Hypervisor drivers are linked directly into the hypervisor when it is built. Drivers include a `driver_t` structure that describes the driver and its operation, using a special `__DRIVER_ATTR` linker directive. The linker collects a table of all such hypervisor drivers. There is currently no support for loadable hypervisor modules.

7.2.5 Writing Drivers

A new hypervisor driver needs to define the `driver_t` structure with its name, attributes, and a pointer to its `drv_ops` call vector. At the minimum, the operations should include `probe` and possibly `init`, if the driver is to be active.

Open and close manage the handle necessary to read and write device data. For polling, the driver can schedule regular service calls. The Tile Processor implements interrupts as IDN messages. The driver client uses a poll call to request an interrupt from the driver.

It might be easier to modify one of the existing drivers than to start from scratch, particularly a driver for the same I/O device.

The test driver (`sys/hv/drivers/test/`) implements all driver operations.

Memprof is another example of a driver with no associated I/O shim. It uses the service call to poll memory shim statistics.

7.3 Optimizing Device Drivers

Achieving good I/O performance requires optimizing performance at the overall system level. For hypervisor device drivers, this includes:

- executing the driver service code
- using the iMesh™ networks
- managing DMA access to physical memory
- managing interactions between the operating system and application programs

7.3.1 Coding Device Drivers

The Tilera MDE includes several high-performance device drivers written in C. Maintaining and debugging drivers is easier in C than Assembly language.

Optimizations are discussed in the *Multicore Development Environment Optimization Guide* (UG515). Common optimizations used in device drivers are function inlining, loop unrolling, global code motion, and if-conversion. Device drivers use compiler intrinsic functions for processor features, such as special-purpose registers (SPRs), and operations such as the CRC instructions.

Most device drivers repeatedly execute some simple service code that handles each application or device request. High-performance drivers execute a service loop on dedicated tiles, handling one request at a time. Because the sequence is unknown, such loops are not suitable for software pipelining. A service loop can be pipelined across several tiles, with each tile executing a stage of the service and passing work along to the next tile.

A driver service loop executes faster if its code and data fit in cache and it makes good use of the large processor register set. One of the advantages of splitting a service loop across tiles is the increase in available cache and registers. Caches are discussed in *Programming the TILE-Gx Processor* (UG505).

Inlined function calls in a service loop avoid unnecessary conflict misses in the processor's two-way associative level-1 instruction cache (Icache), as long as the loop fits in the 32-kilobyte size. As a compromise, infrequently-called functions can be out-of-line. Initialization and finalization code and unusual error condition handlers, whose performance is not significant, are good candidates for out-of-line functions.

Inlining also encourages data to be kept in registers. The remaining loop might benefit from frequency hints to speed up the most performance-critical cases.

7.3.2 DMA

Ethernet and PCIe drivers transfer data between I/O devices and memory using Direct Memory Access (DMA) implemented by the I/O shims.

DMA performance is limited by memory bandwidth and DMA command overhead. The Tile memory system uses 64-bit DDR3 memory shims. At 1600 million transactions per second, each shim has a theoretical bandwidth of 12.8 GB per second. Actual DRAM bandwidth depends on factors such as on page locality and policy, and can be around 60% of the maximum.

Small DMA transfers are limited more by command overhead than by bandwidth. Each DMA transfer requires a shim request. The shims can hold more than one DMA request, but buffering is limited, so some form of flow control might be necessary.

DDR3 accesses use 64-byte bursts, so DMA requires another burst access each time it crosses a 64-byte alignment boundary. In addition, unaligned writes require a ready-modify-write access. 64 bytes is also the L2 cache line size, so unaligned accesses can waste tile cache.

7.3.3 Operating System and Application

I/O is ultimately driven by application programs running under the operating system supervisor. Applications can distribute work over large sets of tiles, but they are also subject to operating system scheduling and resource contention.

Linux can include system monitoring, memory management, and I/O daemon processes that contend for processor time with I/O applications. These can interfere with application latency by preempting worker processes and polluting caches. The `dataplane` Linux boot argument reserves a set of tiles for exclusive use of application processes. Even with this, the cache footprint of Linux means that application programs will suffer more variation in response time than hypervisor drivers.

As noted above, I/O applications need to manage cache consistency with DMA data. *Programming the TILE-Gx Processor* (UG505) discusses memory management strategies.

7.3.4 Performance Analysis

I/O performance is the result of interactions among all the components mentioned above—drivers, networks, operating system, and applications. The *Multicore Development Environment Optimization Guide* (UG515) describes performance analysis tools. The Tile simulator can capture details of processor and cache states and iMesh network traffic. Unfortunately, the simulator does not include some I/O interfaces and is not capable of running in real time with I/O interfaces.

The Linux OProfile utility uses kernel-level instrumentation on real and simulated systems to capture selected system statistics. `mcstat` and `mpipe-stat` are Linux programs that can collect data on memory and Ethernet performance in running systems.

Device drivers can make simple performance measurements using the processor cycle-count registers available through the `get_cycle_count()` function. Note, however, that the cycle count uses slow SPRs and can take around 5 cycles. The Tile simulator displays writes to the PASS register, and the --trace-cycles option includes timestamps.

Often, the distribution of service times is as important as the mean service time. The `__insn_clz` intrinsic can be used to produce a log-scaled histogram of times, sufficient to identify rare but significant delays.

Varying the CPU clock speed in the hypervisor configuration can identify whether an application is CPU-limited.

`tile-sim` has extensive support for simulating XAUI network interfaces with the `--xaui` option using `pcap` network packet traces. Linux `tcpdump` and `tethereal` programs can capture network traffic for input `pcap` files and display the contents of output `pcap` files. Other publicly-available programs can synthesize packet traces with specific packet types.

7.4 Building a Modified Hypervisor

The hypervisor does not support loadable modules or device drivers. Instead, the MDE distribution includes the full source code for the hypervisor, which you can use to build a new hypervisor with custom extensions.

CHAPTER 8 USING THE BARE METAL ENVIRONMENT (BME)

This chapter describes:

- Section 8.1 Overview of Tilera's BME
- Section 8.2 BME Run-Time Utility Features
- Section 8.3 Starting Up a BME Application
- Section 8.4 Using the bme Command to Boot BME Applications
- Section 8.5 Examples of Configuration Stanzas for BME Applications
- Section 8.6 Using the Run-Time Utility Features of the BME

Note: For information about bare metal debugging, see the chapter on "Advanced Debugging in the IDE" in *Using the Tilera IDE* (UG511).

8.1 Overview of Tilera's BME

Tilera provides a Bare Metal Environment (BME), which is a run-time environment that allows advanced users to run applications that require direct access to the hardware. The user application runs in place of the hypervisor, and essentially acts as the operating system for the tiles on which it runs. You the user are fully responsible for all services provided on the BME tiles.

Note: If you are interested only in performance, you should use Zero Overhead Linux (ZOL) instead of BME. For more information, see Section 3.6 "Enabling Zero Overhead Linux (ZOL) With the dataplane Boot Argument" on page 60.

You specify the virtual and physical memory layout for the application and the set of tiles on which it runs. (There is one address space per tile.) Once started, the BME application has full control of the tiles on which it runs. This allows you to leverage all of the Tilera boot and startup infrastructure, and lets your application coexist on a processor with Linux or hypervisor dedicated driver tiles.

The BME provides a C run-time library and has a set of helper routines that can be linked into the BME application and can assist in performing some common tasks.

The source for the BME is included with the MDE release, in case you want to modify any of it based on your own application requirements.

8.2 BME Run-Time Utility Features

There is no operating system, not even a minimal one, on a BME tile. There is no task model and no scheduler.

However, there are some utility features available as part of the run-time library. Most of these features are independent; a few of them might depend on each other, but in general you can pick and choose the ones you want.

The run-time environment features are:

- Ability to install interrupt vectors.
- Ability to add/remove wired ITLB and DTLB entries.
- Ability to add/remove non-wired memory mappings. These are kept track of in a table in memory, and automatically installed as needed by a supplied data translation lookaside buffer (DTLB) miss handler. The handler allows for user-supplied lookup routines to be installed for various regions of the address space.
- Virtual memory allocator, which can be initialized with the "free memory" data received at application startup.
- Physical memory allocator, which can be initialized with the "free memory" data received at application startup.
- Memory allocated from a fixed-size per-tile memory space (mspace).
- `newlib` mspace routines, which provide a `malloc`-style allocator working from a memory pool provided by the application.
- Various `libc` utilities that do not depend on OS system services (for example, the string routines).
- Ability to access the UDN and IDN.
- UDN communication between Linux processes and BME application tiles.
- Ability to share memory between Linux processes and BME application tiles.
- Ability to retrieve the system configuration data made available to BME tiles at startup. An example is all available I/O shims and who (the BME or hypervisor) owns them.
- I/O device setup (configuration read and write).
- Console output by means of `printf()`, implemented by sending IDN messages to the console driver on a hypervisor tile.

Example code demonstrates how a BME application tile might communicate with an application running on Linux. You can customize this code for your own applications.

For more information, see Section 8.6 "Using the Run-Time Utility Features of the BME" on page 107.

8.3 Starting Up a BME Application

You compile and link a BME application more or less like any normal executable, although some minor changes are required. For example, you must replace the normal C run-time startup files with BME-specific versions. The `-mbme` compiler option takes care of these changes.

You build a BME application into the hypervisor file system, in exactly the same way as you place Linux there for non-BME applications. A command and associated subcommands in the hypervisor configuration file allow you to specify the following:

- Which tiles are part of the BME application
- How much memory you want to allocate to the BME application versus how much you want to reserve for Linux
- The data sharing model for the application, which is one of:
  - "Private," which is most like a set of Linux processes. Each tile has its own private copy of the application's data, and its own stack and mspace, which are not shared with the other tiles.

- "Shared," which is most like a pthreaded program. Each tile shares one copy of the application's data.

Note that even "private" applications are free to set up shared memory areas using the mapping support in the BME run time; the model merely affects what gets set up automatically at application boot time.

- Precise physical and virtual placement of the application's memory segments.

  For example, "On tile 1,1, the stack will be 128K long, starting at physical address 0x12340000 on memory controller 2, and will be mapped starting at virtual address 0xF0000000."

  There are sensible defaults for virtually all of the memory setup options so that you only have to specify details if they are important to your application. Also, a grouping primitive makes it easier to set up multiple tiles in the same way.

The hypervisor places data from the executable file into physical memory as requested by the configuration, and then provides virtual address to physical address mappings in the translation lookaside buffer (TLB) as needed to create the requested virtual address layout.

The hypervisor also passes state data to the BME tiles, so that they can determine:

- Which tiles the BME application is running on and the current tile (each tile knows its own tile number)
- Where each tile's memory segments are, physically and virtually
- What memory is still available
- Which I/O devices have been found in the system, what their properties are, and which ones are already being driven by hypervisor drivers
- The contents of an application command line specified in the `.hvc`.

In addition, a small bit of memory is provided for the purpose of shared-memory locks between the BME tiles, to assist in bootstrapping communication between them.

Once running, the BME tiles are at protection level 2 (PL2): all of the minimum protection levels are set to PL2, so the tiles have the ability to use and reconfigure every bit of their own hardware. (PL3 is used for debugger support.) All of the normal hardware setup done by the hypervisor has been done. (For example, the dynamic networks are configured and interrupts are disabled.)

Mappings for the application's memory segments are wired in the TLBs, and the other TLB entries are available for application use. The I/O devices that are not being driven by the hypervisor are also fully available to the BME application. All of the memory allocated for the BME application can be used however the application desires.

8.4 Using the bme Command to Boot BME Applications

To reserve a set of tiles for a BME application, you need to add a new stanza called `bme` to the hypervisor configuration file. The `bme` stanza is similar to the `client` stanza in that it specifies the tiles to be used and the executable program to be run.

In contrast to the `client` stanza, with the `bme` stanza, various subcommands give you extensive control over exactly where various bits of the application's text and data will be physically located, cached, and mapped into each tile's virtual address space.

The `bme` command defines an application running under the BME.

8.4.1 Syntax

`bme` *`exe-name`* { `private` | `shared` } *`tilespec`* [`tilespec ...`]

8.4.2 Arguments

exe-name

The TILE Elf executable file to run as the BME application. This must be a file within the hypervisor file system.

{ private | shared }

This keyword defines the treatment of each tile's data, stack, and memory space areas. One of these must be specified.

If `private` is specified, each tile will get a separate copy of the application's data, and each tile's stack and memory space will be private: allocated at an identical virtual address, and not mapped into other tiles' virtual address space.

If `shared` is specified, each tile will share one copy of the application's data. Each tile will have its own stack and memory space, but they will be mapped at a unique virtual address, and will be mapped into all tiles' virtual address spaces.

See the subcommands below for details on how various regions are placed and mapped.

tilespec is one of:

x,y

One tile.

wxh

A rectangle of tiles *w* wide by *h* high whose upper-lefthand corner is (0,0).

wxh@x,y

A rectangle of tiles *w* wide by *h* high whose upper-lefthand corner is (*x*,*y*).

cpunum

Tile number *cpunum*, using Linux's CPU number-mapping scheme.

cpunum0-cpunum1

Tiles ranging from `cpunum0` through `cpunum1`, inclusive.

^tilespec

Subtract the named tiles from those previously specified. If the first `tilespec` is a subtraction, start with the default set of tiles (which contains all tiles on the chip), else start with none.

8.4.3 memory Subcommand

```
memory size-ctl-0 size-ctl-1 [size-ctl-2 [size-ctl-3]]]
```

This defines the amount of memory reserved for the application on each memory controller. The value is specified in bytes. A `k`, `m`, or `g` can be appended, meaning kilobytes, megabytes, or gigabytes, respectively.

If this line is omitted, or if a specific particular controller's value is omitted, or if the value `default` is given for a controller, the application will be given a share on that controller equal to that of all other applications or clients that do not have explicit sizes specified.

A value of zero disables that memory controller for this application.

The hypervisor is placed at the very top of physical memory on controller zero. The remaining physical memory on that controller, and all of the memory on the other controllers, is allocated to BME applications and client supervisors.

Allocations for BME applications start at the beginning of the controller. Thus, if the first BME command specifies that an application gets 1 GB of memory on controller 0, it will get physical addresses `0x0 - 0x3FFFFFFF` on that controller.

8.4.4 args Subcommand

`args args`

This specifies arguments that will be passed to the application.

8.4.5 group Subcommand

`group` *`tilespec`* `[`*`tilespec`* `...]`

This designates a set of tiles that will use the same options for memory placement.

The `tilespec`s are interpreted relative to the bounding rectangle of the set of tiles specified on the `bme` command. No tile can be in more than one group; tiles need not be in any group.

8.4.6 Placement Subcommands

The placement subcommands are:

- `text`
- `rodata`
- `rwdata`
- `pertile`
- `nearest`
- `hfh_tiles`

These subcommands, when entered before any group command, apply to all tiles; when entered after a group command, they apply only to tiles in that group.

Each argument is treated separately; if a within-group subcommand does not specify an argument, then its value for that group is not affected.

Thus, for each group, the value for each placement subcommand argument is:

- The value specified inside the current group, if so specified;
- The value specified before any groups, if so specified; or
- The default value, if not otherwise specified.

Tiles included within a particular BME application but not within any group are only affected by the placement subcommands that appear before any group definitions.

8.4.7 text Subcommand

Syntax

```
text [ ctl={cnum | nearest} ] [ pa={bottom | top | pa | exe} ]
   [ cache={x,y | hash | none | local} ] [ dtlb={read | write | none} ]
```

This specifies the placement of an application's text (that is, its segments that are marked as executable). Typically this option would be given before any grouping commands, so that there is only one copy of the text. You can use multiple text lines to cause the text to be replicated on multiple memory controllers.

Arguments

- `ctl`

  Specifies on which memory controller the text will reside. This is either a controller number or `nearest`, which means the controller set with the nearest subcommand. The default is `nearest`.

- `pa`

  Specifies the physical address at which the text will be placed:

  - `bottom`

    The lowest appropriate unused range of physical addresses on the target controller will be used. This is the default.

  - `top`

    The highest appropriate unused range of physical addresses on the target controller will be used.

  - *`pa`*

    A particular physical address will be used, relative to the target controller.

    If this address is not appropriately aligned with the corresponding virtual address, it might cause more ITLB entries to be used in mapping the text, or might even make it impossible to map the text.

    This option cannot be used if the executable has more than one text segment.

  - `exe`

    The physical address specified in the ELF header for the text segment will be used.

    Typically, this value is not useful unless you have linked your application with a custom linker script. It is by default set to the virtual address of the segment.

    The ELF header address is only 32 bits. This address is interpreted relative to the target controller, not as an absolute system-wide physical address. Items placed by means of this option must begin within the first 4 GB of the target controller.

    If this address is not appropriately aligned with the corresponding virtual address, it might cause more ITLB entries to be used in mapping the text, or might even make it impossible to map the text.

- `cache`

  Specifies how the text will be cached:

  - `x,y`

The text will be cached coherently on the specified tile, which is interpreted relative to the bounding rectangle of the set of tiles specified on the `bme` command.

- `hash`

  The text will be cached coherently on a set of tiles, using the hash-for-home feature.

- `none`

  The text will be uncached.

- `local`

  The text will be cached non-coherently on each tile. This is the default.

8.4.8 rodata Subcommand

Syntax

```
rodata [ ctl={cnum | nearest} ] [ pa={bottom | top | pa | exe} ]
   [ cache={x,y | hash | none | local} ]
```

This specifies the placement of an application's read-only data (that is, its segments that are marked as readable, not writable, and not executable).

Arguments

- `ctl`

  Specifies which memory controller the read-only data will be resident on. The default is `nearest`. See the `text` subcommand for details.

- `pa`

  specifies the physical address at which the read-only data will be placed. The default is `bottom`. See the `text` subcommand for details.

- `cache`

  Specifies how the read-only data will be cached. The default is `local`. See the `text` subcommand for details.

The virtual address at which the read-only data segment is mapped is taken from the executable. The hypervisor will use the minimum number of TLB entries to map the segment in the DTLB.

8.4.9 rwdata Subcommand

Syntax

```
rwdata [ ctl={cnum | nearest} ] [ pa={bottom | top | pa | exe} ]
   [ cache={x,y | hash | none | local} ]
```

Specifies the placement of an application's read-write data (that is, its segments that are marked as readable, writable, and not executable).

Arguments

- `ctl`

  Specifies which memory controller the read-write data will be resident on. The default is `nearest`. See the `text` subcommand for details.

- `pa`

Specifies the physical address at which the read-write data will be placed. The default is `bottom`. See the `text` subcommand for details.

In `private` mode, where each tile has its own copy of the read-write data, this address specifies the starting point for the array of per-tile data areas.

- `cache`

  Specifies how the read-write data will be cached.

  In `private` mode, the default is `local`; in `shared` mode, the default is `hash`, which means hash-for-home. See the `text` subcommand for details.

The virtual address at which the read-write data segment is mapped is taken from the executable. The hypervisor will use the minimum number of TLB entries to map the segment in the DTLB.

8.4.10 pertile Subcommand

Syntax

```
pertile [ ctl={cnum | nearest} ] [ pa={bottom | top | pa | exe} ]
    [ cache={x,y | hash | none | local} ] [ stack=stacksize ]
    [ heap=heapsize ] [ va=va ]
```

Specifies the placement of an application's per-tile data area, which contains the tile's stack and run-time mspace (heap).

The stack is used for automatic variables within C functions, and in some cases for function arguments; the run-time heap is primarily intended to support the dynamic allocation of data by BME run-time routines, but can also be used by the application directly.

Arguments

- `ctl`

  Specifies which memory controller the per-tile data will be resident on. The default is `nearest`. See the `text` subcommand for details.

- `pa`

  Specifies the physical address at which the base of the per-tile region will be placed. This region contains all per-tile data areas for this group. The default is `bottom`. See the `text` subcommand for details.

- `cache`

  Specifies how the per-tile data area will be cached. In `private` mode, the default is `local`; in `shared` mode, the default is `hash`, which means hash-for-home. See the `text` subcommand for details.

- `stacksize`

  Specifies the size of the stack in bytes. A `k`, `m`, or `g` can be appended, meaning kilobytes, megabytes, or gigabytes, respectively. The default is 64k.

  In `private` mode, extra padding might be added by the hypervisor to the total size of the per-tile area so that each is individually mappable by its respective tiles.

- `heapsize`

  Specifies the size of the heap in bytes. A `k`, `m`, or `g` can be appended, meaning kilobytes, megabytes, or gigabytes, respectively. The default is 64k.

- `va`

  Specifies the lowest virtual address of the per-tile data area.

In `shared` mode, where each tile's stack has a unique virtual address, this address specifies the virtual starting point for the array of per-tile data areas. The default is `0xFC000000`. Each tile's heap directly follows its stack.

- `sharemap=rwdata`

  Specifies that the BME should try to share the DTLB mapping for the per-tile data area with that for the read-write data. This is only possible in `private` mode, where it is the default, and has the effect of extending the read-write data segment by the size of the per-tile data.

  `sharemap=none` specifies that the BME will not try to share the DTLB entry.

  If the mapping is shared, the `cache` option is ignored (because the caching mode used by read-write data will be used instead), and the `va` option is ignored (because a virtual address derived from the read-write data virtual address will be used).

8.4.11 nearest Subcommand

Syntax

`nearest ctl=`*`cnum`*

This specifies which memory controller should be considered "nearest" for the purpose of evaluating `ctl` options on placement subcommands. By default, the "nearest" controller is the one that has the lowest mean physical distance to all of the tiles in a group.

Arguments

`ctl` — specifies which memory controller is treated as nearest.

8.4.12 hfh_tiles Subcommand

Syntax

`hfh_tiles` *`tilespec`* `[` *`tilespec`* `... ]`

This designates the set of tiles that will be used to cache data from pages designated as hash-for-home. *`tilespec`* is as described above, and is relative to the bounding rectangle of the set of tiles in the BME application.

By default, all tiles in the BME application are used for hash-for-home. It is legal to specify different sets of hash-for-home tiles in different groups, but in that case, different groups cannot share data mapped as hash-for-home. Note that such inconsistent hash-for-home mapping might also cause problems if coherent I/O is used.

8.4.13 extra Subcommand

`extra —` *`filename`* `[ ctl=`*`cnum`* `] [ pa={`*`pa`* `| bottom | top} ]`

This specifies the location for the contents of a file in the hypervisor file system.

This data will be read from the file system and written to physical memory at the given location, for use by the application. Unlike all of the other segments described here, this does not cause a segment to be mapped into any tile's virtual address space. Each tile must install appropriate mappings locally using the BME run-time interfaces once the application is running.

You can specify more than one `extra` subcommand.

Arguments

- `ctl`

  Specifies which memory controller the extra data will be resident on. The default is `nearest`. See the `text` subcommand for details.

- `pa`

  Specifies the physical address at which the extra data will be placed. The default is `bottom`. See the `text` subcommand for details.

8.5 Examples of Configuration Stanzas for BME Applications

The following sections provide examples of configuration stanzas for BME applications.

8.5.1 Simple Example Using Automatic Allocation Features

This example uses the automatic allocation features to the maximum extent and thus uses a default memory layout:

```
# Run "app" as a BME application on 32 tiles
bme app private 8x4
  # Give app 4 GB of memory total
  memory 1g 1g 1g 1g
  # Command-line arguments for app
  args app --input xgbe/0
  # Put the program text on shim 0
  text ctl=0
  # Give everybody a 128K stack
  pertile stack=128k
```

8.5.2 More Complex Example Using Explicit Shim Assignment

This example uses explicit shim assignment to balance out utilization, because the normal `nearest` allocation would tend to overuse shims 0 and 1:

```
# Run "app2" as a BME application on 48 tiles
bme app2 shared 8x6
  # Give app 4 GB of memory total
  memory 1g 1g 1g 1g
  # Command-line arguments for app
  args app2 --process
  # Put the program text on shim 0.  Put it and everything else at the top
  # so the low pa's are available on all shims; the app will allocate that
  # space itself.
  text ctl=0 pa=top
  rodata pa=top
  rwdata pa=top
  pertile pa=top stack=128K
  # Load some app data into bottom of all 4 shims
  extra app2-data ctl=0 pa=0
  extra app2-data ctl=1 pa=0
  extra app2-data ctl=2 pa=0
  extra app2-data ctl=3 pa=0
  # Define group in upper-lefthand corner
    group 4x3@0,0
    # Allocate per-tile data on mshim 1
    nearest 0
```

```
# Define group in upper-righthand corner
group 4x3@4,0
  # Allocate per-tile data on mshim 0
  nearest 0
# Define group in lower-lefthand corner
group 4x3@0,3
  # Allocate per-tile data on mshim 3
  nearest 0
# Define group in lower-righthand corner; these tiles are running a
# different task within the app, and thus need much more heap
group 4x3@4,3
  # Allocate per-tile data on mshim 2
  nearest 0
  pertile heap=1m
```

8.6 Using the Run-Time Utility Features of the BME

8.6.1 Interrupts/Traps/Exceptions

By default, all interrupts are masked. Each entry in the default interrupt vector table contains a routine that prints a message that the interrupt cannot be handled, and panics.

Some of the features in BME require the use of interrupts. Each of those features comes with default handlers for each of the necessary interrupts. Those handlers are installed upon initialization of the feature.

An example assembly interrupt routine that calls a C interrupt handler is provided. If you prefer to write interrupt handlers in C, you can install this routine.

API for Interrupt Handling

You can install your own interrupt handlers and unmask interrupts. For information, see the *Bare Metal Environment Runtime API Reference* (UG228) in the "Libraries" section of the MDE Document Index.

8.6.2 Instruction Translation Lookaside Buffer (ITLB)

ITLB entries for the BME application's executable code are wired in the table at start time.

There is no default ITLB miss handling. This is sufficient for most applications, because a single tile will not be running multiple processes with different mapping.

You can load the ITLB yourself.

For more information about installing translation table entries (TTEs), see the *Bare Metal Environment Runtime API Reference* (UG228) in the "Libraries" section of the MDE Document Index.

8.6.3 Data Translation Lookaside Buffer (DTLB)

The BME provides a default DTLB handler that is not installed by default. The entire virtual address space (all 4G) is partitioned into 16MB segments, each of which has a corresponding miss handler. When a DTLB miss occurs for a certain address, the search routine for the corresponding address range is used to find the DTLB information. The TLB miss handler uses that information to populate an entry in the DTLB table. This function pointer is assigned when the memory is first mapped, before the memory is made available to the program.

The default DTLB "master" handler provided with the default memory management system is a 2-level miss handler. The default master handler looks at the address that caused the miss, and looks up in its table the search function to use to find the entry to install. If that function does not exist, the miss exception is propagated. A default search function is provided. This search function does a linear search of the 1-level page table that corresponds to that virtual address range.

For information about memory management, see Section 8.6.4 “Memory Management” on page 108.

API for DTLB Miss Handler

You can install and uninstall a DTLB miss handler using the API. For information, see the *Bare Metal Environment Runtime API Reference* (UG228) in the “Libraries“ section of the MDE Document Index.

8.6.4 Memory Management

When the BME starts, the text segment (code) and the stack are mapped, as is a portion of data memory that is mapped on all tiles (for synchronization). None of the rest of memory available to the tiles is mapped.

A basic memory management system is provided, but you can also devise your own memory management system.

The basic memory management system consists of routines for initializing the basic memory management system, allocating virtual address space, allocating physical memory, mapping virtual addresses to physical memory, wiring DTLB entries, and `malloc`ing from an allocated memory area.

Each tile is responsible for knowing what portion of memory belongs to it. This is determined by the application, either intrinsically, or by communication and negotiation between tiles.

Tiles can share memory by communicating mappings and attributes among themselves. Each tile is responsible for ensuring that shared *va* space is the same on each tile.

For information about the default DTLB master handler, see Section 8.6.3 “Data Translation Lookaside Buffer (DTLB)” on page 107.

API for Memory Management

You can use the API for memory management. For additional information, see the *Bare Metal Environment Runtime API Reference* (UG228) in the “Libraries“ section of the MDE Document Index.

Example of Memory Management

The following code provides a basic example of how to use the memory management API.

First the default DTLB miss handler is installed. Then a pool of virtual addresses is created, using an arbitrary address that is known to not be in use by either the executable or the stack and heap. A pool of physical addresses is created from the global free list. Then the DTLB miss handler is told about the virtual-to-physical mappings. After that, the memory is ready for use. In this example, an `mspace` is created using the base virtual address so that `mspace_malloc()` can be used on this memory.

```
#define NUM_PAGES 16
#define PAGE_SIZE 4096
#define BASE_VA 0xd000000
#define BUF_SIZE 256

int
main(int argc, char** argv)
{
  // Set up default DTLB miss handler.
  int err;
  err = bme_default_dtlb_handler_init();
  if (err)
  {
    printf("Could not initialize DTLB handler! exiting.\n");
    return -1;
  }

  // Make a VA pool.
  bme_va_pool_t va_pool;
  VA base_va = BASE_VA;
  err = bme_create_va_pool(base_va, PAGE_SIZE, NUM_PAGES, &va_pool);
  if (err)
  {
    printf("Could not create VA pool. Error = %d\n", err);
    return -1;
  }

  // Make a PA pool, using the free_mem list from the global data struct.
  bme_global_info_t* global_info = bme_map_global_info();
  PA base_pa = global_info->free_mem[0].base_pa;
  if (global_info->free_mem[0].len < (NUM_PAGES * PAGE_SIZE))
  {
    printf("Error: free mem segment is too small for this test\n");
    return -1;
  }

  bme_pa_pool_t pa_pool;
  err = bme_create_pa_pool(base_pa, PAGE_SIZE, NUM_PAGES, &pa_pool);
  if (err)
  {
    printf("Could not create PA pool. Error = %d\n", err);
    return -1;
  }

  // Map all VAs to PAs.
  bme_memory_attr_t attr = { .flags = BME_MEMORY_MAP_FLAGS_WRITEABLE,
                             .cache_mode = BME_CACHE_MODE_LOCAL };
  err = bme_memory_map(base_va, base_pa, PAGE_SIZE, NUM_PAGES, &attr, 0);

  // Memory is now available for use.  Make an mspace.
  mspace msp = create_mspace_with_base((void*)base_va,
                                       NUM_PAGES * PAGE_SIZE, 0);

  // Allocate memory from the mspace.
  unsigned char* buf = mspace_malloc(msp, BUF_SIZE); }
```

APPENDIX A—HYPERVISOR CONSOLE ESCAPE COMMANDS

This appendix describes the console escape commands for the hypervisor.

The hypervisor supports a special console escape sequence, known as the console debug string. The debug string serves as a command prefix; the precise action taken depends on the character typed after the debug string.

By default, the escape sequence consists of a single control-caret character (ASCII 0x1E); this can be changed, or disabled, in the hypervisor configuration file by using `console_debug` with the `options` command. See Section 2.3.1 "The options Command" on page 18 for more information.

Two of the console escape commands are for general use. See Section A.1 "Console Escape Commands for General Use" on page 111.

The other console escape commands are for use only with Multiple Client Support (MCS). See Section A.2 "Console Escape Commands for Use with More than One Client" on page 111.

Note: The Hypervisor Console Escape Commands are not supported when connecting to the console over a USB connection.

A.1 Console Escape Commands for General Use

These (case-sensitive) console escape commands are available for anyone at the console:

`R`

The `R` command resets the Tile Processor, if it was booted from SROM. This reset is immediate—the client supervisors, and any programs running under their control, are not given an opportunity to clean up. Therefore, use of this command is normally appropriate only in situations where a supervisor has already crashed.

`P`

When the hypervisor receives the `P` command, it puts the UART hardware into protocol mode; this allows an external debugger to read and write on-chip registers by means of specially formatted packets sent to the UART.

A.2 Console Escape Commands for Use with More than One Client

These (case-sensitive) console escape commands are available for anyone, but are useful only if you are working with more than one client. (To enable more than one client, see Section 2.8.3 "Miscellaneous Option" on page 30.)

`0-9`

Typing a digit after the sequence connects the console to the client whose number was typed. This only works for clients with single-digit numbers; for clients with numbers greater than 9, see the `c` command below.

Clients are numbered in the order in which they appear in the hypervisor configuration file; the first client is client 0.

`c`

The `c` command prompts for a multi-digit client number; once the user types the return key, the console is connected to that client. The prompt looks like this:

```
Enter client number:
```

You can use the delete and backspace keys to modify the number as it is being typed.

`/-`

The `+` command connects the console to the client whose client number is one greater than that of the currently connected client. If the currently connected client is the last client, the first client is selected. BME clients (which cannot be connected to the console) are skipped.

Similarly, the `-` command connects the console to the client whose client number is 1 less than that of the currently connected client, and wraps around to the last client from the first.

`l`

The `l` command prints a list of available clients and their numbers, noting which client is currently connected to the console. For example:

```
hv: available clients:
   0. 4x4 @ 0,0, 15 tiles
   1. 4x4 @ 4,0, 12 tiles
   2. 4x4 @ 0,4, 15 tiles, console connected
   3. 4x4 @ 4,4, 12 tiles
```

`h`

The `h` command, or any unrecognized command, prints a help message explaining the available commands, as follows:

```
hv: console command help:
   0-9     connect console to client #
   c#<CR>  connect console to client #
   +/-     connect console to next/prev client
   l       list clients
   R       reboot (if booted from SROM)
   P       enter protocol mode
```

A.3 Console Escape Commands Used with Hypervisor Statistics

These (case-sensitive) console escape commands are available if hypervisor statistics were enabled at boot time:

`d`

This command dumps hypervisor statistics for all tiles to the console.

`C`

This command clears hypervisor statistics for all tiles.

For more information on the hypervisor statistics feature, see Section 2.5 “Gathering Hypervisor Statistics” on page 25.

APPENDIX B—MBOOT CLI COMMANDS

This appendix describes the command-line interface (CLI) commands available for the boot loader, `mboot`.

B.1 `ifconfig` Command

The `ifconfig` command configures a network interface.

Note that this command applies only to `mboot`, not to the booted kernel.

Syntax

```
ifconfig [interface] [ip_address] [-mask network_mask]
[-mac mac_address] [-up | -down]
```

Arguments

Table B-1 describes the arguments to the `ifconfig` command.

Table B-1. Arguments to the ifconfig Command

Argument	Description
`interface`	The name of the network interface. This argument is optional.
`ip_address`	The IP address to be assigned to this interface. This argument is optional.
`-mask network_mask`	Sets the IP subnet mask for this interface. The mask is validated to be a well-formed *dotted decimal quad* representation of a mask with: 1-32 *one* bits as the high-order bits 1-32 *zero* bits as the low-order bits Total number of bits: 32 Default: 255.255.255.0 This argument is optional.

Table B-1. Arguments to the ifconfig Command (continued)

Argument	Description
`-mac` *mac-address*	The MAC address, specified in 6-byte hexadecimal format, in upper or lower case. Colons and dashes can separate digits in the usual ways. This argument is optional. The following are some valid examples: • `001195666D18` • `0-11-95-66-6d-18` • `0:11:95:66:6D:18` The following are some invalid examples that would be rejected: • `0::11:95:66:6d:18` • `0-11:95:66:6d:18` • `0:011:95:66:6d:18`
`-up | -down`	These flags cause the interface driver to be activated (`-up`) or shut down (`-down`). This argument is optional.

B.2 `ping` Command

The `ping` command determines wether a given host is reachable.

Syntax

`ping` *host*

Arguments

Table B-2 describes the arguments to the `ping` command.

Table B-2. Arguments to the ping Command

Argument	Description
host	The IP address of the host to be reached. This argument is required.

B.3 `dhcp` Command

The `dhcp` command enables the Dynamic Host Configuration Protocol (DHCP) client when the system is booting. If one network interface is enabled for DHCP, the system runs the DHCP client to lease an IP address at startup. Otherwise, the system assigns the static IP address stored in the SROM for this interface.

Note that this command applies only to `mboot`, not to the booted kernel.

Syntax

```
dhcp interface yes|no
```

Arguments

Table B-3 describes the arguments to the `dhcp` command.

Table B-3. Arguments to the dhcp Command

Argument	Description
interface	The name of the network interface, such as `gbe0` or `eth0`. This argument is required.
`yes | no`	Enables (`yes`) or disables (`no`) the DHCP client the next time the system is powered on. This argument is required.

B.4 `route` Command

The `route` command sets or retrieves the default gateway in the kernel routing table. When no options are supplied, the entire kernel table is displayed. If the add or delete option is supplied, this command sets or deletes the default gateway route. Currently, this command supports only one single route entry.

Syntax

```
route [add|del] [default] [-mask netmask] [-gw gateway] [-dev interface]
```

Arguments

Table B-4 describes the arguments to the `route` command.

Table B-4. Arguments to the route Command

Argument	Description
`add`	Adds a route entry to the route table. This argument is optional.
`del`	Deletes a route entry from the route table. This argument is optional.
`default`	Currently, the `route` command supports only the default route. This argument is optional.
`-mask` *netmask*	When adding a default route, the netmask defaults to `0.0.0.0`. When deleting a default route, the netmask must be specified.
`-gw` *gateway*	Specifies the gateway. This argument is optional.
`-dev` *interface*	Specifies the network interface to be associated with this route.

B.5 `tftp` Command

The `tftp` command downloads either the boot loader, `mboot`, or the OS image from a remote host using the Trivial File Transfer Protocol (TFTP), and stores it to the specified device. The image is always downloaded to the RAM disk first, and then transferred to a specific device if requested in the command line. Transfers are always done in binary mode.

Syntax

```
tftp host remotefile -hd|-sd|-mem [localfile]
```

Arguments

Table B-5 describes the arguments to the `tftp` command.

Table B-5. Arguments to the tftp Command

Argument	Description
host	The IP address of the TFTP server. This argument is required.
remotefile	The name of the file in the default directory of the TFTP server that is to be downloaded. This argument is required.
`-hd`	Indicates to download the kernel image to a hard disk.
`-sd`	Indicates to download the kernel image to a SATA disk.
`-mem`	Indicates to download the kernel image to memory (RAM disk). Files thus downloaded to memory are deleted after a reboot.
localfile	The name of the file to be saved in the local device, if the `-hd` or `-mem` option is specified. This argument is optional.

B.6 `ftp` Command

The `ftp` command works in the same way as the `tftp` command, except that it requires a user name and password, and expects an FTP host.

Syntax

```
ftp host [-u username] [-p passwd] remotefile -hd|-sd|-mem [localfile]
```

Arguments

Table B-6 describes the arguments to the `ftp` command.

Table B-6. Arguments to the ftp Command

Argument	Description
host	The IP address of the FTP server. This argument is required.
-u *username*	The user name that is used to log into the FTP server. This argument is optional.
-p *passwd*	The login password. This argument is optional.
remotefile	The name of the file in the default directory of the FTP server that is to be downloaded. This argument is required.
-hd	Indicates to download the kernel image to a hard disk.
-sd	Indicates to download the kernel image to a SATA disk.
-mem	Indicates to download the kernel image to memory (RAM disk). Files thus downloaded to memory are deleted after a reboot.
localfile	The name of the file to be saved in the local device, if the -hd or -mem option is specified. This argument is optional.

B.7 ls Command

The ls command lists the available files in a specified device. In particular, this command allows you to see the results of the tftp and ftp commands.

Syntax

```
ls [-hd|-sd|-mem]
```

Arguments

Table B-7 describes the arguments to the ls command.

*Table B-7. Arguments to the **ls** Command*

Argument	Description
-hd	Lists files stored on the hard disk. This argument is optional.
-sd	Lists boot files on the SATA disk.
-mem	Lists file stored in memory (RAM disk). This argument is optional.

B.8 rm Command

The rm command deletes the specified file on the specified device. If the specified file exists, it is silently deleted.

Note: Exercise caution when using the `rm` command. The system might not be able to boot by default after a power cycle.

Syntax

```
rm -hd|-mem filename
```

Arguments

Table B-8 describes the arguments to the `rm` command.

Table B-8. Arguments to the rm Command

Argument	Description
`-hd`	Indicates to delete the specified file from the hard disk.
`-mem`	Indicates to delete the specified file from memory (RAM disk).
`filename`	The full name of the file to be deleted. This argument is required.

B.9 `bootparam` Command

The `bootparam` command sets either the parameters that are passed to the boot loader (`mboot`) program or the default device and image name from which the system is to boot. What you enter with `bootparam` is saved in SROM and persists with subsequent `mboot` boots.

Note that using `bootparam` to set the boot parameters, takes any arguments that it does not recognize (that is not `-dev` *`dev`*, `-img` *`image`*, `-host` *`host`*, or `-initrd` *`initrd`*) and uses them as kernel boot arguments, overriding the previous boot arguments. For example, if you accidentally type `bootparam -igm vmlinux`, it will change your kernel boot arguments to `-igm vmlinux`. Be aware that there is no way to clear just the kernel boot arguments; you must clear the configuration with `clearcfg` and start over.

See Section B.14 “showcfg Command” on page 124. Use `showcfg` to see what the current boot parameters are.

Syntax

Some forms of the `bootparam` command are shown below. All arguments are optional:

```
bootparam [argument_string]
bootparam [-dev device_name] [-img image_name]
bootparam [-dev device_name] [-img image_name] [argument_string]
```

Arguments

Table B-9 describes the arguments to the `bootparam` command.

Table B-9. Arguments to the bootparam Command

Argument	Description
argument_string	Arbitrary number of arguments to be passed to the `mboot` program. Note: Even though we describe this as a string argument, you do not need to use quotes except where you would use them when passed directly to linux. For example: `bootparam TLR_SHEPHERD_ARGS="--listen 1024" TLR_NETWORK=xgbe0`
`-dev` *device_name*	The default device from which the system is to boot. The *device_name* must be `hd`, `sd`, `tftp`, or `net`.
`-host` *hostname* | *IP_address*	The name or address of the TFTP server that will be contacted when booting over TFTP.
`-img` *image_name*	The name of the default image file in the default boot device.
`-initrd` *initrd_name*	The name of the default `initrd` file in the default boot device.

B.10 `boot` Command

The `boot` command loads a boot image file using the specified means and then passes control to it.

Syntax

There are two forms of syntax:

```
boot -net

boot -hd|-sd|-mem|-tftp|-net [-host hostname | IP_address] [[-img image_name]
[-initrd initrd_name]
```

Arguments

Table B-10 describes the arguments to the `boot` command.

Table B-10. Arguments to the boot Command

Argument	Description
`-net`	Indicates to download, then load the boot image from a TFTP server. For this argument, all of the boot information (TFTP server address, boot parameter, and boot image) are obtained through DHCP and are configurable in the DHCP server and TFTP server. See "Example of -net Argument" below.
`-hd`	Indicates to load the boot image from the hard disk.
`-sd`	Indicates to load the boot image from the SATA disk.

Table B-10. Arguments to the boot Command (continued)

Argument	Description
`-mem`	Indicates to load the boot image from memory (RAM disk). For this to work, you must have previously placed an image in memory by means of the `tftp` or `ftp` commands.
`-tftp`	Indicates to download, then load the boot image from a TFTP (Trivial File Transfer Protocol) server.
`-host` *`hostname|IP_address`*	Indicates the hostname for the server or disk.
`-img` *`image_name`*	The name of the boot image file to be loaded.
`-initrd` *`initrd_name`*	The name of the `initrd` file to be loaded. This is optional. The `initrd` file must be in CPIO format. It can be compressed, but it must be able to be recognized and uncompressed by the Linux kernel. Typical supported compress format includes xz and gzip. An example is the `initramfs.cpio.xz` file which is shipped in the MDE. Use this option when you build an image without embedding a CPIO initramfs file.

Example of -net Argument

To use the `-net` argument, you need to set the `next-server` and `filename` options in the DHCP server. For example: setting `dhcpd.conf` for the ISC DHCP server:

```
next-server 192.168.0.1;
filename "bootinfo.gbe0";
```

`next-server` is the host from which to download the boot information file and other files for the booting.

`filename` specifies the boot information file, which contains boot information for different MAC addresses. Each boot information set is separated by MAC address entries.

Here is a simple example of a boot information file:

```
macaddr 00:26:DD:0:1:15
enable 1
file vmlinux.122627
# comment line
args cache_hash=allbutstack
args TLR_NETWORK=gbe0
preboot preboot/preboot.sh

macaddr default
args tile_net.cpus=10-30
file vmlinux.123096
```

The entries in the boot information file are:

- `macaddr` (required): MAC address to indicate that the boot information that follows is for this interface with this MAC address. A *default* MAC address indicates that this is for interfaces whose boot information does not exist or is not enabled.

- `enable`: Flag to disable/enable this boot information set. By default, this flag is set to 1.
- `file` (required): Boot image file.
- `args`: Boot parameters for the boot image.
- `preboot`: Shell script that can be executed before loading the boot image file.

The boot information file, shell script, and boot image file are all downloaded through TFTP, so all of these files need to be well configured in the `tftp` path.

Note: If you use a command like `boot -net -host A -img B`, the `-host` and `-img` arguments are ignored.

B.11 `reboot` Command

The `reboot` command initiates a soft reboot of the system.

Syntax

```
reboot
```

Arguments

None.

B.12 `exit` Command

The `exit` command quits the CLI program, and loads the default OS image from the default device specified in the `bootparam` command.

Syntax

```
exit
```

Arguments

None.

B.13 `help` Command

The `help` command displays help on the specified command; if a command is not supplied, displays help on all available commands.

Syntax

```
help [command]
```

Arguments

Table B-11 describes the arguments to the `help` command.

Table B-11. Arguments to the help Command

Argument	Description
command	The command for which to display help text. This argument is optional.

B.14 `showcfg` Command

The `showcfg` command displays all configuration data currently stored in the SROM.

Syntax

```
showcfg
```

Arguments

None.

B.15 `clearcfg` Command

If neither the `boot` nor the `user` option is present, the `clearcfg` command clears all configuration data currently stored in the SROM. Otherwise, it deletes only the `bootparam` or `userparam` sector, respectively.

Syntax

```
clearcfg [boot|user]
```

Arguments

Table B-12 describes the arguments to the `clearcfg` command.

Table B-12. Arguments to the clearcfg Command

Argument	Description
boot	Clear the `bootparam` sector in the SROM. This argument is optional.
user	Clear the `userparam` sector in the SROM. This argument is optional.

INDEX